

## RITARDI, PERDITE E THROUGHPUT NELLE RETI A COMMUTAZIONE DI PACCHETTO

Internet può essere vista come *un'infrastruttura che fornisce servizi alle applicazioni distribuite in esecuzione sui sistemi periferici*. Spostare grandi quantità di dati tra due sistemi istantaneamente e senza perdite non è possibile: le reti limitano il *throughput*, cioè la *quantità di dati al secondo che può essere trasferita tra due sistemi periferici, introducendo ritardi tra questi ultimi e possono anche perdere pacchetti*.

### RITARDO NELLE RETE A COMMUTAZIONE DI PACCHETTO

Un pacchetto parte da un host (la sorgente), passa attraverso una serie di router e conclude il viaggio in un altro host (la destinazione). A ogni tappa, il pacchetto subisce vari tipi di ritardo a ciascun nodo (host o router) del tragitto. Di tali ritardi, i principali sono:

- *ritardo di elaborazione;*
- *ritardo di accodamento;*
- *ritardo di trasmissione;*
- *ritardo di propagazione;*

Complessivamente formano il *ritardo totale di nodo (nodal delay)*. Le prestazioni di molte applicazioni per Internet sono pesantemente influenzate dai ritardi di rete.

Supponiamo di avere due router, A che invia e B che riceve:

### RITARDO DI ELABORAZIONE

Tempo richiesto per esaminare l'intestazione del pacchetto e per determinare dove dirigerlo fa parte del ritardo di elaborazione (processing delay). Questo può anche includere altri fattori, tra i quali il tempo richiesto per controllare errori a livello di bit eventualmente occorsi nel pacchetto durante la trasmissione dal nodo a monte al router A. Nei router ad alta velocità questi ritardi sono solitamente dell'ordine dei microsecondi o inferiori. Dopo l'elaborazione, il router dirige il pacchetto verso la coda che precede il collegamento al router B.

### RITARDO DI ACCODAMENTO

Una volta in coda, il pacchetto subisce un ritardo di accodamento (queuing delay) mentre attende la trasmissione sul collegamento. La lunghezza di tale ritardo per uno specifico pacchetto dipenderà dal numero di pacchetti precedentemente arrivati, accodati e in attesa di trasmissione sullo stesso collegamento. Se la coda è vuota e non è in corso la trasmissione di altri pacchetti, allora il ritardo di accodamento per il nostro pacchetto sarà nullo. Se il traffico è pesante e molti altri pacchetti stanno anch'essi aspettando la trasmissione, il ritardo di accodamento sarà lungo. I ritardi di accodamento possono essere dell'ordine dei microsecondi o dei millisecondi.

### RITARDO DI TRASMISSIONE

I pacchetti, nelle reti a commutazione di pacchetti, sono trasmessi secondo la politica first-come-first-served (il primo che arriva è il primo a essere servito). Il nostro pacchetto può essere trasmesso solo dopo la trasmissione di tutti quelli che lo hanno preceduto nell'arrivo. Il ritardo di trasmissione (transmission delay) è pari a  $L/R$ , dove  $L$  è la

lunghezza del pacchetto e  $R$  bps è la velocità di trasmissione. Questo è il tempo richiesto per trasmettere tutti i bit del pacchetto sul collegamento. Anche i ritardi di trasmissione sono nell'ordine dei microsecondi o dei millisecondi.

### RITARDO DI PROPAGAZIONE

Una volta immesso sul collegamento, un bit deve propagarsi fino al router B. Il tempo impiegato è il ritardo di propagazione (propagation delay). Il bit viaggia alla velocità di propagazione del collegamento, che dipende dal mezzo fisico (fibra ottica, doppino in rame e così via) ed è compresa nell'intervallo che va dai  $2 \times 10^8$  m/s ai  $3 \times 10^8$  m/s, (quest'ultimo è la velocità della luce). Il ritardo di propagazione è dato da  $d/v$ , dove  $d$  è la distanza tra i due router, mentre  $v$  è la velocità di propagazione nel collegamento. Nelle reti molto estese i ritardi di propagazione sono dell'ordine dei millisecondi.

### DIFFERENZA TRA RITARDO DI TRASMISSIONE E DI PROPAGAZIONE

Il ritardo di trasmissione è la quantità di tempo richiesta da parte del router per trasmettere in uscita il pacchetto, e dipende dalla lunghezza del pacchetto e dalla velocità di trasmissione del collegamento, non viene influenzato dalla distanza tra i due router. Il ritardo di propagazione, invece, è il tempo richiesto per la propagazione di un bit da un router a quello successivo, e dipende viceversa dalla distanza tra i router senza essere influenzato dalla lunghezza del pacchetto dalla velocità di trasmissione del collegamento.

### RITARDO DI ACCODAMENTO E PERDITA DI PACCHETTI

La componente più complessa e interessante del ritardo totale di nodo è il ritardo di accodamento,  $d_{acc}$ . A differenza degli altri tre ritardi (elaborazione, trasmissione e propagazione), quello di accodamento può variare da pacchetto a pacchetto. Indichiamo con  $a$  la velocità media di arrivo dei pacchetti nella coda, espressa in pacchetti al secondo.  $R$  è la velocità di trasmissione, ossia la velocità (in bit al secondo) alla quale i bit vengono trasmessi in uscita dalla coda. Supponiamo per semplicità che tutti i pacchetti siano lunghi  $L$  bit: la velocità media di arrivo dei bit in coda è di  $La$  bit/s. Il rapporto  $La/R$ , detto intensità di traffico, è molto importante nella stima del ritardo di accodamento. Se  $La/R > 1$ , allora la velocità media di arrivo dei bit nella coda supera la velocità alla quale i bit vengono ritrasmessi in uscita da essa. In questa situazione, la coda tenderà a crescere senza limiti e il ritardo di coda tenderà all'infinito. Pertanto, una delle regole è progettare il sistema in modo che l'intensità di traffico non superi 1. In realtà il ritardo non tende all'infinito perché le code sono limitate. Quando il buffer associato alla coda è pieno ed arriva un pacchetto, esso non può essere memorizzato ed il router lo eliminerà (buffer overflow). Le prestazioni di un nodo si misurano quindi non solo in termini di ritardo ma anche in termini di pacchetti persi.

### THROUGHPUT

Il throughput prevede due tipologie di se stesso: throughput istantaneo e throughput medio. Dati due end point A e B dove A invia un file, il throughput istantaneo in ogni istante di tempo è la velocità (in bps) alla quale B sta ricevendo il file. Se il file consiste di  $F$  bit e il trasferimento richiede  $T$  secondi affinché B riceva tutti gli  $F$  bit, allora il throughput medio del trasferimento del file è di  $F/T$  bps.

## LIVELLI DEI PROTOCOLLI E MODELLI DI SERVIZI

I **protocolli** specificano le modalità di invio e ricezione dei **pacchetti**. I protocolli definiscono il formato, l'ordine e il significato dei messaggi, le azioni da compiere all'atto dell'invio e della ricezione, la dimensione degli spinotti, i materiali usati, ecc. ecc. Internet è una rete a commutazione di pacchetto. I protocolli di Internet vengono raccolti nello stack TCP/IP a 5 livelli: applicazione, trasporto, rete, collegamento, fisico.

## STACK TCP/IP

### LIVELLO APPLICAZIONE

Il livello di applicazione (application layer) è la sede delle applicazioni di rete e dei relativi protocolli. Include molti protocolli, quali HTTP (che consente la richiesta e il trasferimento dei documenti web), SMTP (che consente il trasferimento dei messaggi di posta elettronica) e FTP (che consente il trasferimento di file tra due sistemi remoti). Un protocollo a livello di applicazione è distribuito su più sistemi periferici: un'applicazione in un sistema periferico, tramite il protocollo, scambia pacchetti di informazioni con l'applicazione in un altro sistema periferico.

### LIVELLO DI TRASPORTO

Il livello di trasporto (transport layer) di Internet trasferisce i messaggi del livello di applicazione tra punti periferici gestiti dalle applicazioni. In Internet troviamo due protocolli di trasporto: TCP e UDP. TCP fornisce alle applicazioni un servizio orientato alla connessione, che include la consegna garantita dei messaggi a livello di applicazione alla destinazione e il controllo di flusso (ossia la corrispondenza tra le velocità di mittente e destinatario). Inoltre, TCP fraziona i messaggi lunghi in segmenti più brevi e fornisce un meccanismo di controllo della congestione, in modo che una sorgente regoli la propria velocità trasmissiva quando la rete è congestionata. Il protocollo UDP fornisce alle proprie applicazioni un servizio senza connessione che è davvero un molto semplice, senza affidabilità, né controllo di flusso e della congestione.

### LIVELLO DI RETE

Il livello di rete (network layer) di Internet si occupa di trasferire i pacchetti a livello di rete, detti datagrammi, da un host a un altro. Il protocollo Internet a livello di trasporto (TCP o UDP) in un host di origine passa al livello sottostante un segmento e un indirizzo di destinazione. Il livello di rete mette poi a disposizione il servizio di consegna del segmento al livello di trasporto nell'host di destinazione. Il livello di rete comprende il protocollo IP, che definisce i campi dei datagrammi e come i sistemi periferici e i router agiscono su questi campi. Esiste un solo protocollo IP; tutti gli apparati di Internet che presentano un livello di rete lo devono supportare. Il livello di rete contiene svariati protocolli di instradamento che determinano i percorsi che i datagrammi devono seguire tra la sorgente e la destinazione.

### LIVELLO DI COLLEGAMENTO

Il livello di rete instrada un datagramma attraverso una serie di router tra la sorgente e la destinazione. Per trasferire un pacchetto da un nodo (host o router) a quello successivo sul percorso, il livello di rete si affida ai servizi del livello di collegamento. A ogni nod il livello di rete passa il datagramma al livello sottostante, che lo trasporta al nodo successivo. In questo nodo, il livello di collegamento passa il datagramma al livello di rete superiore. I servizi forniti dal livello di collegamento dipendono dallo specifico protocollo utilizzato: alcuni protocolli garantiscono la consegna

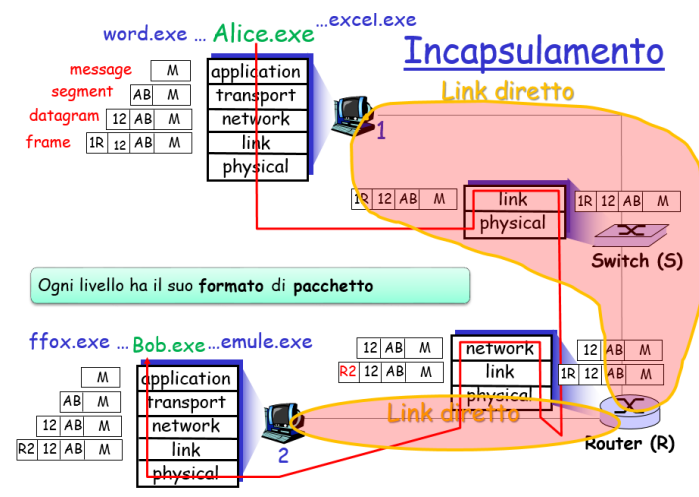
affidabile, ossia dal nodo che trasmette al nodo che riceve su un collegamento. Esempi di livello di collegamento sono Ethernet e Wi-Fi. Dato che i datagrammi in genere devono attraversare diversi collegamenti nel loro viaggio dalla sorgente alla destinazione, un datagramma potrebbe essere gestito da differenti protocolli a livello di collegamento lungo le diverse tratte che costituiscono il suo percorso.

## LIVELLO FISICO

Mentre il compito del livello di collegamento è spostare interi frame da un elemento della rete a quello adiacente, il ruolo del livello fisico (physical layer) è trasferire i singoli bit del frame da un nodo a quello successivo. I protocolli di questo livello sono ancora dipendenti dal collegamento e in più dipendono dall'effettivo mezzo trasmissivo (per esempio, doppino o fibra ottica): Ethernet ad esempio presenta vari protocolli a livello fisico: uno per il doppino intrecciato, uno per il cavo coassiale, uno per la fibra ottica e via dicendo. In ciascuno di tali casi, i bit sono trasferiti lungo il collegamento secondo differenti modalità.

## INCAPSULAMENTO

Presso un host mittente, un messaggio a livello di applicazione (application-layer message) M viene passato a livello di trasporto. Nel caso più semplice, questo livello prende il messaggio e gli concatena informazioni aggiuntive che saranno utilizzate dalla parte ricevente del livello di trasporto. Il messaggio a livello di applicazione e le informazioni di intestazione a livello di trasporto costituiscono il segmento a livello di trasporto (transport-layer segment) che incapsula il messaggio a livello di applicazione. Il livello di trasporto, quindi, passa il segmento al livello di rete, che aggiunge informazioni di intestazione proprie del livello di rete (H), quali gli indirizzi dei sistemi periferici di sorgente e di destinazione, andando così a creare un datagramma a livello di rete (network-layer datagram). A questo punto, il datagramma viene passato al livello di collegamento, il quale aggiunge le proprie informazioni di intestazione creando un frame a livello di collegamento (link-layer frame). Quindi, a ciascun livello, il pacchetto ha due tipi di campi: quello di intestazione e quello di payload (il carico utile trasportato). Il payload è tipicamente un pacchetto proveniente dal livello superiore. Il processo di incapsulamento può essere più complesso rispetto a quello descritto sopra. Per esempio, un messaggio grande può essere diviso in più segmenti a livello di trasporto (i quali possono a loro volta essere divisi ciascuno in più datagrammi a livello di rete). Al momento della ricezione, tale segmento deve essere ricostruito a partire dai datagrammi costitutivi.



## LIVELLO APPLICAZIONE

Il livello applicazione è il livello nel quale viene progettata la comunicazione tra gli end-point finali, e non ci si preoccupa di come avviene la comunicazione ai livelli inferiori. A questo livello i programmatori definiscono quali sono i processi di un programma e li fanno comunicare con altri processi di un differente sistema, ed è dunque il livello sviluppatore. Nel farlo, bisogna valutare la tipologia dell'applicazione di rete che si va a realizzare e il tipo di comunicazione che vogliamo che intercorrano.

NB: il livello applicazione non fa parte del TCP/IP, ma bisogna comunque vedere come le applicazioni si interfacciano con la rete.

## ARCHITETTURE DI RETE

Distinguiamo tre diverse tipologie di *architettura di rete*:

### -CLIENT SERVER:

In questa tipologia di rete, abbiamo un server sempre acceso che fornisce determinati servizi. I vari client ad intermittenza eseguono delle connessioni al server richiedendo i vari servizi. Un client non necessariamente deve avere sempre lo stesso ip e i client normalmente non hanno interazione di rete. I server hanno un indirizzo ip fisso ed un numero di porta accessibile ed aperta che dipende dal tipo di servizio offerto (ogni porta identifica un particolare servizio). Di solito gli ISP (Internet Service Provider, cioè struttura commerciale o organizzazione criminale (criminale non c'è è solo per scherzo) che offre agli utenti servizi internet) hanno delle batterie di server note come DATA CENTER o CONTENT DELIVERY NETWORK (CDN). Anche la chat di FB funziona con un server.

Se digito [www.fb.com](http://www.fb.com) avviene una magia (scusate il tecnicismo) che mi fa connettere al server più vicino e più scarico a disposizione (non comunico sempre con lo stesso IP). Esempio di rete client-server: stiamo scherzando vero? Chi non lo sa per favore lo cerchi oppure cambi facoltà. Grazie.

-PEER TO PEER: Nelle reti peer-to-peer, non esiste un server specifico che offre servizi, ma ciascun host lavora sia come client che come server. Posso vederla come un app web dove ho una costellazione di macchine che partecipa alla messa in piedi di una applicazione. Nelle p2p non servono server: i sistemi comunicano tra di loro direttamente. I peer possono cambiare indirizzo IP, e con i p2p posso condividere le risorse, potrei anche condividere la potenza di calcolo e non solo semplici file. Il p2p non è ben visto dai fornitori di servizi di telecomunicazione, ce ne accorgiamo con abbonamenti con alto download basso upload, perché se avessi upload alti potrei inviare film e robe varie ad alte velocità facendo i miei cloud, e non li vedrei più su infinity ad esempio.

Esempio di peer-to-peer: Delle p2p pure pure pure non ne esistono. La rete Kad è la cosa più vicina a questa architettura, ma anche lì la lista dei peer va messa su un server.

-RETI IBRIDE: Sono reti che a seconda delle operazioni da svolgere, si comportano come CLIENT-SERVER o come P2P (PEER-TO-PEER). Esempi:

Napster/Kad+eDonkey

- Il trasferimento file era p2p;
- La ricerca dei file era centralizzata;
- i peer registravano i nomi dei file su un server centrale;
- i peer interrogavano il server centrale;

Messaggeria istantanea:

- La chat avviene in p2p;
- Il file transfer pure;
- Il rilevamento della presenza on-line è invece centralizzata.

MESSENGER: ha un server centralizzato con una memoria condivisa che dice chi è online, chi no, che ip è connesso. Le chat vanno su server, il trasferimento file in p2p. O almeno tentava, perché spesso le comunicazioni p2p sono messi in difficoltà dai firewall. Con Skype ed altre app molto aggressive sui firewall la cosa si è accentuata di parecchio.

CURIOSITA': Skype è sulla porta 80 perché è quella meno filtrata a livello di firewall.

### ESIGENZE DEL LIVELLO APPLICATIVO

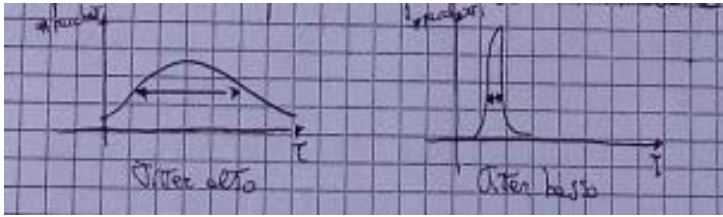
Uno sviluppatore, dopo aver identificato la struttura/architettura di rete, dovrà chiedersi quali sono le principali esigenze della sua applicazione: tempi di trasferimento rapidi, affidabilità della comunicazione, larga banda per la trasmissione di grandi quantità di dati. Questi parametri sono quelli utilizzati per la valutazione della QoS (Quality of Service) di una rete. Le esigenze cambiano a seconda della tipologia di applicazione di rete che si va a realizzare. Si ricorda che la qualità della connessione può dipendere anche dal momento in cui ci si connette, perché magari potrebbe esserci una situazione di congestione (traffico).

**-AFFIDABILITA':** I pacchetti trasmessi all'interno di un'applicazione, si possono perdere per vari motivi: buffer di ricezione saturo, cicli nella rete, congestione... L'affidabilità di una comunicazione rappresenta il tasso di potenziale perdita di pacchetti all'interno di un'applicazione di rete. Un trasferimento dati affidabile prevede un controllo sulla perdita dei pacchetti e prevede quindi la possibilità di recuperare queste perdite.

**-BANDA:** La banda rappresenta il flusso massimo espresso in bit/sec che può viaggiare attraverso una rete (cioè da un punto all'altro), e dipende dal canale di interconnessione migliore che un pacchetto ha incontrato nel cammino da un host ad un altro. La banda è dunque un fattore variabile da invio ad invio perché dipende dal cammino che intraprende ogni pacchetto, ed inoltre il flusso A->B può essere diverso dal flusso B->A.

**-LATENZA:** La latenza si definisce come ritardo di trasmissione ed è un fattore che dipende principalmente dalla distanza tra mittente e ricevente, oltre che da altri fattori. La latenza è molto difficile da misurare in quanto rappresenta **il tempo che un singolo bit (o meglio, un singolo pacchetto) impiega a circolare da A a B (cioè a percorrere tutto il link)**. (Importantissimo! I ANNI SULLA LATENZA E' ROMPI SCATOLE, VUOLE SAPERLA COME DIO COMANDA), dipende da molti fattori e può essere vista come la somma di questi fattori. Può essere stimata calcolando il tempo di andata e ritorno A->B->A e dividendo grezzamente (il grezzamente importantissimo) per due. Il tempo ottenuto è noto come *Round Trip Time (RTT)*.

**-JITTER:** rappresenta la probabilità che due pacchetti arrivino a destinazione in disordine, e non c'è un modo preciso di calcolarlo. Più questo valore è alto, maggiore è la probabilità che i pacchetti di un frame vengano ricevuti in ordine sparso.



Jitter alto e basso. Sull'asse x abbiamo il tempo, sull'asse y i pacchetti.

**Latenza e banda** sono due fattori differenti ma si influenzano a vicenda:

-ALTA LATENZA → DIMINUIZIONE BANDA: se due host sono separati da centinaia di Km di distanza, la latenza della connessione tra di essi risulterà essere molto alta. Anche se il canale avesse una buona banda, questa sarebbe influenzata dal ritardo nell'inviare i pacchetti.

-BASSA BANDA → AUMENTO DI LATENZA : su un canale di banda ristretta passano meno informazioni rispetto a quante ne passano su un canale di banda larga, e questo inevitabilmente causa un aumento di latenza.

Il livello applicativo sfrutta il LIVELLO DI TRASPORTO inferiore per garantire la comunicazione dei messaggi tra end-point. Il livello di trasporto fornisce varie funzionalità al livello applicazione dipendenti dai fattori QoS. In particolare, internet fornisce due protocolli a livello di trasporto che forniscono servizi al livello applicativo: **TCP** (Transfer Control Protocol) e **UDP** (User Datagram Protocol). Quando si va a sviluppare un'applicazione di rete, bisogna decidere quale protocollo utilizzare tra TCP e UDP.

## SERVIZI TCP

TCP è un protocollo orientato agli oggetti...no scherzo, è un protocollo orientato alla connessione e fornisce un trasferimento affidabile di dati. Le sue tre caratteristiche principali sono:

- ORIENTAMENTO ALLA CONNESSIONE: prima di cominciare l'invio di messaggi a livello applicativo, mittente e ricevente si inviano dei messaggi di sincronizzazione con i quali si mette in ascolto tra di loro mediante una procedura nota come **3-WAY-HANDSHAKING** (anche questa cosa è importantissima ed è una cosa alla quale IANNI TIENE PARTICOLARMENTE).

- TRASFERIMENTO AFFIDABILE: implementa un meccanismo di detecting dei loss packet (rilevamento dei pacchetti persi, ma in inglese è più figo o raul che dir si voglia) che consente di mantenere un trasferimento quanto più affidabile possibile. Il protocollo TCP prevede un certo limite di ritrasmissione dei segmenti, raggiunto il quale solleva un errore a livello applicazione. Garantisce inoltre l'ordine di arrivo corretto dei pacchetti. RICORDA: In TCP, ogni pacchetto perso lo pago in latenza. Ad esempio, essendo garantito l'ordine di arrivo dei pacchetti, se perdo il pacchetto uno, il pacchetto due non può essere mandato fin quando non viene mandato uno. Perciò l'affidabilità si paga in banda e latenza.

- CONTROLLO DELLA CONGESTIONE: Il TCP prevede un meccanismo di controllo della congestione tale per cui in presenza di vie di comunicazione con banda differente, vi è una sorta di accordo sulla velocità di trasmissione ideale che permette di non intasare le reti con bande minori.

Queste funzionalità fanno del protocollo TCP un protocollo estremamente affidabile.

## REGOLE DI TCP :

- Ogni host possiede più interfacce di rete (wi-fi, 3g, 4g ecc).  
Se passo da wifi a 3g i download si fermano e poi ripartono, perché il server stava parlando ad un ip ma io l'ho cambiato, quindi deve ripartire ed iniziare la comunicazione con il nuovo indirizzo ip.
- Ogni interfaccia è associata ad un diverso indirizzo Ip nella forma 160.97.47.24 (quattro byte separati da punti)
- Ogni interfaccia possiede 65535 "porte" di ingresso
- Le porte sono numerini che identificano tubicini di arrivo.
- Spesso ad un indirizzo IP è associato un nome testuale (FQPN) (es. [www.mat.unical.it](http://www.mat.unical.it))

## SERVIZI UDP

Il protocollo UDP è un protocollo senza connessione e privo di trasferimento affidabile. E' estremamente leggero in quanto non prevede alcuna sincronizzazione e alcun meccanismo di detecting di pacchetti persi. Quando un segmento UDP non raggiunge per qualsiasi motivo il ricevente, esso viene definitivamente perso. Inoltre, il protocollo UDP non garantisce l'ordine di arrivo dei pacchetti. UDP resta però un protocollo particolarmente veloce grazie alla sua leggerezza ed alla sua semplicità. Ad esempio, la sola connessione prevede l'invio di tre pacchetti, che può essere visto come latenza x3. Il protocollo UDP permette di impostare la quantità a cui "sparare": se si imposta 1 Gigabit e la rete lo supporta, allora sparerò ad un GigaBit, ma nel caso in cui la rete non supportasse tale limite, ci sarebbero perdite enormi. Si potrebbe dunque sparare sempre alla massima velocità consentita, ma di quello che sparo potrebbe tornare indietro la metà (se non di meno!). UDP può essere usato nelle conversazioni via telefono e nelle videochiamate, e ci rendiamo conto di questo quando effettuiamo una telefonata: le conversazioni sono pacchettizzate ed ogni tanto se non va bene la linea capita di sentire a "pezzi": questo perché se viene perso un pacchetto non si può più recuperare. Perciò UDP viene utilizzato quando si vuole una bassa latenza e si accetta il perdere pacchetti. Compra anche tu UDP. UDP: STAY FAST, STAY SCLERANN.

## SCELTA DEL PROTOCOLLO DA UTILIZZARE

Un programmatore dovrà scegliere, sulla base delle varie funzionalità offerte, a quale protocollo a livello di trasporto affidarsi per il genere di comunicazione che intende mettere in atto.

| APPLICAZIONE          | AFFIDABILITA'  | BANDA             | LATENZA           | PROTOCOLLO TRASPORTO |
|-----------------------|----------------|-------------------|-------------------|----------------------|
| Trasferimento file    | <u>Massima</u> | <u>Elastica</u>   | <u>No</u>         | <u>TCP</u>           |
| E-mail                | <u>Massima</u> | <u>Elastica</u>   | <u>NO</u>         | <u>TCP</u>           |
| Web                   | <u>Massima</u> | <u>Elastica</u>   | <u>NO</u>         | <u>TCP</u>           |
| Audio/Video real time | <u>Media</u>   | <u>Alta</u>       | <u>Bassa</u>      | <u>UDP</u>           |
| Giochi interattivi    | <u>Media</u>   | <u>Pochissima</u> | <u>Bassissima</u> | <u>UDP</u>           |
| Messaggistica         | <u>Massima</u> | <u>Elastica</u>   | <u>Media</u>      | <u>TCP</u>           |



## PROCESSI E SOCKET

A livello applicativo la comunicazione tra due programmi situati su due sistemi differenti avviene tra *processi*. I processi comunicano tra di loro scambiandosi dei messaggi in un formato specificato dal tipo di protocollo applicativo che si utilizza. Esistono due tipologie di processi, **processi client** e **processi server**. Il nome non dipende da dove risiede il processo, ma è da contestualizzare all'interno di una sessione di comunicazione.

**All'interno di una sessione di comunicazione**, il *processo client* è un particolare processo che inizializza la comunicazione, mentre il *processo server* è il processo che resta in attesa di essere contattato.

NB: Non è sempre così, a volte il chiamante vuole darti servizi.

ROLL BACK: il client chiede e chiude la connessione, poi il server apre la connessione e chiama il client.

Per poter ricevere ed inviare messaggi verso e da una rete, un processo ha bisogno di una specifica interfaccia che prende il nome di **socket**. Questo perché i processi necessitano di parlare tra loro anche sulla stessa macchina, ma i metodi usati per queste comunicazioni sono poco portabili, perciò devo utilizzare uno standard per tutti che è il socket.

Un **socket** è un'interfaccia software che fa da mediatrice tra il livello di applicazione e quello di trasporto. Funge da ingresso/uscita verso il mondo esterno, e l'invio dei dati passanti tramite essa viene gestito dal livello trasporto. Ogni processo può aprire socket in ascolto su delle determinate porte. Ciascun processo client può aprire socket di connessione sui processi server identificandoli attraverso **ip dell'host/server e numero di porta** nel seguente formato: **"INDIRIZZO IP : NUM PORTA" (192.168.152.3 : 53) .**

Ogni host può avere da 1 a 65.534 socket in ascolto su porte diverse, ed ogni host server può stabilire più connessioni diverse sulla stessa porta.

IP1: PORTA1 ↔ IP2:PORTA2

IP1: PORTA1 ↔ IP3:PORTA3

Non è possibile invece che un server apra più di un socket in ascolto sulla stessa porta.

## PROGRAMMAZIONE DI UN SOCKET TCP

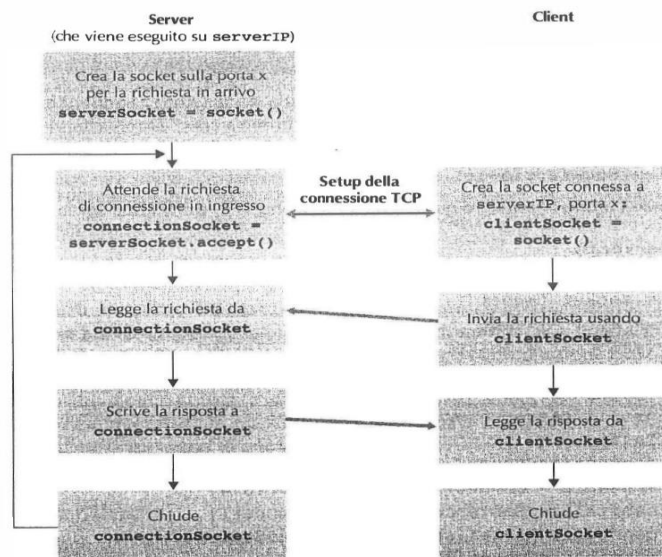
Possiamo quindi distinguere tra due diversi tipi di socket:

**-SOCKET DI ASCOLTO:** Sono attaccati da un lato ma dall'altra parte non è collegato nulla, sono in attesa che qualcuno arrivi a chiedere una connessione. Identificato dalla coppia indirizzo ip – porta (come già specificato ne ho da 1 a 65534).

**-SOCKET CONNESSO:** Arriva un client chiede di attaccarsi al socket e nasce una connessione, la connessione va a buon fine e diventa ESTABLISHED, e il socket connesso è identificato da 4 coordinate: ip del server, porta del server, ip del client, porta del client

**Esempio:** 160.97.47.1:4111<->160.97.7.13:53

E' grazie a questo meccanismo che posso avere più connessioni sulla stessa porta in contemporanea.



La programmazione di un socket TCP.

## QUALCHE COMANDO PER LE CONVERSAZIONI ED I SOCKET

Utilizzando da linea di comando “**netstat**” posso monitorare le varie conversazioni che avvengono a livello trasporto.

**netstat -n** : sono le conversazioni ip aperte al momento.

Su linux con **watch nestat -n** , il comando viene lanciato ogni due secondi, così è possibile tenere d'occhio le conversazioni aperte.

NB: **watch** si può lanciare su qualunque cosa.

Se vedo **wait** ecc, sono stati finali di chiusura della connessione, quindi non posso parlarci dentro ne posso aspettarmi che qualcuno mi parli dall'altra parte.

**netstat -a** : vedo tutte le connessioni, anche i socket in ascolto.

Se vedo 0.0.0.0, vuol dire che il socket ascolta su tutte le interfacce, se ho un ip ascolta solo un indirizzo che è quello che si legge.

Se voglio sapere che processo ho attaccato, uso **netstat -b** in windows, **netstat -p** in linux. Questa cosa può farla solo solo l'amministratore.

Dando il comando **ls -l**, se vedo s vicino ad un elemento, vuol dire che quell'elemento è un socket.

Esiste una app per windows, tcp view, dove posso vedere anche tutti i pacchetti ricevuti ed inviati per ciascun socket.

## PROTOCOLLI A LIVELLO APPLICAZIONE

Un protocollo a livello di applicazione definisce come i processi di un'applicazione, in esecuzione su sistemi periferici diversi, si scambiano i messaggi. In particolare, un protocollo a livello di applicazione definisce:

- i tipi di messaggi scambiati (per esempio, di richiesta o di risposta);
- la sintassi dei vari tipi di messaggio (per esempio, quali sono i campi nel messaggio e come vengono descritti);
- la semantica dei campi, ossia il significato delle informazioni che contengono;
- le regole per determinare quando e come un processo invia e risponde ai messaggi.

È importante distinguere tra applicazioni di rete e protocolli a livello di applicazione.

Un protocollo a livello di applicazione è solo una parte (molto importante) di un'applicazione di rete. Ad esempio, il web è un'applicazione client server che consente agli utenti di ottenere su richiesta documenti dai web server. L'applicazione web consiste di molte componenti, tra cui uno standard per i formati di documento (HTML), browser (per esempio: Firefox), web server (per esempio: Apache, Microsoft IIS) e un protocollo a livello di applicazione, HTTP. Il protocollo HTTP definisce il formato e la sequenza dei messaggi scambiati tra browser e web server. Perciò è solo una parte dell'applicazione web.

## PROTOCOLLO SMTP (SINGLE MAIL TRANSFER PROTOCOL)

Il protocollo SMTP gestisce il trasferimento delle mail all'interno dell'infrastruttura di posta elettronica. Essa contiene diverse componenti:

- USER AGENT: sono le applicazioni che interagiscono con l'utente offrendo il servizio di posta (app mail)
- MAIL SERVER: sono i server dove risiedono le mail per un determinato dominio di posta elettronica. Ogni mail server contiene svariate *mail box*, ovvero cartelle con le mail ricevute per ogni cliente, ed un *buffer di uscita* contenente le mail pronte per essere spedite verso l'esterno. Ogni mail server è competente per un determinato dominio di posta elettronica (uno per libero.it, uno per yahoo ecc).

NB; Ci sono mail server che non sono competenti, e ci sono casi in cui abbiamo gruppi di mail server (come in gmail)

All'interno dell'infrastruttura i mail server comunicano tra di loro per il trasferimento delle mail. Quando il mittente scrive una mail, lo user agent invia questa mail al mail server di competenza, che corrisponde a quello con il dominio utilizzato per l'invio della mail dallo stesso user agent. Il mail server mittente smisterà direttamente la mail al server mail del ricevente che lo inserisce nel suo buffer di uscita. Qui il mail server lo smista nella mail box dell'utente corretto. La comunicazione tra i server è regolata dal protocollo SMTP. L'infrastruttura dei mail server è un'architettura P2P in quanto ogni mail server può funzionare da client o da server a seconda che debba inviare o ricevere delle mail.

SMTP utilizza una connessione TCP per il trasferimento delle mail, e questo lo rende un protocollo altamente affidabile. I numeri di porta usati per le comunicazioni SMTP sono:

- 25 (SMTP normale);
- 465 (SMTP con crittografia);
- 587 (SMTP con crittografia);

### TRASFERIMENTO DI UNA MAIL A-> B

- 1) Lo user agent di A invia la mail al mail server di competenza nella sua coda di uscita;
- 2) Il mail server mittente apre una connessione TCP sulla porta 25 (oppure 465 o 587 che usano la crittografia, anche se la 465 è deprecata ma ancora molto usata) con il mail server del ricevente. Per trovare il ricevente, viene interrogato il dominio di posta elettronica del ricevente nell'intestazione della mail e viene fatta una richiesta DNS (il protocollo è MX, Mail eXchanger). Questo perché per parlare sul web ho bisogno di un indirizzo ip, quindi ho una conversazione dns che serve alla traduzione in ip, ma è un po diversa da quella dei siti web: in input ho un dominio di posta e voglio avere l'ip del mail exchanger (MX) (dominio di competenza).
- 3) Il mail server A funge da client SMTP e quello B da server SMTP. Tra i due avviene una sincronizzazione prima del trasferimento della mail, tramite un handshaking SMTP.
- 4) Il client SMTP invia il messaggio al server SMTP attraverso il socket TCP.
- 5) Il processo server SMTP sul server B riceve il messaggio e lo ripone nella mail box di B.
- 6) Quando B lo ritiene opportuno, accende il proprio user agent e controlla la posta in arrivo, visualizzando così la mail di A.

Cosa importante da notare, è che la conversazione SMTP avviene tra due soli mail server senza alcun intermediario, è questo avverrebbe anche si trovassero a centinaia di KM di distanza.

### FASI DELLA COMUNICAZIONE SMTP

La comunicazione SMTP può quindi essere suddivisa in tre fasi:

- HANDSHAKING SMTP: il client SMTP ed il server SMTP si scambiano alcuni messaggi preliminari che servono alla sincronizzazione;

- TRASFERIMENTO MAIL;

- CHIUSURA DELLA CONNESSIONE.

Ad ogni messaggio inviato dal client SMTP, il server risponde con un codice identificativo della risposta e qualche stringa descrittiva. E' possibile inviare una mail con il protocollo SMTP utilizzando da terminale il comando **telnet**. Telnet è un comando, un servizio web accessibile sulla porta 20, che può essere usato per aprire un socket su un server sulla porta che voglio io.

ESEMPIO:

*telnet smtp.libero.it 25*

### **FASE 1: HANDSHAKING**

S: 220 smtp.libero.it // ricorda: se invece di smtp ho esmtp è una versione estesa di smtp.  
C: HELO hotmail.it //helo serve a salutare  
S: 250 HELLO hotmail.it, pleased to meet you  
C: MAIL FROM: <vic@hotmail.it>  
S: 250 vic@hotmail.it ... SENDER OK.  
C: RCPT TO: <qualcunoacaso@libero.it>  
S: 250 qualcunoacaso@libero.it ... RECIPIENT OK

### **FASE 2: TRASFERIMENTO**

C: DATA  
S: 354 Enter mail, end with "." on a line by itself  
C: Ciao  
C: Come stai?  
C: .  
S: 250 Message accepted for delivery

### **FASE 3: CHIUSURA CONNESSIONE**

C: QUIT  
S: 221 closing connection

CODICI:

- 220: connessione stabilita;
- 250: conferma;
- 354: inserimento messaggio;
- 221: chiusura della connessione.

Ogni mail server tiene una mail nella sua coda fin quando non riesce a processarla oppure non scade un time-out di 4-5 giorni, quindi è molto molto raro che si perda una mail. Essendo SMTP uno dei primi protocolli ad essere progettati, le sue caratteristiche sono semplici e arcaiche (arcaiche lo ha detto il professore proprio, infatti c'è sia sugli appunti di Melissari che sui miei). Infatti SMTP prevede il trasferimento dei messaggi codificandoli internamente in ASCII a 7 bit. Perciò prima dell'invio uno user agent deve trasformare il contenuto di qualsiasi mail (che sia testuale o allegato) in ASCII, e la stessa cosa va fatta in fase di ricezione. Questa cosa potrebbe rappresentare una forte restrizione per il protocollo SMTP.

SMTP pone problemi di sicurezza legati al fatto che non c'è confidenzialità, per questo ci si sta spostando dalla porta 25 alle 587, ma la 25 è ancora aperta ed è facilissimo mandare e-mail anonime se si conosce l'mx. (TECNICISMO DELL'ANNO: mx puttano, riceve e scambia con chiunque, uno molto famoso per questo è l'mx di libero).

CURIOSITA: In SMTP non puoi “spingere” i file zip perché solitamente l'ultimo byte è a uno e quindi va ricodificato.

### FORMATO DEI MESSAGGI SMTP

Un messaggio SMTP è formato da un *header* contenente informazioni formate da una parola chiave seguita da due punti seguiti dal valore. L'header è distaccata dal *body* attraverso una linea vuota. Sia header che body sono testo in ASCII leggibile. Esempi di intestazioni importanti sono:

- FROM: sender@mail.it
- TO: receiver@hotmail.it
- SUBJECT: oggetto

Dato che SMTP prevede messaggi interamente codificati in ASCII, esiste un protocollo che gestisce i tipi di codifica utilizzati per gli allegati, noto come **MIME (Multimedia Mail Extension)**

### PROTOCOLLI DI ACCESSO ALLE MAIL

Il passo 6 del trasferimento di una mail nell'infrastruttura di posta elettronica, ovvero al lettura da parte del destinatario della mail ricevuta, è gestita da protocolli diversi dal protocollo SMTP, detti *protocolli di accesso alle mail*.

#### IL PROTOCOLLO POP 3 (POST OFFICIAL PROTOCOL)

E' un protocollo molto semplice per l'accesso alla mail box utente di un mail server. Entra in gioco durante la connessione TCP tra lo user agent ed il mail server riceventi. Per comunicare il POP3 con il server, lo user agent apre una connessione TCP con esso sulla porta 110. Poi il POP3 prevede 3 passi:

- **AUTENTICAZIONE**: è un'autenticazione molto poco sicura in quanto scambia username e password in chiaro sulla rete;
- **TRASFERIMENTO**: recupera i messaggi in arrivo;
- **AGGIORNAMENTO**: dopo che i messaggi sono stati trasferiti allo user agent POP3 può decidere se cancellarli dalla mail box del server o mantenerli.

ESEMPIO DI POP3:

*telenet POP3server 110*

```
S: +OK POP3 Server Ready
C: User Vincenzo
S: +OK
C: Pass nonteladiromai
S: +OK User successfully logged on
C: list
```

S: 1 498  
S: 2 912  
S: .  
C: retr 1  
S:<.....  
S: .....>  
S: .  
C: del 1  
C: quit  
S: +OK POP3 Server Signing off

I possibili messaggi di risposta da parte di un server POP3 sono:

- **+OK** : comando corretto
- **ERR**: errore

Il client POP3 esegue nell'ordine:

- **list**: lista il contenuto della mailBox;
- **retr x**: recupera la mail numero X;
- **del x**: elimina la mail numero X.

Usando *list-retr-del*, si implementa la modalità *leggi e cancella*. E' anche possibile implementare la modalità *leggi e mantieni*.

### IL PROTOCOLLO CHAP

Nonostante la sua semplicità lo renda leggero e versatile, POP3 ha un grande problema che è quello della sicurezza. Per rendere sicura l'autenticazione su POP3 si usa il protocollo **CHAP (Challenge-Handshake Authentication Protocol)**. Questo protocollo garantisce l'autenticazione al mail server senza effettuare lo scambio delle credenziali. Questo avviene tramite una fase di handshaking che si svolge nel seguente modo:

- 1) Il client POP3 richiede l'autenticazione (conosce la sua password);
- 2) Il server POP3 risponde con una challenge, vale a dire un numero speciale.
- 3) Solo chi conosce la password è in grado di risolvere la challenge, pertanto il client, ricevuta la challenge, applica la sua password e invia il risultato al server.
- 4) Il server riceve il risultato controllando la stringa ricevuta e dà l'accesso al client.

### IL PROTOCOLLO IMAP (INTERNET MAIL ACCESS PROTOCOL)

IMAP è un protocollo di accesso alla mail; è più complesso di POP3 in quanto fornisce funzionalità per creare cartelle e spostare le mail da una cartella all'altra.

### RICAPITOLANDO:

Se devo inviare una mail uso SMTP dopo essermi connesso al mail server, se devo consultare le mail utilizzo IMAP o

POP3.

### PROTOCOLLO DNS (DOMAIN NAME SYSTEM)

**DNS** è un protocollo di livello applicativo e un database distribuito gerarchico. Fornisce un sistema di traduzione di hostname (www.qualchesito.it) a indirizzo ip (162.192.13.1). Il protocollo DNS sfrutta il protocollo **UDP** ed utilizza la porta 53: *in wireshark possiamo quindi filtrare le connessioni dns* selezionando il protocollo UDP e la porta 53.

Il database DNS è distribuito in una ragnatela di numerosi server speciali noti come *DNS server*, predisposti secondo una struttura gerarchica.

### **SCENARIO DI FUNZIONAMENTO DEL DNS**

- 1) Un client richiede un collegamento o una risorsa attraverso un URL o un hostname;
- 2) Il client esegue il lato client di DNS ed estrapola il nome dell'host dall'URL;
- 3) il client invia una richiesta (query) ad un DNS server;
- 4) Il DNS server risponde con l'IP corrispondente a quel nome, così il client può proseguire con la connessione.

### **SERVIZI DNS**

Il protocollo DNS offre i seguenti servizi:

- **traduzione**: converte l'host name ad ip e viceversa;

- **host aliasing**: quando un host ha un nome completo può avere dei sinonimi. DNS fornisce un sistema di traduzione dai sinonimi a quello noto come *canonic name*.

- **mail server aliasing**: il servizio è noto come MX (Mail eXchanger) e consiste nel fornire i nomi dei mail server sotto un determinato dominio.

- **load balancing**: il DNS contribuisce a implementare un meccanismo molto importante per la gestione del traffico, il meccanismo di *bilanciamento di carico*. Quando un particolare sito è sottoposto a intenso traffico, i fornitori scelgono di replicarlo su più server per smistare il traffico entrante. Ciascun server ha un determinato indirizzo IP ma appartengono tutti ad unico dominio (ad esempio cnn.com). Il protocollo DNS fornisce la lista di IP diversi per un determinato host name variando però l'ordine della lista. In questo modo un client che seleziona il primo ip della lista sarà sottoposto ad un meccanismo di rotazione degli IP periodico. Questo permette di bilanciare il traffico in rete smistandolo opportunamente nel CDN (Content Delivery Network, visto in Client-Server).

### **DNS: DATABASE DISTRIBUITO GERARCHICO**

Il protocollo DNS utilizza un sistema di database distribuiti e strutturati in modo gerarchico per motivi di bilanciamento di traffico, di sicurezza, di distanza e di manutenzione. Si possono identificare tre livelli, cioè tre classi di DNS server:

- **Root server**: sono all'apice della gerarchia; ne esistono 13 data center in tutto Internet, per un totale di 250 server circa;
- **TLD (Top Level Domain) server**: sono server di primo livello e rappresentano domini quali .org, .com, .net,



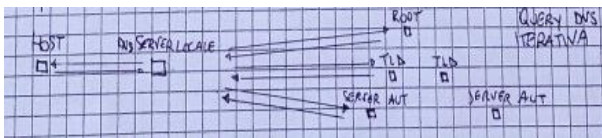
- .it ecc;
- **DNS Server Autoritativo:** rappresenta il DNS server per un determinato dominio pubblico e gestisce tutti i suoi sottodomini.

A questa gerarchia va aggiunto un altro DNS server noto come **DNS Server Locale** che funge da “resolver”, cioè da proxy, da intermediario tra il client e la gerarchia di DNS server.

## LE CONVERSAZIONI DNS

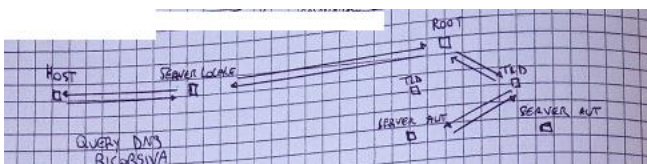
La conversazione DNS può avvenire in due diversi modi: *query iterative* e *query ricorsive*.

-**query iterative:** il client DNS invia la richiesta con l'hostname per intero al server locale; questo contatta per primo il root server che analizza l'URL, estrae il dominio di primo livello e restituisce gli IP dei TLD relativi a quel dominio. Il server locale interroga ora questi IP TLD che analizza l'host name e restituiscono l'IP i gli IP del server autoritativo per quel dominio. Il server locale invia ora la richiesta al server autoritativo che conoscendo i suoi sottodomini può rispondere con l'IP del sottodominio.



Query DNS iterativa

-**query ricorsive:** il server locale interroga il root server, ma la conversazione prosegue all'interno della gerarchia tra root → TLD → autoritativo fino a raggiungere la base; una volta recapitato l'IP, si risale il cammino a retroso fino al root server che smista la response al server locale che l'aveva richiesta.



Query DNS ricorsiva

## DNS CACHING

Una richiesta DNS prevede un buon numero di richieste, cosa che potrebbe appesantire di molto il traffico di rete. Viene quindi introdotto un meccanismo di caching, detto DNS caching, grazie al quale un server DNS che riceve una qualche risposta DNS da un altro server DNS può memorizzare in cache questa informazione, in modo che alla successiva richiesta potrà rispondere direttamente senza dover interagire con altri server. Queste informazioni sono sottoposte ad invalidazione per un periodo di 2 giorni per mantenerle aggiornate.

## RECORD DNS

I server DNS memorizzano record di risorse contenenti i seguenti campi:

<Name, Valu, Type, TTL>

Name e Value dipendono dal tipo e sono le associazioni valore chiave.

- Type = A : record che associa ad un host name il corrispondente IP;

- Type = NS : record che associa ad un dominio il server autoritativo;

- Type = CNAME : record che associa ad un dominio di IP il valore canonico;

- Type = MX : record che associa ad un dominio il server mail per quel determinato dominio.

### GIOCARE CON IL DNS: NSLOOKUP

Da shell con il comando *nslookup* posso consultare i dns. Una volta dato il comando:

Il simbolo ">" indica che sono in nslookup quindi devo parlare l'nslookuppese.

Utilizzando:

> www.mat.unical.it

Ottengo server ed indirizzo del sito specificato.

RICORDA: il server 8.8.8.8 è il server che mi ha dato la risposta, non la risposta stessa. Infatti 8.8.8.8 è l'indirizzo del dns di google.

Se provo a tradurre un sito molto grande:

> www.google.com

Non esce due volte lo stesso ip.

Il comando punto invio chiude la richiesta.

250 è il codice di tutto ok!

Poi mi dice il codice con cui è accaduto il messaggio.

Se ho finito faccio QUIT ed il server chiude il socket.

Ci sono diversi tipi di query che posso fare con *nslookup*:

- **Query di tipo A**: in ingresso il nome in uscita l'ip;
- **PTR** : do l'ip e in output ottengo il nome testuale;
- **MX** : mi dà l'mx;

*set type=* è il comando che permette di scegliere la query da eseguire.

Es:

`set type=MX`

Ora in input non devo dare il nome di server ma il dominio della mail (es google.com) ed ottengo una lista che sono i vari mx ordinati per priorità e l'ultimo è il primario.

Esistono anche mx farlocchi (se faccio mat unical escono nomi scemi) che servono per lo spam, perché solitamente i bot sono fatti in modo da prendere in modo casuale uno di quelli, così gli faccio perdere tempo e lo faccio finire sulle blacklist che sono consultabili pubblicamente.

**set type=NS** : Posso chiedere il server dns autoritativo per 1 determinato dominio. Ogni dns che ha la competenza su un dominio ha un suo database ed i nameserver sono i dns autoritari.

**set type=CNAME**: Mi da il nome canonico.

**exit** : Esce da nslookup.

**I SET TYPE PERMETTONO DI PRENDERE IL TIPO DI RECORD DNS SPECIFICATO.**

CONSIGLIO: NSLOOKUP POTREBBE ESSERE UNA TIPICA DOMANDA DI ESAME. INVITO LA BANDA BASSOTTI A SMANETTARE DA TERMINALE CON IL SUDDETTO PER EVITARE BRUTTE SOPRESE.

### PROTOCOLLO HTTP (HYPERTEXT TRANSFER PROTOCOL)

Http è il protocollo di livello applicazione della comunicazione, che quindi riguarda la comunicazione ed il trasferimento delle pagine web. Una pagina web è un documento costituito da oggetti (file html, immagini ecc). Ogni oggetto è indirizzabile tramite un URL (Uniforme Resource Locator), che rappresenta un indirizzo leggibile. Un URL ha la seguente struttura:

<http://www.unical.it/portale>

*http = protocollo; //www.mat.unical.it= host name; /portale= percorso, path.*

Il percorso rappresenta il path della risorsa localizzata sul server con nome di dominio host name. Per richiedere le pagine web ed i loro oggetti, HTTP utilizza e analizza gli URL degli oggetti. Il protocollo HTTP lavora in un'architettura client server dove il client richiede le risorse ed i web server rispondono con le risorse parlando conversazioni HTTP.

Un **browser web** (come ad esempio Mozilla o Chrome) implementa la componente client dell'HTTP, mentre un **web server** (come ad esempio Apache), che ospita gli oggetti web indirizzabili tramite URL, implementa il lato server.

HTTP utilizza connessioni TCP, di conseguenza è un protocollo altamente affidabile. La porta utilizzata dal protocollo HTTP è la porta 80 (443 per l'HTTP con crittografia), ed il protocollo prevede che siano dei server in ascolto su questa porta che sappiano parlare l'HTTP.

Quando un client HTTP vuole comunicare e scambiare oggetti con un web server *avviene una richiesta detta HTTP request con relativa risposta detta HTTP response, che sono messaggi aventi un proprio formato.* Il protocollo HTTP è

stateless, senza stato, il che significa che su ogni connessione non mantiene informazioni precedenti sulle altre connessioni. Per “imitare” questo meccanismo, si sono creati i cookie ed il *meccanismo della sessione*.

## TIPOLOGIE DI CONNESSIONE

Http presenta due tipologie di connessione:

- HTTP CON CONNESSIONE NON PERSISTENTE (HTTP/1.0): Con questa modalità è possibile trasmettere un oggetto per ogni connessione TCP. Il client invia la HTTP request con la richiesta di qualche risorsa. Il web server risponde tramite HTTP response con la risorsa richiesta e chiude la connessione. Se la risorsa appena ricevuta aveva altri oggetti tipo immagini da scaricare, il client aprirà una nuova connessione per ciascuno degli oggetti.

- HTTP CON CONNESSIONE PERSISTENTE (HTTP/1.1): in questa modalità il web server non chiude la connessione dopo l'invio della prima HTTP response, dando così la possibilità al client di richiedere altre risorse sulla stessa connessione. Esistono due tipi di connessione : *persistente senza pipeling* e *persistente con pipeling*.

Nella *connessione persistente senza pipeling*, il client prima di inviare la request per il prossimo oggetto, attende la response di quella corrente. In termini di RTT abbiamo un RTT per ogni oggetto.

Nella *connessione persistente con pipeling*, che rappresenta la modalità di default, il client invia HTTP request in parallelo senza attendere la response, quindi abbiamo all'incirca un RTT per tutti gli oggetti.

Nell'HTTP 1.1, la connessione TCP viene chiusa dal web server dopo che riscontra un certo periodo di inattività del client.

## I MESSAGGI HTTP

Come già detto, i messaggi HTTP sono di due tipi: request e response.

### HTTP REQUEST MESSAGE

|        |             |               |                   |
|--------|-------------|---------------|-------------------|
| metodo | url risorsa | versione http |                   |
| GET    | /portale/   | HTTP/1.1      | RIGA DI RICHIESTA |

|                         |  |
|-------------------------|--|
| HOST: unical.it         |  |
| Connection: close       |  |
| User-Agent: Mozilla/5.0 |  |
| Accept-language: it     |  |

RIGHE DI INTESTAZIONE

La riga di richiesta è formata da i seguenti campi:

- campo metodo: indica il metodo con cui il client vuole effettuare la request. Ci sono vari metodi possibili, che sono:

- GET (HTTP 1.0 e 1.1): si richiede l'oggetto specificato nell'URL, il cui contenuto viene restituito nel corpo.

-POST (HTTP 1.0 e 1.1): si richiede l'oggetto come con GET. In seguito verrà specificata la differenza tra i due.

-HEAD (HTTP 1.0 e 1.1): si richiede solo l'intestazione della response associata alla request corrente.

-PUT (solo HTTP 1.1): il client vuole trasmettere un oggetto al web server;

-DELETE (solo HTTP 1.1): il client vuole cancellare un oggetto che si trova sul web server.

-campo url: indica l'url della risorsa, il path sul web server;

-campo versione http: comunica al web server la versione di HTTP che il client è in grado di parlare.

Le righe di intestazione possono essere formate da più campi rispetto a quelli specificati. I principali sono:

-host: indica l'host name del web server al quale si vuole spedire la request;

-connection: il browser comunica al server che sta usando un HTTP a connessione non persistente e che pertanto deve chiudere la connessione dopo aver ricevuto questa request.

-user-agent: indica il tipo di browser utilizzato. E' molto importante affinché il web server capisca che tipo di risorsa deve inviare al browser secondo il tipo.

-accept-language: indica la lingua che il client preferisce ricevere dal server.

### HTTP RESPONSE MESSAGE

| Versione http                                           | codice stato | messaggio stato |                       |
|---------------------------------------------------------|--------------|-----------------|-----------------------|
| HTTP/1.1                                                | 200          | OK              | RIGA DI STATO         |
| Connection: close                                       |              |                 | RIGHE DI INTESTAZIONE |
| Date: Thu, 6 Aug 1998 12:00:15 GMT                      |              |                 |                       |
| Server: Apache/1.3.0 (Unix)                             |              |                 |                       |
| Last-Modified: Mon, 22 Jun 1998 13:50:46 GMT            |              |                 |                       |
| Content-Length: 6821                                    |              |                 |                       |
| Content-Type: Text/html                                 |              |                 |                       |
| ← LINEA VUOTA = {CR (Curriage Return) + LF (Line Feed)} |              |                 |                       |
| [DATI DATI DATI...                                      |              |                 | CORPO                 |

Nella response http la riga di stato è composta da:

-campo versione http: è il campo che indica che versione HTTP parla il web server;

-campo codice di stato http: rappresenta il codice di stato. Un codice di stato è identificativo di un determinato stato della risposta:

-200: richiesta effettuata con successo;

-301: indica che la risorsa è stata permanentemente spostata e il browser reindirizza direttamente al percorso nuovo perché viene allegato nella response nel campo LOCATION;

-400: indica che il server non riesce a comprendere la request (*bad request*);

-404: risorsa non presente nel server (*not found*);

-505: versione di HTTP non supportata dal web server.

-campo messaggio di stato: è esplicativo assieme al codice di stato dello stato della risposta.

Le righe di intestazione si spiegano da solo. NON ROMPERE IL CAZZO (Cit. appunti di Giovanni Melissari che in quel momento si è ricordato di avere le sue cose).

### DIFFERENZA TRA GET E POST

Entrambi i metodi richiedono risorse web il cui contenuto sarà messo nel campo body dell'HTTP response. Ciò che li differenzia è il modo con cui comunicano i campi contenuti in una form: con il metodo GET il contenuto dei campi della form è appeso all'URL della risorsa, pertanto in un HTTP request con GET il corpo del metodo resta tipicamente vuoto. Con il metodo POST il contenuto dei campi form viene inserito nel body dell'HTTP request corrispondente.

### HTTP CON TELNET

In telnet posso usare HTTP con la seguente sintassi:

telnet (apro telnet)

set local echo (se non metto questo non vedrò cosa mi dice il server)

open www.mat.unical.it 80 (apro la connessione)

GET/ (richiesta)

Ricorda: sulla porta 443 non è possibile fare GET perché è crittografata.

### I COOKIES

I cookies (biscottini) sono un meccanismo inventato per cercare di rendere statefull il protocollo HTTP: questo

perché si aveva l'esigenza di autenticare un utente, fornire contenuti particolari, spiargli, stalkerarlo, derubarlo (W i cookies!). I cookies lavorano con quattro componenti:

-un campo set cookie nell'intestazione dell'HTTP response;

-un campo cookie nell'intestazione dell'HTTP request;

-un file di salvataggio gestito dal browser, contenente i cookies da usare;

-un database di cookies con le associazioni <cooky, utente>, gestito dal web server.

### FUNZIONAMENTO DEI COOKIES

- 1) A manda un' HTTP request normale al web server B;
- 2) B si accorge che non ha ancora assegnato alcun cooky, quindi crea un nuovo identificativo e lo salva nel database associandolo all'utente in questione (A); dopodichè invia un HTTP response col campo *set-cookies : ID*.
- 3) A riceve il response e salva il cooky nel proprio file associandolo al nome di dominio dell'host, ad esempio *Amazon : ID*.
- 4) Da questo momento ogni volta che A richiede una risorsa a B, lo fa con un HTTP request contente come intestazione *cookie : ID*.

Questo consentirà a B di inviare ad A contenuti in risposta personalizzati. Con i cookie è come se il web ci avesse dato un identificativo ed i nostri pacchetti HTTP possano essere usati dai web server (se ci va bene) per ricostruire ad esempio le pagine preferite, riconoscere il tempo preciso in cui ho visitato una pagina e molto altro. I cookie non viaggiano sempre in chiare all'interno delle intestazioni HTTP, molte volte sono crittografate e queste da un certo senso ci da sollievo. Ogni cookie ha una data di scadenza, dopo la quale viene rimosso. Un cookie si definisce di sessione quando appena chiudo la sessione viene eliminato. Un cookie funziona sino alla sua scadenza.

### WEB CACHING – SERVER PROXY

Una *web cache* o *proxy server* è un servizio che fa da intermediario alle richieste di risorse web, memorizzandole in una proprio memoria (detta *cache*). Il browser di un client può essere configurato affinché qualsiasi richiesta HTTP uscente (io qui leggo “*uscente dal suo bel culetto*”, ora non so ma direi che lo omettiamo che è meglio. La cosa bella è che è sia sui miei appunti che su quelli di Melissari, perciò credo che l'anni in quel momento si sia dato un attimo al virtuosismo) vada prima a finire in un proxy server. Se la risorsa richiesta è presente nella sua cache, allora il proxy risponde direttamente, altrimenti fa una richiesta al server web di origine e fa una copia della risorsa nella sua cache. Il proxy è contemporaneamente sia server (quando dialoga con il client) che client (quando dialoga con il server di origine). Il web caching si è sviluppato su Internet per due ragioni:

- 1) riduce in modo sostanziale i tempi di ricezione degli oggetti web da parte del client, in quanto il proxy è situato tipicamente assieme alle reti locali delle aziende/enti che lo installano, per cui il collegamento client/proxy è estremamente più veloce di quello col mondo esterno;
- 2) diminuisce il traffico in rete. Tipicamente l'accesso ad Internet rappresenta un collo di bottiglia, per cui trasportarlo è molto pesante dal punto di vista del tempo; una cache può evitare di trasportare l'intero traffico sull'access point di uscita su Internet e quindi diminuire il traffico circolante.

Il web caching è ottimo da questi due punti di vista, ma introduce un problema: se l'originale è stata modificata, la copia in cache risulterà diversa?

Per ovviare a questo problema, è stato introdotto il meccanismo del **GET condizionale**. Una HTTP request condizionale è una particolare HTTP request con comando GET ed il campo

*IF-modified-since: DATA\_ULTIMA\_MODIFICA*

Quando un proxy riceve una richiesta di un oggetto web che ha nella cache, invia un HTTP request condizionale al server di origine il quale risponderà con l'oggetto solo se la data di ultima modifica dell'oggetto richiesto è più recente di quella dichiarata dal proxy nel campo If-modified-since. Se l'oggetto posseduto dal proxy è aggiornato (sì, è aggiornato e non aggiornato, ma ho immaginato che aggiornato possa essere un aggiornamento in cui nessuno crede tipo quello a Windows 10 e dovevo rendervi partecipi del battutone del secolo), il server risponde con un HTTP response con codice **304 Not Modified** ed il campo corpo vuoto. Questo consente di non saturare la rete inviando pacchetti inutilmente.

### DIFFERENZE TRA HTTP E SMTP

Entrambi i protocolli vengono usati per trasferire file da un host a un altro. HTTP trasferisce file (spesso chiamati oggetti) da un web server a un web client (solitamente un browser). SMTP trasferisce file (ossia messaggi di posta elettronica) da un mail server a un altro. Durante il trasferimento, sia HTTP persistente sia SMTP utilizzano connessioni persistenti. Esistono però sostanziali differenze. Innanzitutto HTTP è principalmente un **protocollo pull**: qualcuno carica informazioni su un web server e gli utenti usano HTTP per attiarle a se (*pull*) dal server. La connessione TCP viene iniziata dalla macchina che vuole ricevere il file. SMTP invece è un **protocollo push**: il mail server di invio spinge (*push*) i file al mail server in ricezione. La connessione TCP viene iniziata dall'host che vuole spedire il file. Un'altra differenza è che SMTP deve comporre l'intero messaggio (compreso il corpo) in ASCII a 7 bit. Anche se il messaggio contiene caratteri che non appartengono ad ASCII a 7 bit (per esempio, caratteri con accenti o dati binari come un'immagine), il messaggio deve essere comunque codificato in ASCII a 7 bit. HTTP invece non impone questo vincolo. Un'altra importante differenza riguarda la gestione di un documento che contiene testo e immagini (insieme ad altri possibili tipi di media). HTTP incapsula ogni oggetto nel proprio messaggio di risposta HTTP. La posta elettronica invece colloca tutti gli oggetti in un unico messaggio.



## **LIVELLO DI TRASPORTO**

Il livello di trasporto è un'importante strato di interfacciamento tra il livello 3 (livello di rete) e quello sottostante (livello applicazione). Il livello di trasporto è collocato esattamente sopra quello di rete nella pila di protocolli. Mentre un protocollo a livello di trasporto mette a disposizione una comunicazione logica tra processi che vengono eseguiti su host diversi, un protocollo a livello di rete fornisce comunicazione logica tra host. In particolare il livello di trasporto mette a disposizione del livello applicativo una comunicazione logica tra processi applicativi di macchine differenti, dando l'impressione che la comunicazione avvenga tra processi direttamente connessi. Nel client il livello di trasporto riceve i messaggi dal livello applicativo, li frammenta (solo se necessario) e li incapsula in segmenti, prima di passarli al livello di rete. In ricezione, il livello trasporti riceve il datagramma del livello di rete sottostante, lo lavora per ottenere il segmento e lo smista al processo opportuno del livello applicativo. Anche se il livello di trasporto può essere vincolato dai servizi offerti dal livello sottostante, esso può comunque implementare da se dei servizi verso il livello applicativo. E' possibile infatti che un protocollo di trasporto garantisca un trasferimento affidabile di dati, anche quando il livello di rete non lo è. Un protocollo a livello trasporto può offrire i seguenti servizi:

- *Multiplexing e Demultiplexing;*
- *Trasferimento dati affidabile (rilevazione errori, perdita, ordinamento dei pacchetti);*
- *Controllo di flusso;*
- *Controllo di congestione.*

## **MULTIPLEXING E DEMULTIPLEXING**

Multiplexing e demultiplexing definiscono come il servizio di trasporto da host a host fornito dal livello di rete possa diventare un servizio di trasporto da processo a processo per le applicazioni in esecuzione sugli host. Il multiplexing di trasporto, è il meccanismo con il quale un protocollo di trasporto raggruppa i messaggi del livello applicazione e crea dei segmenti, dotandoli di intestazione di trasporto con porta sorgente e porta destinazione. I segmenti indicano semplicemente il nome con il quale vengono identificati i pacchetti nel livello di trasporto. Il demultiplexing di trasporto è il meccanismo con il quale il protocollo di trasporto riceve i datagrammi dal livello di rete e leggendo i campi dell'intestazione di trasporto smista i segmenti ai socket corrispondenti. Ogni socket è identificato attraverso degli identificatori univoci e i segmenti di trasporto contengono quei campi che consentono di identificare il socket di ascolto di destinazione. Questi campi sono il *numero porta di origine* ed il *numero porta di destinazione*. del numero di porta di origine e il campo del numero di porta di destinazione. I segmenti UDP e TCP presentano anche altri campi. Ogni socket ha dunque un numero di porta, perciò quando un segmento arriva al livello di trasporto, esso esamina il campo porta destinazione e dirige il segmento verso il socket corrispondente.

## **PROTOCOLLI PER IL TRASFERIMENTO AFFIDABILE DEI DATI**

Un protocollo di trasporto può garantire il trasferimento dei dati affidabile al livello superiore. Offre un'astrazione al livello superiore che il canale di comunicazione tra i processi comunicanti sia affidabile. Una prima soluzione al

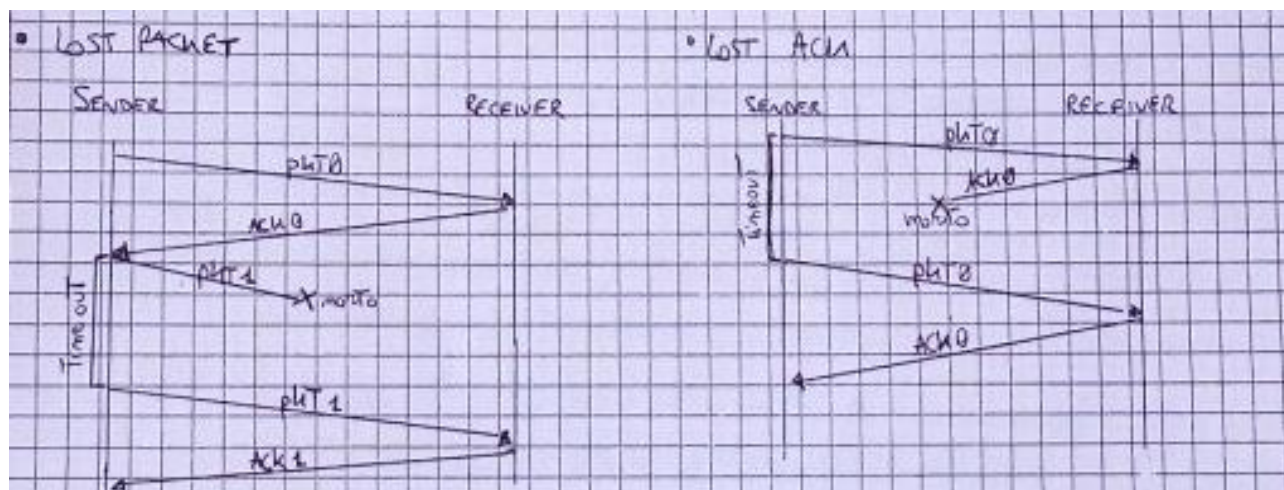
problema fu trovata con il meccanismo, ripreso poi dalla formula 1, di **STOP & WAIT**.

### PROTOCOLLO STOP&WAIT

Scenario: due processi su macchine diverse devono comunicare attraverso un canale non affidabile. Il meccanismo **stop&wait** prevede per ogni invio da parte di A un meccanismo di riceuta di ritorno da parte di B detto **ACKNOLEGEMENT (ACK)**. Inviato un segmento, A resta in attesa dell'ACK relativo a quel segmento che gli dia la conferma che B ha ricevuto il segmento in questione. A non può passare all'invio del pacchetto successivo fino a quando non rieve l'ack di quello corrente (da qui deriva il nome **stop&wait**). Per non cadere in tempi di attesa eternamente lunghi, dopo che A invia il segmento fa partire un timer, entro la fine del quale si aspetta di ricevere l'ACK. Se l'ACK non arriva, A ritrasmette il segmento : già da qui si intuisce l'importanza di questo timeout che gioca un rolo importantissimo nel protocollo. Nello stop&wait possono esserci scenari anomali, ed il protocollo ha dei comportamenti specifici per ognuno di essi.

### LOST PACKET E LOST ACK

Ognuno degli interlocutori in gioco non sa se l'altro interlocutore ha ricevuto, inviato o altro. Il meccanismo dell'ACK serve al sender per capire che il receiver ha ricevuto il pacchetto. E' possibile cha anche gli ACK vadano persi e non solo i pacchetti. Il sender non è in grado di distinguere tra le due tipologie di perdite, poichè in entrambi i casi non ha nessuna conferma. Per questo motivo il comportamento è lo stesso: avviene la ritrasmissione del pacchetto di cui non ho ricevuto l'ACK.



Nell'immagine si vedono le situazioni descritte sopra. A sinistra la perdita di pacchetti, a destra quella dell'ACK. A sinistra degli schemi il timeout.

### PREMATURE TIMEOUT

In questo scenario, l'ACK del primo pacchetto ritarda. Questo fa sì che il sender invii nuovamente un PKT0 (pacchetto zero, scritto così per adeguarsi alle figure) a causa della prematura scadenza del timeout. Il receiver così riceve due pacchetti 0, scartando il secondo ma inviando comunque un ACK. Il sender riceverà 2 ACK per 0, ma la seconda verrà ignorata. Il protocollo **stop&wait** lavora in maniera massiccia con le ritrasmissioni, ma si attiene ad un numero totale di ritrasmissioni, che una volta superato comporta l'invio di un messaggio di errore da parte del

• **PREMATURE TIMEOUT**

The diagram shows a SENDER and a RECEIVER. The SENDER sends a packet labeled 'PMTU'. The RECEIVER receives it and sends back 'ACK0'. The SENDER then sends 'PMTU' again, but it is received by the RECEIVER as 'PMTU (SCARSA) DUPLICATO'. The SENDER then sends 'ACK0' (labeled 'GIÀ CIEVUTO (MOROSA)'). The RECEIVER then sends 'ACK1'.

Per questo grave problema, si sono sviluppati nuovi protocolli sulla base di questo modello, in particolare i protocolli che sono oggi effettivamente usati, che sono i *protocolli SLIDING-WINDOW* o *protocolli a FINESTRE SCORREVOLI*.

## PROTOCOLLI PIPELING

Il problema principale dello stop&wait è che per poter inviare il prossimo pacchetto in attesa è necessario attendere l'ACK, cosa che provoca tempi di attesa e causa uno scarso utilizzo della banda a disposizione. Per risolvere la problematica, si è scelto di consentire al mittente di inviare più pacchetti contemporaneamente senza attendere l'ACK. Questo porta all'implementazione dei **protocolli di pipeling (a tubo)**, la cui gestione è più complessa ma che consentono di moltiplicare l'utilizzo della banda: se ad esempio in un dato istante ci sono tre pacchetti in volo contemporaneamente, l'uso della banda risulterà triplicato rispetto a quello dello stop&wait. Questa tipologia di protocolli è nota anche come **protocolli a finestra scorrevole (sliding-window)** in quanto mantengono un intervallo di  $N$  pacchetti che possono essere inviati dal buffer.

## PROTOCOLLO SELECTIVE REPEAT

Il **protocollo selective repeat (ripetizione selettiva)** è un protocollo di trasporto per il trasferimento dati affidabile che sfrutta il meccanismo delle finestre scorrevoli. I protocolli a finestra scorrevole usano una window, cioè **un intervallo di pacchetti pari a  $N$  che sono i pacchetti momentaneamente in attesa di ACK o in attesa di essere inviati**. Ogni sender non può avere più di  $N$  pacchetti in attesa di ACK. Se identifichiamo con base il pacchetto più vecchio in attesa di ACK, possiamo identificare alcuni intervalli:

- **finestra  $[base ; base + N - 1]$**  : rappresenta la finestra scorrevole, in questo intervallo sono presenti nel buffer di invio i pacchetti già trasmessi in attesa di ACK o slot liberi pronti per essere riempiti da pacchetti provenienti dal livello applicazione;
- **$[0; base - 1]$** : pacchetti fuori sequenza, rappresenta l'intervallo di pacchetti che hanno già ricevuto un ACK e pertanto sono già stati trasmessi al livello superiore o inferiore a seconda che sia sender o receiver;
- **$[base + N; dim\ buffer]$**  : pacchetti fuori sequenza, in questo intervallo cadono gli slot del buffer che sono momentaneamente disattivati, cioè con pacchetti che sono in attesa di essere inviati, oppure vuoti.

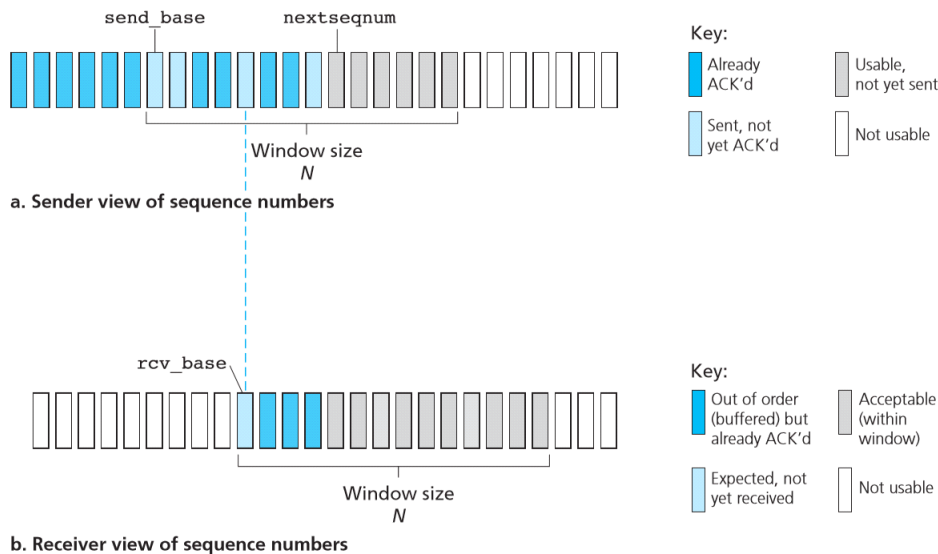
Nei protocolli a finestra scorrevole, il numero di sequenza di ogni pacchetto non è più un solo bit (1 oppure 0) come in stop&wait, ma un campo di  $k$  bit. Abbiamo quindi a disposizione  $2^k$  numeri di sequenza  $([0; 2^k])$ , ma tutte le operazioni su di essi avvengono in modulo  $2^k$ , producendo un insieme ciclico di  $2^k$  elementi il cui numero più grande è  $2^k - 1$ , a seguito del quale si riparte da 0. Un altro valore importante nella finestra, è il **nextseqnum**, vale a dire il numero di sequenza del prossimo pacchetto da inviare all'interno della finestra scorrevole. Questo numero varia nell'intervallo  **$[base; base + N - 1]$**  a seconda degli ACK ricevuti e dei prossimi pacchetti pronti ad essere inviati. Il **nextseqnum** si trova solo sul mittente.

## SENDER E RECEIVER NEL PROTOCOLLO SELECTIVE REPEAT

- Se il sender riceve dati dal livello applicazione, controlla qual è il prossimo numero di sequenza libero e se rientra nella finestra lo impacchetta e lo spedisce, altrimenti lo bufferizza restando in attesa;
- Per ogni pacchetto inviato si conta un timeout, alla fine del quale viene ritrasmesso il pacchetto;
- Se il sender riceve un ACK all'interno della finestra, allora etichetta il pacchetto come ricevuto. Se e solo se l'ACK ricevuto ha numero di sequenza uguale a send base allora sposta la finestra verso il primo pacchetto che non ha ricevuto ACK.
- Se il receiver riceve un pacchetto nella finestra, allora genera un ACK per quel pacchetto; se il pacchetto appena ricevuto è uguale a rcv base allora viene spostata la finestra verso il primo slot vuoto e viene

consegnato il pacchetto al livello superiore;

- Se il receiver riceve un pacchetto fuori sequenza, lo ignora ma manda un ACK al sender.



Il protocollo selective repeat. La prima sequenza riguarda il sender: per Already ACK'd si intende che l'ACK di quel pacchetto è ricevuto. Not usable indica che lo slot è vuoto o in attesa. Usable, not yet sent è in stato di pronto, mentre Sent not yet ACK'd significa che attende l'ACK. La seconda sequenza è invece per il receiver. Qui Out of order indica che non è rispettato l'ordine nel buffer ma che l'ACK è già stato inviato, Expected indica che l'ACK è atteso ma non ancora ricevuto, Not usable indica che lo slot non è utilizzabile, Acceptable dice che è accettabile all'interno della finestra. Si noti che i pacchetti che si trovano nell'intervallo  $[0; \text{base}-1]$  hanno già ricevuto un ACK e sono stati trasmessi al livello corretto. Nell'intervallo  $[\text{base} + N; \text{dim Buffer}]$  invece (cioè quelli a destra della finestra) sono i pacchetti fuori sequenza in attesa di essere inviati oppure slot del buffer vuoti.

Il protocollo è noto come selective repeat perchè per ogni pacchetto viene trasmessa un ACK selettiva propria.

## PROTOCOLLO GO-BACK-N

Il protocollo go-back-n prevede un **ACKNOWLEDGMENT CUMULATIVO**, che indica che **tutti i pacchetti con il numero di sequenza minore o uguale a quello corrente sono stati correttamente ricevuti**. Prevede una finestra scorrevole soltanto sul mittente mentre il receiver tiene conto soltanto del prossimo numero di sequenza pronto a ricevere. Il sender si comporta più o meno allo stesso modo del sender del protocollo selective repeat, a differenza che quando riceve un ACK ( $n$ ), seleziona come pacchetti ricevuti tutti i pacchetti in attesa fino ad  $n$ . Per cui sposta la finestra al pacchetto  $n+1$ . Questo comporta la nascita di un errore che il go-back-n non riesce a valutare: supponiamo che il sender invii pacchetto 8 e pacchetto 9. Supponiamo che il receiver riceva soltanto il pacchetto 9, ed invii un ACK(9) al sender che lo riceve e marca sia il pacchetto 8 che il pacchetto 9 come correttamente ricevuti. Questo comporta una perdita di pacchetto. Se il sender non riceve un ACK ed il timer scade, ritrasmette tutti i pacchetti che non hanno ricevuto ACK in quella finestra.

## PROTOCOLLO TCP (TRANSMISSION CONTROL PROTOCOL)

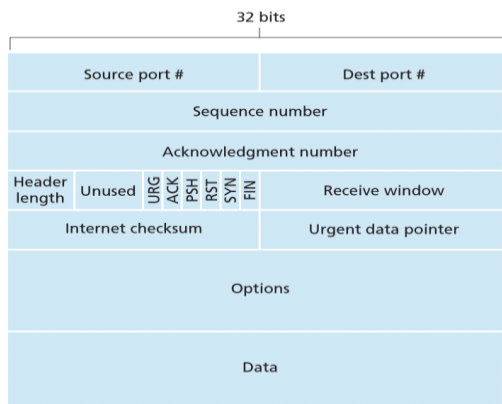
Il protocollo TCP è assieme al protocollo IP uno dei più importanti, se non il più importante, protocollo dello stack TCP/IP di Internet. E' il protocollo di livello di trasporto che fornisce al livello applicativo un trasferimento dati affidabile, una trasmissione orientata alla connessione e alcuni servizi di controllo di flusso e di congestione sulla rete. Grazie a TCP, i protocolli di livello applicativo che implementano comunicazioni con esso, hanno la garanzia di una comunicazione stabile ed affidabile.

### CARATTERISTICHE DEL PROTOCOLLO TCP

Il protocollo TCP implementa una comunicazione full-duplex, cosa che permette la presenza di due flussi di comunicazione contemporanei A->B e B->A. Inoltre, il protocollo implementa un trasferimento orientato alla connessione, il che significa che prima di trasferire i dati i due interlocutori si scambiano una serie di segmenti di servizio volti a sincronizzarli l'uno con l'altro. Questa fase di sincronizzazione preliminare è nota come THREE-WAY-HANDSHAKE perchè avviene (salvo duplicazioni) tramite lo scambio di tre segmenti TCP. Ogni protocollo TCP in fase di sincronizzazione alloca un buffer di invio e un buffer di ricezione che utilizzerà per i flussi in entrata e in uscita. La presenza di due buffer distinti per ogni processo TCP è la chiave per mantenere la comunicazione full-duplex.

### FORMATO DI UN SEGMENTO TCP

Un segmento TCP consiste di campi di intestazione (da un minimo di 20 byte, variabile) e un campo dati contenente il messaggio proveniente dal livello applicativo.



*Un segmento TCP*

- I campi numero di sequenza e numero di ACK contengono dei valori importanti nel servizio di trasferimento affidabile dei dati e per il riordino dei segmenti;
- Il campo lunghezza intestazione contiene la lunghezza dell'intestazione in quanto questa può variare al variare del campo opzioni;
- Il campo FLAG contiene dei bit come ACK che indica che il segmento in questione è un ACK, SYN che indica che si tratta di un segmento SYN, FIN che indica che si tratta di un pacchetto FIN (questi flag vengono usati nell'apertura e nella chiusura della connessione). Se il bit PSH ha valore 1 il destinatario dovrebbe inviare immediatamente i dati al livello superiore. Si utilizza il bit URG per indicare nel segmento la

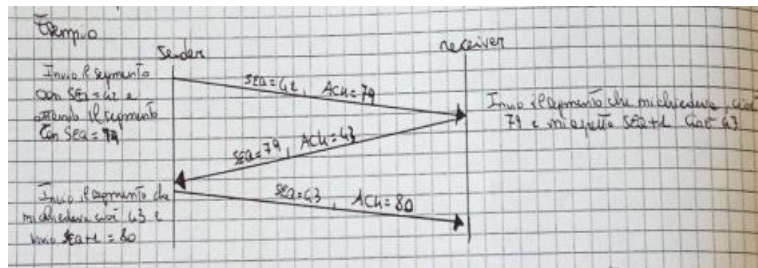


presenza di dati che l'entità mittente a livello superiore ha marcato come "urgenti". La posizione dell'ultimo byte di dati urgenti viene denotata dal campo puntatore ai dati urgenti, di 16 bit. Quando ci sono dati urgenti, TCP deve informare l'entità destinataria al livello superiore e passarle un puntatore alla fine dei dati urgenti. Nella pratica, PSH, URG e il puntatore non vengono usati.

- Il campo finestra di ricezione serve per indicare la propria grandezza della finestra ed è importante nel meccanismo di controllo di flusso per scambiarsi le proprie finestre di ricezione;
- Il campo checksum accoglie il risultato del checksum sul segmento a livello di trasporto.

### NUMERI DI SEQUENZA E NUMERI DI ACKNOWLEDGMENT

Questi due campi giocano un ruolo fondamentale nei meccanismi di trasferimento dati affidabile del TCP. TCP vede i dati provenienti dal livello applicativo come un flusso di byte. Attraverso alcuni valori (MSS, Maximum Segment Size) suddivide questo flusso in segmenti e assegna ad ogni byte del flusso un numero. Il numero di sequenza in TCP rappresenta il numero associato al primo byte del segmento in questione. Il numero di acknowledgment invece è il numero di sequenza del byte successivo che un processo TCP attende dal suo interlocutore.

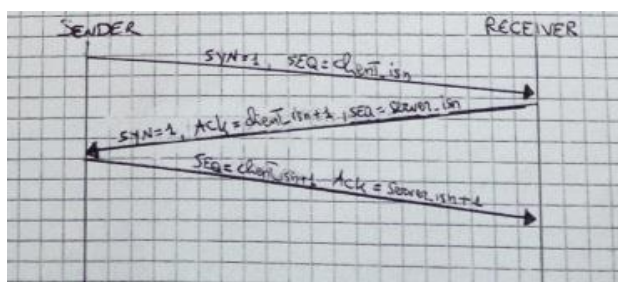


Nella figura si vede un'esempio di comunicazione. Il sender invia il segmento con  $SEQ=42$  ed attende il segmento con  $SEQ=79$ . Il receiver invia il segmento richiesto e aspetta  $SEQ+1$  cioè 43. Poi il sender invia il segmento richiesto (43) e aspetta  $SEQ+1$  cioè 80.

Il meccanismo di numerazione dei byte è importante proprio nello scenario dei protocolli a finestra scorrevole; è importante anche l'assegnazione di un numero di sequenza iniziale: questo non deve avvenire sempre di uno stesso numero (tipo 0) altrimenti favorirebbe il fenomeno dei pacchetti vagabondi, ma deve partire da un numero pseudocasuale. Il campo numero di sequenza è un campo a 32 bit per cui abbiamo  $2^{32}$  possibili numeri di sequenza (4 miliardi).

### THREE-WAY-HANDSHAKE

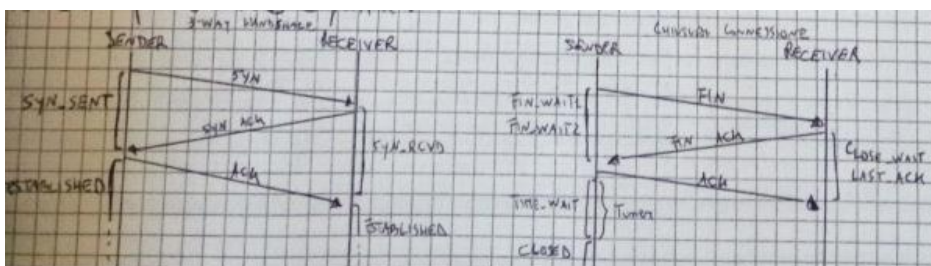
Il three-way-handshake è il meccanismo TCP attraverso il quale due interlocutori aprono una connessione TCP sincronizzandosi su due flussi.



- Passo 1: il client vuole aprire la connessione, pertanto il processo che ha aperto la socket di connessione invoca TCP per inviare un pacchetto TCP speciale che ha il flag SYN=1 e porta di destinazione uguale a quella definita dalla socket. Questo speciale segmento è noto come segmento SYN ed è il messaggio con il quale un client richiede una connessione TCP. Inoltre il client definisce in modo pseudocasuale un numero di sequenza iniziale client\_isn e lo invia al server;
- Passo 2: il server riceve il pacchetto SYN, quindi apre la socket di connessione e alloca i due buffer e le variabili TCP. Poi invia un pacchetto con SYN e ACK settati ad uno e un numero di sequenza definito in modo pseudocasuale server\_isn. A questo punto della connessione è aperto il flusso A->B.
- Passo 3: il client riceve il segmento SYNACK, quindi alloca i due buffer aprendo il flusso di comunicazione B->A e inviando un ACK con numero di sequenza pari a client\_isn+1 e ACK=server\_isn+1. I precedenti messaggi hanno il campo vuoto ma questo può contenere dati. A questo punto la connessione è infatti stabilita, i flussi sono entrambi generati e i due processi sono sincronizzati su numeri di sequenza per trasferirsi dati.

### CHIUSURA DELLA CONNESSIONE

Una connessione TCP può essere chiusa in qualsiasi momento attraverso una procedura ben definita sia dal client che dal server. Quando un processo vuole chiudere la connessione invia un pacchetto con il flag FYN==1 (pacchetto FYN). Il server riceve il pacchetto e dealloca i buffer rispondendo con un segmento FYNACK, cioè con flag FYN==ACK==1. Il client riceve il FYNACK e dealloca anch'esso i buffer rispondendo a sua volta con un ACK di conferma di connessione. Dopo l'invio di questo ACK vi è una fase di timer (circa 30s) al seguito della quale il sender stabilisce confermata la chiusura. Le procedure di apertura e chiusura della connessione sono particolari di TCP e sono un meccanismo piuttosto delicato che viene implementato con STOP&WAIT. Queste procedure definiscono per ogni processo TCP degli stati della connessione che variano e possono essere definiti.



Apertura (a sinistra) e chiusura (a destra) delle connessioni TCP. Si possono leggere i vari stati delle procedure (SYN\_SENT, ESTABLISHED ecc).

### CONTROLLO DEL FLUSSO TCP

TCP utilizza due buffer, uno di invio ed uno di ricezione, per mantenere il flusso di trasferimento full-duplex tra mittente e ricevente. Tuttavia è necessario comprendere che i flussi di segmenti in arrivo nel buffer dalla rete possono arrivare da una "frequenza" molto diversa da quella con cui il livello applicativo del ricevente li legge dal buffer, contribuendo a svuotarlo. Il controllo del flusso TCP si occupa di risolvere questa problematica. Se la velocità con cui il destinatario riceve i segmenti inviati dal mittente è molto più alta di quella con cui il livello applicativo li legge, è possibile che il buffer di ricezione vada in overflow causando la perdita di pacchetti. Il controllo di flusso utilizza il campo FINESTRA DI RICEZIONE del segmento TCP, che indica lo spazio libero nel buffer di ricezione del processo che ha spedito il segmento in analisi. Questo spazio libero è dinamico in quanto il buffer è continuamente



sottoposto a letture e ricezioni. Il processo che riceve un pacchetto con finestra di ricezione pari ad esempio a  $rwnd$ , deve mantenere il numero di pacchetti senza acknowledgment minore o uguale a questa soglia.

$$LastByteSent - LastByteAck \leq rwnd, \text{ con } 0 \leq rwnd \leq rcvBuffer$$

In questo modo è possibile controllare il numero di pacchetti inviati al destinatario ed evitare di mandare in overflow il suo buffer di ricezione. Dato che TCP è una comunicazione full-duplex, ogni processo TCP ha una finestra di ricezione propria distinta, che in alcuni momenti può anche essere uguale alla finestra di ricezione dell'interlocutore. Un processo TCP può comunicare la sua finestra di ricezione fin tanto che ci sono segmenti da inviare, cosa che introduce una problematica. Supponiamo che B (receiver) abbia saturato il suo buffer e invii un segmento ad A con  $rwnd = 0$ . Questo sarà l'ultimo segmento inviato in quanto non ha più dati da trasmettere. L'host A, ricevendo un segmento da B con  $rwnd = 0$ , è costretto a tacere fin quando non riceverà un altro  $rwnd$  non nullo, cosa che non accadrà visto che B ha finito di trasmettere. Per risolvere questo problema, TCP chiede ad A di inviare a B un segmento di un byte quando riceve  $rwnd$  nullo, al quale B risponderà con un ACK e finestra di ricezione aggiornata al suo nuovo valore.

### FRAMMENTAZIONE DEI MESSAGGI IN SEGMENTI : MSS (MAXIMUM SEGMENT SIZE)

Quando TCP riceve il flusso di messaggi dalle applicazioni a livello superiore deve poter suddividere questo flusso in segmenti. La loro dimensione non può essere scelta casualmente, ed infatti TCP sfrutta la MSS, che rappresenta la massima taglia possibile del campo dati di un segmento. Per calcolare la MSS, il protocollo TCP dialoga con un particolare protocollo di servizio di livello 3 che gli comunica la MTU (Maximum Transfer Unit) della rete di comunicazione tra due endpoint. TCP calcola la MSS, che sarà uguale a

$$MSS = MTU - 40 \text{ byte}$$

, dove i 40 byte rappresentano la somma delle taglie delle intestazioni TCP ed IP.

### CONTROLLO DI CONGESTIONE TCP

Il controllo di congestione TCP è un meccanismo attraverso il quale TCP riesce ad evitare la creazione di congestione all'interno della rete di comunicazione tra due host interlocutori. A differenza del controllo di flusso che influenza l'efficienza della comunicazione degli estremi della connessione, questo controllo ha impatto sulla qualità della comunicazione sulla rete, cioè sull'insieme dei nodi intermedi che compongono la rotta tra i due host comunicanti. L'approccio TCP consiste nell'imporre a ciascun mittente dei limiti sulla velocità di invio in funzione della congestione in rete presente in un dato istante. Se TCP si accorge che la rete è congestionata allora diminuisce la frequenza di trasmissione dei segmenti, altrimenti la aumenta. Questo approccio deriva dal fatto che se sul canale c'è congestione, allora invece di contribuire a inondare questo canale con una quantità di pacchetti elevata provo a diminuire la quantità immaginando di essere io stesso la prima causa di congestione. Il controllo di congestione mette in gioco una nuova variabile detta FINESTRA DI CONGESTIONE ( $cwnd$ ) che impone un vincolo sulla velocità di trasmissione del mittente.

$$LastByteSent - LastByteAcked \leq \min \{rwnd, cwnd\}$$

Questo vincolo accomuna e mette in relazione controllo di flusso e controllo di congestione. La finestra di congestione a differenza di quella di ricezione non deve essere comunicata tra gli interlocutori, in quanto è un'informazione utile solo al singolo host. Essendo  $cwnd$  il numero di byte che è possibile tenere in volo in un dato istante, la velocità di trasmissione sul canale sarà  $(cwnd/rtt)$  byte/s. Aumentando e diminuendo  $cwnd$  si aumenta e si diminuisce la velocità di trasmissione. TCP, per poter far questo, deve rilevare quando avviene una perdita all'interno

del canale tra gli host intermediari senza però conoscerne minimamente la struttura. Per far ciò TCP marca come “evento di perdita”:

- La scadenza di un timer, quindi la mancata ricezione di un ACK nel tempo previsto;
- La ricezione di 3 ACK duplicati, cioè 4 ACK identici (uno originale più 3 doppi).

In presenza di questi eventi TCP diminuisce la sua cwnd. Se invece TCP riceve ACK correttamente, la cwnd viene incrementata. Questa caratteristica di auto-completamento è nota come TCP auto-temporizzato.

### ALGORITMO DI CONGESTIONE TCP

L'algoritmo TCP che gestisce tutti i meccanismi per il controllo della congestione è noto come algoritmo di controllo congestione TCP, e si compone di 3 passi:

- *SLOW START*;
- *CONGESTION AVOIDANCE*;
- *FAST RECOVERY*;

#### SLOW START

Durante questa fase, TCP stabilisce la connessione ponendo  $cwnd=1\text{ MSS}$  che è un valore molto basso. Quindi dato che la banda è molto più ampia il protocollo vuole subito sapere qual è la banda disponibile. Comincia quindi cwnd in modo esponenziale ad ogni ACK correttamente ricevuto (fa un incremento moltiplicativo). Questa fase di crescita si prolunga fino a quando non avviene un evento di perdita (timer scaduto o triple ACK), e a questo punto TCP tiene conto di una variabile nota come *SOGLIA* o *THRESHOLD* che è tipicamente inizializzata a 8. Nel momento in cui avviene una perdita la finestra cwnd viene posta nuovamente a 1 e threshold viene posto alla metà del valore di cwnd quando aveva rilevato la perdita.

$$Threshold = cwnd/2;$$

$$cwnd=1\text{MSS};$$

Se invece la fase di crescita esponenziale si prolunga fino a quando il valore cwnd risulta essere uguale a threshold, l'algoritmo passa alla fase di congestion avoidance.

#### CONGESTION AVOIDANCE

Nella fase di congestion avoidance TCP aumenta il valore di cwnd ogni ACK corretto in modo lineare, proponendo un approccio più cauto rispetto a quello del slow start. In congestion avoidance quando si riscontra un timeout il comportamento è lo stesso del slow start (quindi  $threshold=cwnd/2$  e  $cwnd=1\text{ MSS}$ ), e l'algoritmo passa in slow start. Se invece riceve un triple ACK duplicato, allora il protocollo TCP passa in FAST RECOVERY, ed i valori vengono aggiornati nel seguente modo:

$$Threshold = cwnd;$$

$$cwnd = cwnd/2;$$

Il che significa che la finestra viene solo dimezzata.

## FAST RECOVERY

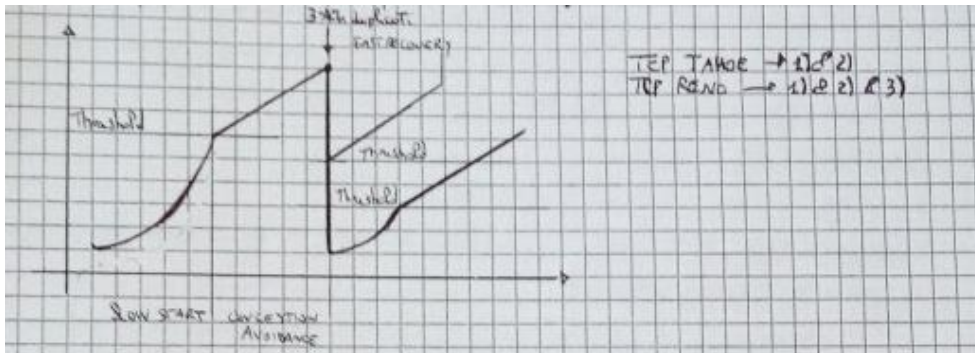
Nella fase di fast recovery TCP, cwnd viene incrementato di 1 MSS per ogni ACK duplicato ricevuto nella fase precedente che lo ha portato qui. A questo punto non appena arriva l'ACK corretto del pacchetto perso, l'algoritmo passa in congestion avoidance. Se invece avviene un timeout:

$$\text{Threshold} = \text{cwnd}/2;$$

$$\text{cwnd} = 1 \text{ MSS};$$

E l'algoritmo va in slow start.

Le prime due fasi dell'algoritmo sono obbligatorie, mentre la terza è stata aggiunta recentemente. L'algoritmo di congestione con solo le prime due fasi è noto come **TCP Tahoe**, mentre quello che comprende tutte e tre le fasi prende il nome di **TCP Reno**. Entrambe le versioni sono attualmente usate insieme ad altre più ottimizzate. Proprio per l'approccio ad incremento e decremento, il controllo di congestione TCP è noto come controllo di congestione a incremento additivo e decremento moltiplicativo, disegnando il tipico grafico a dente di sega.



Il grafico del controllo congestione. Si notano le tre fasi (SLOW START, CONGESTION AVOIDANCE e FAST RECOVERY) e i valori assunti dalla variabile Threshold.

## IMPOSTAZIONE DEL TIMEOUT DI RITRASMISSIONE

Il **timeout di ritrasmissione** in TCP gioca un ruolo di fondamentale importanza nei meccanismi di trasferimento affidabile di dati, nel controllo di flusso e soprattutto nel controllo di congestione. E' importante che questo timeout venga impostato nel modo migliore possibile per evitare timeout prematuri o latenze di trasmissione. Il timeout si basa su un valore detto *RTT* (Round Trip Time) che **rappresenta il tempo in cui un segmento viene inviato fino al tempo di ricezione del corrispettivo ACK**. Questo valore è un parametro che tende molto a variare, in quanto dipendente da molti fattori quali traffico in rete, pacchetti persi, banda dei canali di trasmissione. In pratica è un valore che cambia di trasmissione in trasmissione. Per calcolare l'RTT, TCP fa uso di un protocollo di servizio di livello 3. Il vero valore di RTT è noto come *sampleRTT*, ma TCP fa una media ponderata calcolando per ogni *sampleRTT* un *estimatedRTT*. Essendo una stima, questo nuovo valore ha oscillazioni che riflettono quello dell'originario ma di gran lunga più contenute. Il timeout TCP viene impostato a *estimatedRTT* più un certo valore che è anch'esso un riflesso delle variazioni. Questo valore è la *devianza del sampleRTT dall'estimatedRTT*. Il timeout TCP risulterà quindi uguale a:

$$\text{Timeout} = \text{estimatedRTT} + \text{devRTT}$$

## PROTOCOLLO UDP (TRASPORTO NON ORIENTATO ALLA CONNESSIONE)

Il protocollo UDP fa il minimo che un protocollo di trasporto dovrebbe fare. Implementa la funzione di multiplexing/demultiplexing e una forma di controllo degli errori molto semplice, non aggiungendo nient'altro al protocollo di livello di rete IP. Se si sceglie UDP anziché TCP, l'applicazione dialoga quasi in modo diretto con IP. UDP prende i messaggi dal processo applicativo, aggiunge il numero di porta di origine e di destinazione per il multiplexing/demultiplexing, aggiunge altri due piccoli campi e passa il segmento risultante al livello di rete. Questi incapsula il segmento in un datagramma IP e quindi effettua un tentativo di consegnarlo all'host di destinazione. Se il segmento arriva a destinazione, UDP utilizza il numero di porta di destinazione per consegnare i dati del segmento al processo applicativo corretto. In UDP non esiste handshaking tra le entità di invio e di ricezione a livello di trasporto. Per questo motivo, si dice che UDP è un protocollo (al contrario di TCP) non orientato alla connessione.

Un'applicazione viene implementata con UDP per diversi motivi:

- **Controllo più fine a livello di applicazione su quali dati sono inviati e quando:** Non appena un processo applicativo passa dei dati a UDP, quest'ultimo li impacchetta in un segmento che trasferisce immediatamente al livello di rete. TCP, invece, dispone di un meccanismo di controllo della congestione che ritarda l'invio a livello di trasporto quando uno o più collegamenti tra l'origine e la destinazione diventano troppo congestionati. TCP inoltre continua ad inviare il segmento fino a quando viene notificata la sua ricezione da parte della destinazione, senza preoccuparsi del tempo richiesto per un trasporto affidabile. Dato che le applicazioni in tempo reale spesso richiedono una velocità minima di trasmissione e non sopportano ritardi eccessivi nella trasmissione dei pacchetti mentre tollerano una certa perdita di dati, il modello di servizio TCP non si adatta particolarmente bene a queste esigenze. Queste applicazioni possono usare UDP e implementare, come parte dell'applicazione, funzionalità aggiuntive rispetto al servizio minimale offerto dal protocollo.
- **Nessuna connessione stabilita:** TCP utilizza un handshake a tre vie prima di iniziare il trasferimento dei dati. UDP invece "spara" dati a raffica senza alcun preliminare. Quindi UDP non introduce alcun ritardo nello stabilire una connessione. Questo è il motivo principale per cui DNS utilizza UDP anziché TCP (con cui risulterebbe molto più lento). HTTP, invece, usa TCP, dato che l'affidabilità è importantissima per le pagine web con testo, anche se il ritardo per stabilire una connessione TCP in HTTP contribuisce in maniera significativa ai ritardi associati allo scaricamento di documenti web.
- **Nessuno stato di connessione:** TCP mantiene lo stato della connessione nei sistemi periferici. Questo stato include buffer di ricezione e di invio, parametri per il controllo della congestione e parametri sul numero di sequenza e di acknowledgment. Queste informazioni di stato sono richieste per implementare il servizio di trasferimento dati affidabile proprio di TCP e per fornire il controllo di congestione. UDP, invece, non conserva lo stato della connessione e non tiene traccia di questi parametri. Per questo motivo, un server dedicato a una particolare applicazione può generalmente supportare molti più client attivi quando l'applicazione utilizza UDP anziché TCP.
- **Minor spazio usato per l'intestazione del pacchetto:** L'intestazione dei pacchetti TCP aggiunge 20 byte, mentre

UDP solo 8.

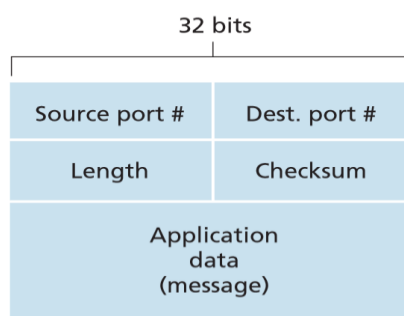
Le applicazioni possono ottenere un trasferimento dati affidabile anche con UDP. Ciò avviene se l'affidabilità è implementata nell'applicazione stessa (per esempio, con meccanismi di notifica e ritrasmissione come quelli che studieremo successivamente). Si tratta tuttavia di un compito non banale, che terrebbe occupato un programmatore per molto tempo. Avere affidabilità direttamente nell'applicazione consentirebbe ai processi applicativi di comunicare in modo affidabile senza essere soggetti ai vincoli sulla velocità trasmissiva imposti dai meccanismi di controllo della congestione di TCP.

| Applicazione                   | Protocollo a livello di applicazione | Protocollo di trasporto sottostante |
|--------------------------------|--------------------------------------|-------------------------------------|
| Posta elettronica              | SMTP                                 | TCP                                 |
| Accesso a terminali remoti     | Telnet                               | TCP                                 |
| Web                            | HTTP                                 | TCP                                 |
| Trasferimento file             | FTP                                  | TCP                                 |
| Server di file remoti          | NFS                                  | generalmente UDP                    |
| Dati multimediali in streaming | generalmente proprietario            | UDP o TCP                           |
| Telefonia su Internet          | generalmente proprietario            | UDP o TCP                           |
| Gestione di rete               | SNMP                                 | generalmente UDP                    |
| Protocolli di instradamento    | RIP                                  | generalmente UDP                    |
| Traduzione di nomi             | DNS                                  | generalmente UDP                    |

Protocolli usati dalle diverse applicazioni.

## SEGMENTI UDP

L'intestazione UDP presenta solo *quattro campi di due byte ciascuno*. I *numeri di porta* consentono all'host di destinazione di trasferire i dati applicativi al processo corretto (ossia di effettuare il demultiplexing). Il campo *lunghezza* specifica il numero di byte del segmento UDP (intestazione più dati). Un valore esplicito di lunghezza è necessario perché la grandezza del campo dati può essere diversa tra un segmento e quello successivo. L'host ricevente utilizza il *checksum* per verificare se sono avvenuti errori nel segmento. In realtà, oltre che sul segmento UDP, il checksum è calcolato anche su alcuni campi dell'intestazione IP.



Un segmento UDP.

## CHECKSUM UDP

Il checksum UDP serve per il rilevamento degli errori. Viene utilizzato per determinare se i bit del segmento UDP sono stati alterati durante il loro trasferimento (per esempio, a causa di disturbi nei collegamenti o quando sono stati memorizzati in un router) da sorgente a destinazione. Lato mittente, UDP effettua il complemento a 1 della somma di tutte le parole da 16 bit nel segmento, e l'eventuale riporto finale viene sommato al primo bit. Tale risultato viene posto nel campo checksum del segmento UDP.

Esempio:

Come esempio supponiamo di avere le seguenti tre parole di 16 bit:

```
0110011001100000
0101010101010101
1000111100001100
```

La somma delle prime due è:

```
0110011001100000
0101010101010101
-----
101101110110101
```

Sommando la terza parola al risultato precedente otteniamo:

```
101101110110101
1000111100001100
-----
0100101011000010
```

Notiamo che il riporto di quest'ultima somma è stato sommato al primo bit. Il complemento a 1 si ottiene convertendo i bit 0 in 1 e viceversa. Di conseguenza, il checksum sarà 1011010100111101.

In ricezione, si sommano le tre parole iniziali e il checksum. Se non ci sono errori nel pacchetto, l'addizione darà 1111111111111111, altrimenti se un bit vale 0 sappiamo che è stato introdotto almeno un errore nel pacchetto. UDP mette a disposizione un checksum perché non c'è garanzia che tutti i collegamenti tra origine e destinazione controllino gli errori. Inoltre, anche se i segmenti fossero trasferiti correttamente lungo un collegamento, si potrebbe verificare un errore mentre il segmento si trova nella memoria di un router. Dato che non sono garantiti né l'affidabilità del singolo collegamento né il rilevamento di errori in memoria, UDP deve mettere a disposizione a livello di trasporto un meccanismo di verifica su base end-to-end se si vuole che il servizio trasferimento dati sia in grado di rilevare eventuali errori. Dato che si suppone che IP funzioni correttamente con tutti i protocolli di secondo livello è utile che il livello di trasporto fornisca un controllo degli errori come misura di sicurezza. Sebbene metta a disposizione tale controllo, UDP non fa nulla per risolvere le situazioni di errore. Alcune implementazioni di UDP si limitano a scartare il segmento danneggiato, altre lo trasmettono all'applicazione con un avvertimento.

## LIVELLO DI RETE

Il livello di rete è quello in cui Alex Sandro la mette in mezzo, Dybala di petto fa la sponda, Higuaiiiiiiiiiin... RETE! RETE! Juventus in vantaggio! El pipita!

Oltre a questo, in modo secondario, il livello di rete si occupa del trasferimento dei pacchetti all'interno della rete. A questo livello gli interlocutori non sono gli end-point finali ma **tutti i router intermedi che compongono** la via del samur... **la via dal mittente al destinatario** (ok, la finisco. Giuro che la finisco). Nella panoramica di un trasferimento dati quindi a questo livello troveremo i due estremi, mittente e destinatario, che attraversano tutta la pila di protocolli per intero, e poi tutta l'intercapedine di router (tutti i router, ma sta parola mi piaceva), particolari dispositivi che implementano i primi tre livelli dello stack. A questo livello, il pacchetto dati è definito come **DATAGRAMMA**. All'interno di esso ho una particolare intestazione, e la cosa fondamentale di questa intestazione sono gli IP di mittente e destinatario. Quindi posso definire il datagramma come una busta contenente gli IP e poi i vari dati. A questo livello si perde il concetto di porta: ricevo in input datagrammi e spingo all'esterno datagrammi, senza guardare cosa c'è dentro. In pratica, a questo livello si smazzano i datagrammi guardando gli IP, non importa di sapere cosa c'è dentro. L'interfaccia tra livello tre e quattro è un insieme di funzionalità con una coppia di buffer di invio e ricezione, ed ho una coppia per ciascun indirizzo IP, e non una coppia per ogni porta. Esempio: supponiamo di avere 10 socket. Se inviano tutti, il buffer di uscita sarà sempre lo stesso, passo da un buffer per socket ad un buffer per interfaccia. Il buffer si gestisce con due primitive:

- `Send(...)`: prende i dati e li mette in uscita (invia datagrammi);
- `Receive()`: riceve un mex dal livello tre (riceve datagrammi).

Entrambi sono bloccanti, quindi se il buffer di ingresso è vuoto si ferma la `receive()`, se è pieno si blocca la `send()`.

### FUNZIONALITA' DEL SERVIZIO DI RETE

Il servizio di rete offre:

- **Incapsulamento del segmento proveniente dal livello di trasporto** in un pacchetto di livello tre. Il livello di trasporto invia il segmento in rete spostandolo sul livello 3 attraverso la primitiva di sistema `send()`;
- **Forwarding** di un pacchetto;
- **Comunicazione con livello 2 e livello 4** tramite protocollo di servizio *ICMP*.

### MODELLO DI SERVIZIO INTERNET

Il protocollo di rete di internet gestisce un trasferimento in una complessa costellazione di router intermedi. Gli interlocutori esterni sono gli host mittente e quello destinatario. Avendo un ruolo importante nello smistamento in rete dei pacchetti, si potrebbe pensare che il livello di rete si proponga, oltre che all'inoltro e all'instradamento, anche di effettuare alcuni meccanismi di affidabilità. Sappiamo però (perché lo sappiamo in quanto abbiamo studiato le cose di prima, vero?) che nello stack TCP/IP c'è il protocollo TCP a livello di trasporto che garantisce questo servizio al posto del livello di rete. Questo modello di servizio per il livello 3 è noto come **MODELLO BEST-EFFECT** (al massimo degli sforzi). Secondo questo modello, il livello di rete **si propone di fare il massimo affinché possa portare ogni pacchetto a destinazione, senza però dare alcuna garanzia** (sembra proprio la pubblicità "quell'antico vaso andava portato in salvo. Sembrava impossibile... ma l'abbiamo rotto!" so di aver detto che la smettevo ma non posso, o mi sopportate o cazzi :D ).

Questo modello è stato pensato dai progettisti affinché si rendano il più semplice possibile i protocolli di livello 3 e si lasci loro completa concentrazione sugli algoritmi di routing. Questo inoltre lascia la possibilità di implementare i meccanismi di trasferimento affidabile negli host, che essendo tipicamente dei calcolatori, sono potenti e adatti a questa implementazione.

## FORWARDING & ROUTING

Questa funzionalità è quella che caratterizza il livello 3.

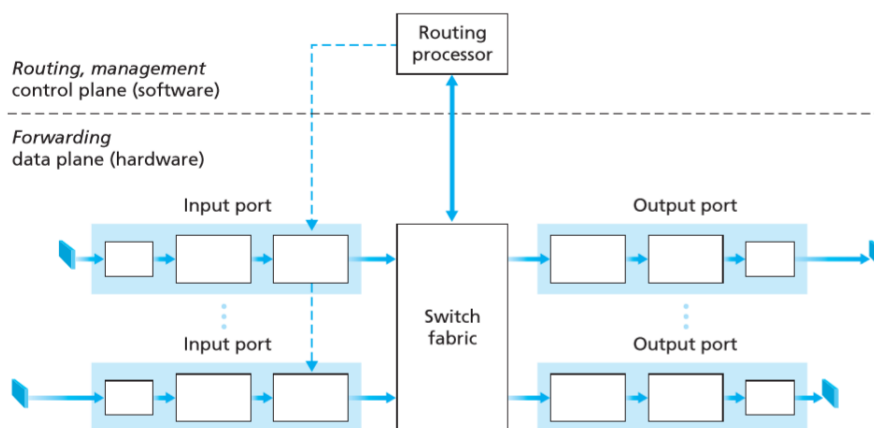
- **Forwarding (inoltro):** quando un pacchetto raggiunge un router attraverso una interfaccia, il router deve scegliere in relazione alla destinazione di quel pacchetto, su quale interfaccia di uscita deve trasmettere affinché prenda la strada giusta (o adeguata in quel momento) per raggiungere la destinazione;
- **Routing (instradamento):** un algoritmo di instradamento stabilisce il percorso intero che un pacchetto deve intraprendere per andare da mittente a destinatario.

Nonostante ogni router faccia una scelta "locale" su dove smistare il pacchetto in arrivo, è necessario che alcuni algoritmi di instradamento *stabiliscano le rotte giuste per permettere ai router di effettuare la giusta scelta locale*: questo significa che in realtà i router non hanno una visione globale della comunicazione tra end-point, ma contribuiscono in modo individuale alla costruzione di un pezzo della strada tra mittente e destinatario. L'insieme delle scelte fatte da ogni router portano il pacchetto a destinazione.

## ROUTER

Elemento centrale del livello di rete è quindi il **router**, vale a dire **un commutatore di pacchetti a livello di rete**. E' composto da almeno due interfacce di rete in quanto il suo scopo è quello di smistare pacchetti. Gli host generano traffico e ricevono traffico, il router riceve traffico in input ma il destinatario non è lui, quindi deve inoltrare le informazioni che gli arrivano. Il router deve essere configurato, e a volte ha una shell a volte una interfaccia web. Ad esempio nel router di casa, il router riceve dati con un IP che non è il suo. I datagrammi avranno l'indirizzo del router solo se parlo a lui direttamente.

## STRUTTURA DI UN ROUTER



La struttura di un router.



- **Input port:** le porte di ingresso accolgono i pacchetti in arrivo dall'esterno, e sono dispositivi fisici che lavorano a livello fisico (primo blocco, *termination line*), livello di collegamento (secondo riquadro) e livello di rete (terzo riquadro, *coda di input*);
- **Output port:** analogamente alla porta di ingresso, quella di uscita raccoglie i pacchetti provenienti dalla switching fabric che devono essere esternati. Anch'esso è composto da una *coda di output* (primo blocco), da un da una *terminazione* di livello due e di livello uno;
- **Switching fabric:** si tratta di una scheda hardware che collega fisicamente ogni porta di ingresso con tutte quelle di uscita. Avendo un ruolo cruciale nel forwarding è particolarmente specializzato in questo;
- **Routing processor (routing cpu):** è un vero e proprio processore che ha il compito di implementare ed eseguire gli algoritmi di instradamento. La routing CPU comunica con le porte in ingresso e con lo switching fabric e invia informazioni su dove inviare il pacchetto (quindi quale sarà la porta di uscita).

Tutti i componenti del router sono fabbricati in modo che sia possibile instradare i pacchetti nel modo più efficiente possibile. Lo switching fabric lavora su una scala temporale nell'ordine dei nanosecondi, mentre il routing processor lavora nell'ordine dei millisecondi / secondi. Ogni router mantiene una tabella nota come **FORWARDING TABLE**, che contiene per ogni rotta possibile sui nodi adiacenti le porte di uscita giuste. La *forwarding table* è centrale nel procedimento di inoltra. Questa tabella viene popolata e mantenuta costantemente aggiornata dal routing processor. Quando un segmento si trova nella coda di input e vuole passare nella switching fabric per poi arrivare alle porte di uscita, interroga il routing processor che, leggendo il suo IP di destinazione, elabora la richiesta e gli comunica la porta di uscita opportuna. Il pacchetto così passa nella switching fabric che attraverso la sua architettura hardware (a bus condiviso oppure a crossbar) lo trasferisce alla porta corretta. Ogni router è dotato di uno **SCHEDULATORE DI PACCHETTI** che in relazione al traffico di rete ed alle eventuali priorità dei pacchetti, **regola l'accesso alla switching fabric**. Quando una delle due code si riempie, avviene **una perdita di pacchetti**.

## **PROTOCOLLI DI LIVELLO DI RETE**

Il livello di rete presenta vari protocolli che lavorano insieme al fine di gestire la trasmissione nodo a nodo:

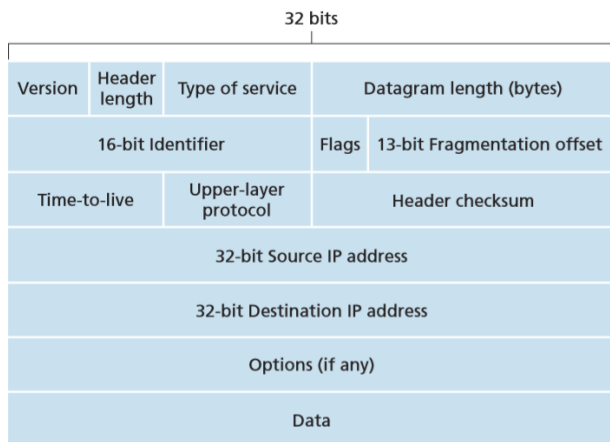
- **Protocolli di routing:** sono RIP (*Routing-Information Protocol*), OSPF (*Open Shortest Path First, della CISCO*), BGP (*Border Gateway Protocol, per grossi router*).
- **Protocollo IP:** è il fondamento di internet; standardizza il formato dei pacchetti, degli indirizzi IP e delle assegnazioni;
- **Protocollo ICMP:** protocollo di servizio, comunicazione di errori, comunicazione con livello 2 e livello 4.

## **PROTOCOLLO IP (Internet Protocol)**

Il protocollo IP, come già accennato, è il protocollo di livello 3 che standardizza il formato dei pacchetti, degli indirizzi IP e delle assegnazioni, posto alla base di Internet.

## **FORMATO INTESTAZIONE PACCHETTO IP**

Un pacchetto IP è formato da una intestazione di circa 20 byte avente i seguenti campi:



Pacchetto IP. L'intestazione di 20 byte comprende fino a IP di destinazione (incluso).

- **Versione:** indica la versione del protocollo IP (4 o 6);
- **Lunghezza intestazione:** essendo l'intestazione variabile a causa del campo opzioni, questo campo indica l'inizio della sezione *PayLoad (Data)* dando un'indicazione sulla lunghezza dell'intestazione;
- **ID, Flag, Spiazzamento di frammentazione:** sono importanti nella procedura di frammentazione dei pacchetti IP;
- **TTL (Time To Live):** indica il numero di nodi massimo entro il quale il pacchetto può circolare in rete (massimo numero di host attraversabili). Ad ogni salto il valore di TTL si decrementa di uno. Ogni router è tenuto a determinare questo campo, e se è zero deve scartare immediatamente il pacchetto sollevando un errore tramite un pacchetto ICMP che comunica la morte del pacchetto per TTL, quindi il pacchetto viene killato e se questo non accade rischio di creare traffico e di perdere troppo tempo, perché il pacchetto gira aspettando un router libero. Il TTL è stato introdotto per evitare che un pacchetto circoli all'infinito nella rete o incontri cicli.
- **Protocollo:** questo campo è il punto di comunicazione tra livello di trasporto e livello di rete. Indica infatti il protocollo adottato a livello di trasporto ed è utile nella fase di demultiplexing a livello 4.
- **Checksum:** è la checksum dell'intestazione.

### **FRAMMENTAZIONE DEI PACCHETTI IP**

Il livello di rete è *per forza di cause influenzato dal livello di collegamento*: un router può trasmettere i suoi pacchetti attraverso link architetturealmente molto diversi tra di loro. In particolare a seconda del link scelto IP deve frammentare i pacchetti in più datagrammi se la dimensione originale è maggiore della massima soglia trasmissiva su quel canale. Il livello di collegamento mette a disposizione del livello 3 (e quindi del protocollo IP) un valore noto come *MTU (Maximum transfer Unit)* che indica la massima taglia di datagrammi che può passare su un link. Il protocollo IP frammenta i suoi pacchetti in relazione a questo valore. Ogni router chiede al livello link la MTU del canale su cui sta trasferendo il pacchetto. Se la MTU è maggiore della taglia del pacchetto, lo trasmette altrimenti lo frammenta in  $\text{DimPack}/\text{MTU}$  (arrotondato per difetto) pacchetti di taglia MTU più un pacchetto residuo. I router si occupa della frammentazione, mentre la ricomposizione dei frammenti viene fatta dagli end-point finali (questo per garantire la semplicità del servizio offerto da IP) i cui sistemi operativi sono dotati di funzionalità di livello 3 che

implementano queste cose. I campi citati prima servono proprio per ricomporre in pacchetto a partire dai suoi frammenti:

- *ID* è un numero che identifica ciascun pacchetto (funziona ad autoincremento);
- *Spiazzamento di frammentazione* indica l'ordine dei frammenti;
- *Flag* indica quale di questi frammenti è l'ultimo (0 ultimo, 1 altrimenti).

#### INDIRIZZI IP VERSIONE 4 (IPV4)

Un indirizzo IP è un numero a 4 byte (32 bit) che viene scritto nella notazione decimale puntata:

**193.32.216.9**

Equivalente a

11000001.00100000.11011000.00001001

*Un host ha tipicamente una interfaccia di rete, un router almeno 2.* Ad ogni interfaccia di rete viene assegnato un indirizzo IP. Affinché un router possa smistare un pacchetto alla giusta destinazione, dovrebbe conoscere in modo globale tutti gli IP delle interfacce di rete su cui si affacciano i dispositivi e dovrebbe scegliere tra questi quale strada intraprendere per raggiungere la destinazione. La stessa struttura degli indirizzi IP evita questa situazione, strutturando la rete in sottoreti e consentendo di raggruppare queste sottoreti tramite un unico IP. La strategia di assegnazione degli indirizzi IP è nota come CIDR (Classes InterDomain Routing). Un indirizzo IP è suddiviso in due parti:

- Sezione dedicata alla sottorete (X bit);
- Sezione dedicata all'individuazione degli host di quella sottorete (32 – X bit).

Per esempio se destiniamo 24 bit per l'indirizzo di sottorete, il seguente ip avrà:

*193.23.216.9, vale a dire 24 bit per la struttura e 8 bit per gli host.*

Con CIDR è possibile scegliere qualsiasi valore per la sottorete da 1 a 32. La scelta dei bit dedicati alla sottorete è importantissima. Questi bit dedicati all'individuazione della sottorete sono noti come MASCHERA DI SOTTORETE. Per cui l'indirizzo sopra può essere scritto come

192.23.216.9/24, dove 24 è la maschera (ed indica il numero di bit dedicati alla sottorete), e che indica che l'indirizzo di sottorete vale 193.23.216.0. I restanti otto bit possono essere usati per gli host di questa sottorete. In conclusione abbiamo:

**SUBNET IP:** 192.23.216.0

**SUBNET MASK IP:** 255.255.255.0 oppure /24

**PC1 IP:** 193.23.216.1

**PC2 IP:** 193.23.216.2

**PC254 IP:** 192.23.216.254

Con 8 bit posso assegnare  $2^8 = 256$  possibili indirizzi IP. Tuttavia due di questi 256 sono già assegnati, e sono l'indirizzo di sottorete (193.23.216.0 nel nostro caso) ed un particolare indirizzo noto come BROADCAST che è:

**BROADCAST IP:** 193.23.216.255

Perciò posso avere  $2^8 - 2$  indirizzi IP disponibili. L'assegnazione di un IP alle sottoreti valuta la quantità di host presenti al loro interno e in relazione a quello si sceglie la maschera opportuna. *PICCOLA CURIOSITA'. Se faccio ping 10.2.1.255 sto pingando in un sol colpo tutti gli host della subnet perchè i calcolatori sono progettati per avere il broadcast come indirizzo jolly che contatta tutti gli host. Ora questo ping è disabilitato. Il ping di questo verso una vittima ignara significava dargli addosso, in una maschera /16 con  $2^{16}$  IP, 60.000 richieste! Il trucco stava nel pingare con l'indirizzo ip del fesso!! u.u ora è disabilitato, non è semplice fare questo giochino xD*

| MASCHERA | #SOTTORETI             | #HOST                          |
|----------|------------------------|--------------------------------|
| /24      | $2^{24}=16.777.216$    | $2^8 - 2 = 254$                |
| /25      | $2^{25}=33.554.432$    | $2^7 - 2 = 126$                |
| /30      | $2^{30}=1.073.744.874$ | $2^2 - 2 = 2$ rete punto-punto |

Un aspetto importante degli indirizzi IP è che una volta stabilita la sezione di sottorete, è possibile individuare i 32-X bit restanti tra altre sottoreti, appartenenti a quella di partenza, generando un meccanismo di assegnazione gerarchico.

ESEMPIO

*193.23.1.0/24 si divide in due 25*

**193.23.1.0/25                      193.23.1.128/25**

La suddivisione di una sottorete in altre sottoreti può avvenire solo in multipli di 2 in quanto ogni passaggio da una maschera all'altra aggiunge due sottoreti a quella originale:

/24 → /25: 2 sottoreti;    /24 → /26: 4 sottoreti;    /24 → /27: 8 sottoreti;    /24 → /30:  $2^6$  sottoreti.

La strategia di assegnamento CIDR è stata sostituita di recente da quella *CLASSFUL ADDRESSING*, che restringe l'uso della maschere da 8-16-24 bit esclusivamente (si parla di maschere **di classe A (8), B(16) e C (24)**). In generale l'indirizzamento IP è un meccanismo che rende internet versatile ed estremamente compatto grazie proprio alla sua struttura. Un router che vede un pacchetto in entrata con IP destinazione 193.23.1.1 visita la sua tabella di forwarding e al suo interno vi è una rotta, ad esempio

| SUBNET    | MASCHERA      | ETH |
|-----------|---------------|-----|
| 193.1.1.0 | 255.255.255.0 | 1   |

e tramite un semplice calcolo bit a bit il router capisce che l'IP di destinazione appartiene alla subnet 193.1.1.0/24 e che se vuole raggiungerla deve smistarla sull'interfaccia 1. **RICORDA: Le maschere sono, a livello di bit, sequenze di tutti uno tutti a sinistra seguiti da tutti zero a destra. Le maschere con i "buchi" non esistono!!!**

## ASSEGNAZIONE DEGLI INDIRIZZI IP

Un amministratore di rete che vuole assegnare degli IP alla sua rete, deve richiedere un blocco ad un ISP (Internet Service Provider). Questo sceglierà dai suoi IP disponibili (che a sua volta gli sono stati assegnati da un'entità globale nota come *ICANN, Internet Corporation for Assigned Name and Numbers*) il blocco di IP da concedere all'amministratore. Una volta assegnati, ogni host della sottorete può avere un indirizzo IP statico oppure uno dinamico. Il protocollo *DHCP (Dynamic Host Configuration Protocol)* gestisce l'assegnazione automatica e dinamica di un indirizzo ad un host che si collega ad una rete. Quando avviene questo, l'host appena in rete che vuole un IP avvia una comunicazione con DHCP (tipicamente sono comunicazioni broadcast) attraverso un server DHCP che gli assegna un indirizzo IP. Questo può essere temporaneo, allo scadere del tempo avviene un'altra conversazione che cambierà l'IP. Gli IP dinamici sono utili in reti wireless o LAN con molti client ma con un numero di connessioni minore dei posti disponibili.

## ROUTING TABLES

Qualsiasi dispositivo in rete (host o router) possiede una **ROUTING TABLE** che raccoglie le rotte possibili in relazione ai nodi adiacenti e alle interfacce. Le tabelle sui router sono più grandi e complesse di quelle degli host. La struttura di una routing table può cambiare a seconda del sistema operativo installato sulla macchina. Nelle routing tables ho un rigo per ogni destinazione, e per ogni destinazione indico che gateway devo usare. Se volessi memorizzare indirizzi IP "singoli", con una tabella di 2gb di indirizzi non avrei problemi (avrei  $2^{32}$  che è il massimo di IP). Ma i router domestici hanno 32mb di ram (normalmente), quindi questa forwarding table non si può avere e non ha manco senso. Conviene raggruppare gli indirizzi in qualche modo e fare gruppi di indirizzi. Ad esempio nel sistema postale non mando per ogni indirizzo, guardo il CAP. Raggruppo per CAP e poi invio, la parte dell'indirizzo importerà solo alla destinazione. Per questo sfruttiamo le maschere per raggruppare gli IP. Quando un host o un router si ritrovano ad effettuare forwarding su un pacchetto consultano la routing tables confrontando l'IP di destinazione. Questo matching ha però un ordine specifico:

- 1) Si applica la rotta con maschera più specifica (da 32 a 0) che corrisponde alla destinazione del pacchetto in questione;

ESEMPIO

DA 193.23.1.37

| <i>Destination</i> | <i>Gateway</i> | <i>SubMask</i>  | <i>IFace</i>    |
|--------------------|----------------|-----------------|-----------------|
| 193.23.1.0         | 10.0.2.1       | 255.255.255.0   | eth0            |
| 193.23.1.0         | 10.0.3.1       | 255.255.255.128 | eth1 (MATCHING) |

In questo esempio viene matchato il secondo campo, cioè la seconda rotta, in quanto la sua maschera è più stretta (25 contro 24).

- 2) A parità di maschera, si matcha la rotta con *METRICA* più bassa; la metrica è un fattore con il quale il sistema operativo stima la qualità di quel percorso in un determinato istante;
- 3) A parità di metrica, si lascia la scelta al sistema operativo.

## PROTOCOLLO ICMP

ICMP è un protocollo di servizio che si occupa di comunicare eventuali messaggi di errore destinati a IP e messaggi di servizio per il livello di trasporto. Di preciso ICMP si colloca proprio sopra IP in quanto i messaggi ICMP viaggiano all'interno dei datagrammi IP. Alcuni messaggi ICMP sono:

- *echo request / echo reply*, utilizzati dal programma bash PING;
- *unreachable network/ unreachable host/ unreachable protocol/ unreachable port*;
- *Time-To-Live exceeded*

## FIREWALLING

Assieme ai router, un altro dispositivo molto importante nella panoramica della trasmissione a livello di rete è il FIREWALL. Il firewall è un commutatore di pacchetti con un software che se programmato può regolare il traffico in entrata e uscita e quindi fungere da porta antiincendio (in inglese appunto "firewall", usatissimo nella famosa serie TV Chicago Fire, in cui giovani informatici travestiti da pompieri si trovano a combattere gli attacchi di hacker che scaldano talmente tanto i PC facendo nascere spaventosi incendi). Quando un pacchetto passa dal router, il firewall può prendere due decisioni:

- ACCEPT, lasciar transitare il pacchetto;
- DROP, distruggere il pacchetto e impedirne il passaggio.

Esistono due tipologie di fw: STATELESS e STATEFULL.

### FIREWALL STATELESS

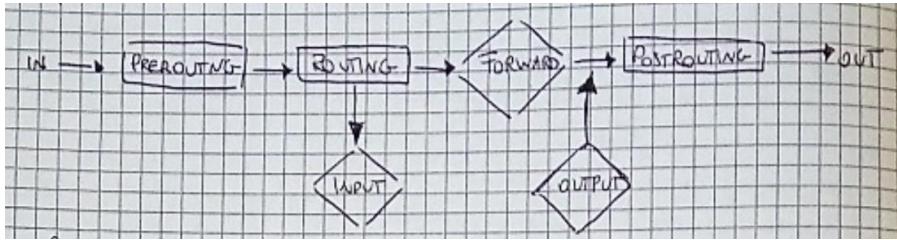
Ogni volta che deve essere analizzato un pacchetto è come fosse la prima volta in quanto questo tipo di fw non tiene traccia della possibile appartenenza ad eventuali conversazioni precedenti.

### FIREWALL STATEFULL

Tiene conto dell'appartenenza dei pacchetti a comunicazioni già stabilite, consentendone eventualmente il traffico diretto. E' il caso del firewall di Linux, che si chiama IPTABLES.

## **IPTABLES**

Il firewall IPTABLES è composto da CATENE, ognuna delle quali può contenere specifiche regole di firewalling che impongono restrizioni o libertà ai pacchetti consultati da quella catena. Le catene sono cinque:



- Catena di PREROUTING: è usata per pacchetti appena in entrata nel firewall e per manipolare eventualmente l'IP prima di rimetterlo in uscita;
- Catena di POSTROUTING: è usata per pacchetti che sono già stati processati dal firewall e per cambiargli l'indirizzo IP prima che escano dal fw;
- Catena di FORWARD: è la catena che valuta i pacchetti in transito nel fw;
- Catena di INPUT: serve per valutare i pacchetti destinati localmente al fw;
- Catena di OUTPUT: serve per valutare i pacchetti in partenza localmente dal fw.

### PROTOCOLLO NAT (NETWORK ADDRESS TRANSLATION)

Con gli indirizzi IP versione v4, cioè a 32 bit, è possibile carpire in totale circa 4 miliardi di host. La rete Internet però è in realtà composta da molte più entità (tra sottoreti, server, host, dispositivi mobili, router). Gli IP sono dunque limitati e questo pone un problema di limitazione: se un ISP ha esaurito quelli a sua disposizione non può più assegnarne. La conversione degli IP risolverà il problema, ma nel frattempo si è pensato ad un protocollo noto come NAT (Network Address Translation). Questo protocollo, implementato dai router, prevede un meccanismo invasivo di traduzione dagli indirizzi IP sorgente e destinazione di un pacchetto. Tra gli IP esista una particolare fascia, quella degli **IP PRIVATI o REAME (10.0.0.0)**, che sono destinati a dispositivi interni e a comunicazioni interne alle reti tra le sottoreti. Per questo non possono essere usati al di fuori di esse su Internet. Grazie al NAT esistono **centinaia di migliaia di sottoreti con indirizzi privati che si scambiano dati tra loro**, ma quando è il momento di interagire con internet, **il router NAT esegue una traduzione degli IP sorgenti da IP privati ad IP pubblici**. In questo modo tutti gli host dietro il router identificati da indirizzi privati sono nascosti e identificati da un unico indirizzo IP pubblico. Per poter effettuare il matching però, essendo una corrispondenza 1:N, non basta il solo IP ma serve anche il numero di porta in modo da identificare gli host esterni nel seguente modo:

*IP PUBBLICO / PORTA PUBBLICA ↔ IP PRIVATO / PORTA PRIVATA*

Ogni router NAT ha una tabella in cui memorizza queste corrispondenze nota come **NAT TABLE**.

### FUNZIONAMENTO DEL NAT

#### pacchetto interno

- 1) Un pacchetto proveniente dall'interno della rete raggiunge il router NAT con il suo IP privato e numero porta privato;
- 2) Il router NAT prende il pacchetto e genera una nuova porta pubblica che non è presente nella tabella; inserisce la nuova tupla nella tabella, modifica il pacchetto con in nuovo IP sorgente e #porta di origine e lo invia;

### pacchetto esterno

- 1) Un pacchetto proveniente dall'esterno della rete raggiunge il router NAT con IP pubblico e #porta X. Il router NAT *consulta la sua NAT TABLE e verifica la corrispondenza con quel numero porta*;
- 2) Esegue quindi la traduzione modificando il pacchetto e lo invia al destinatario della rete corretto.

All'interno della rete i pacchetti compaiono esclusivamente con gli IP privati e i numeri di porta originari. La traduzione avviene nell'inoltro verso l'esterno e all'entrata nella situazione inversa. Per effettuare un NAT e configurare un router affinché possa effettuare PORT FORWARDING, è necessario agire sulle catene di PREROUTING e POSTROUTING:

- *Un pacchetto interno con destinazione Internet*, viene tradotto nella catena di POSTROUTING;
- *Un pacchetto esterno con destinazione interna*, viene tradotto nella catena di PREROUTING.

Quello che si scrive nella catena di FORWARD vedrà dunque solo ed esclusivamente gli indirizzi interni degli host, cioè quelli privati.

### CONSIDERAZIONI SUL NAT

Il NAT non è ben visto dai "puristi" della rete, in quanto è molto invasivo nel cambiare IP e #Porta. Non risolve definitivamente la mancanza di IP (per questo si sta passando a IPV6), e le reti P2P soffrono parecchio della presenza dei router NAT.



## LIVELLO LINK

Il livello di collegamento si occupa di regolare la trasmissione dei dati fra dispositivi adiacenti, collegati da un unico canale trasmissivo (detto link). A livello di collegamento, gli interlocutori possono essere host, router, switch o hub, chiamati tutti **NODI**. Essendo che tra un nodo e l'altro ci possono essere canali trasmissivi fisicamente diversi (doppino, cavo di rame, aria...) il livello di collegamento ed i suoi protocolli dipendono strettamente da questo. Ci sono quindi due categorie di link trasmissivi:

- **Link punto-punto**: canale composto da due soli nodi, come ad esempio i collegamenti tra un router ed un altro o uno switch (*protocollo PPP, Point to Point Protocol*);
- **Link broadcast**: unico canale condiviso da più stazioni, che quindi possono anche trasmettere in contemporanea, come ad esempio un canale Ethernet con cavo coassiale oppure l'aria per il wireless e così via (protocolli *CSMA/CD* per Ethernet e *CSMA/CA* per WLAN 802.11).

Entrambe le categorie di link necessitano di protocolli di livello 2 (livello collegamento) che regolino l'accesso al canale trasmissivo e la trasmissione sul canale. Soprattutto dalla seconda categoria possiamo comprendere la scelta di chiamare un agglomerato di pc con il nome di **Dominio di Collisione**: quando un canale è condiviso da numerosi nodi, questi possono trasmettere in contemporanea e generare una collisione di dati sul canale che genera la distorsione dell'informazione.

### SERVIZI OFFERTI DAL LIVELLO 2

Il livello due offre i seguenti servizi:

- **Incapsulamento**: I datagrammi IP vengono incapsulati all'interno di **FRAME**. Il formato di un *frame* varia a seconda del tipo di link e quindi del protocollo di livello 2;
- **Accesso al link**: Il livello 2 fornisce dei protocolli che regolano l'accesso al mezzo trasmissivo, noti come *protocolli MAC (Medium Access Control)*;
- **Servizio affidabile**: Nonostante TCP si occupi già del trasferimento dati affidabile, è possibile che alcuni protocolli di livello 2 si occupino della stessa cosa, perché è molto più efficiente occuparsi da subito della perdita di informazione piuttosto che lasciare al mittente la possibilità di accorgersene e quindi ritrasmettere e rieffettuare il giro fino a qui.
- **Correzione di errori**: Il servizio di rilevazione e correzione degli errori offerto dal livello due è molto più preciso ed affidabile di quello ai livelli superiori in quanto implementato nell'hardware.

I protocolli di livello due sono implementati direttamente su hardware, sulla **SCHEDA DI RETE** o **NIC (Network Interface Card)**. A partire dal livello 2, si comincia ad evidenziare l'influenza dell'hardware fisico con cui sono implementati i mezzi trasmissivi.

### PROTOCOLLI MAC

I protocolli MAC sono i protocolli che regolano l'accesso al mezzo trasmissivo, cioè la modalità con cui un frame può essere indirizzato sul canale in modo da evitare perdite e ottimizzare l'uso della banda. La presenza di un protocollo MAC è quasi superflua in un canale punto-punto in quanto vi è la presenza di soli due nodi interlocutori.

Infatti i protocolli MAC si trovano abbondantemente nei canali trasmissivi condivisi (broadcast). Questi protocolli prendono il nome di protocolli di condivisione, e se ne distinguono tre categorie diverse a seconda della topologia della rete:

- Protocolli a suddivisione del canale;
- Protocolli ad accesso casuale;
- Protocolli a turni (o a rotazione);

### PROTOCOLLI A TURNI (POLLING E TOKEN-RING)

I protocolli a turni prevedono che si rispetti un determinato turno a rotazione prima che una stazione possa trasmettere sul segnale. Il protocollo POLLING prevede la presenza di un nodo principale che detti la sequenza del turno e mandi dei messaggi di accordo alla stazione che può dialogare sul canale. Quest'ultima può trasmettere per un determinato lasso di tempo alla fine del quale il nodo principale darà l'accesso sul canale ad un altro nodo. Lo svantaggio principale è che se si guasta il nodo principale (occhio al tecnicismo, rullo di tamburi) 'so cazzi. Inoltre se un nodo non vuole trasmettere e ne è presente solo uno che vuole inviare frame, questi dovrà comunque aspettare il suo turno. Il protocollo TOKEN-RING prevede la presenza di un particolare frame che circola nella rete, chiamato TOKEN (gettone). Solo chi possiede il token (in vendita in tutte le salette, 1 token 1 euro, 3 token 1 euro e 50) può trasmettere sul canale, ed ogni nodo passa il token al nodo successivo alla fine della propria trasmissione. Anche in questo caso lo svantaggio principale è rappresentato dal tempo. I protocolli a turni sono i primi modelli di MAC.

### PROTOCOLLI A SUDDIVISIONE DEL CANALE (TDM, FDM, CDMA)

Il multiplexing a divisione di tempo, TDM (time division multiplexing) e quello a divisione di frequenza, FDM (frequency division multiplexing), sono due tecniche che possono essere utilizzate per suddividere la larghezza di banda di un canale broadcast fra i nodi che lo condividono. Supponiamo che il canale supporti  $N$  nodi e che la sua velocità di trasmissione sia di  $R$  bps. TDM suddivide il tempo in intervalli di tempo (time frame) e poi divide ciascun intervallo di tempo in  $N$  slot temporali (time slot). Il time frame di TDM non deve essere confuso con l'unità di dati a livello di collegamento che le schede di rete si scambiano, che è anch'essa chiamata frame. Ogni slot è quindi assegnato a uno degli  $N$  nodi. Ogni volta che un nodo ha un frame da inviare, trasmette i bit del frame durante lo slot di tempo assegnatogli. Le dimensioni dello slot sono solitamente stabilite in modo da consentire la trasmissione di un singolo frame. TDM consente a un nodo di parlare per un intervallo di tempo stabilito. Lo stesso tempo spetterà quindi a un altro nodo, e così via. Una volta che tutti hanno avuto l'opportunità di "parlare", la sequenza si ripete. TDM viene utilizzato in quanto riesce a evitare le collisioni ed è perfettamente imparziale: ciascun nodo ottiene, durante ciascun intervallo di tempo, un tasso trasmissivo di  $R/N$  bps (bit per secondo). TDM presenta due problematiche precise. Anche quando non vi sono altri nodi che devono inviare un frame, quello che vuole trasmettere è costretto ad attendere il suo turno nella sequenza di trasmissione e a non superare il limite medio di  $R/N$  bps. Se ci fosse un solo nodo a voler comunicare mentre tutti gli altri volessero ascoltare (caso molto raro) TDM risulta una pessima scelta di protocollo per l'accesso multiplo.

Mentre TDM suddivide il canale condiviso in intervalli di tempo, FDM lo divide in frequenze differenti (ciascuna con una larghezza di banda di  $R/N$ ) e assegna ciascuna frequenza a un nodo. Quindi, a partire da un canale da  $R$  bps, FDM crea  $N$  canali di  $R/N$  bps. Anche FDM evita le collisioni e divide equamente la larghezza di banda tra gli  $N$  nodi.

Tuttavia, anche con FDM la larghezza di banda è limitata a  $R/N$ , pure quando vi è un solo nodo che ha un frame da spedire. Un terzo protocollo di suddivisione del canale è l'accesso multiplo a divisione di codice (CDMA, code division multiple access). Mentre TDM e FDM assegnano rispettivamente ai nodi slot di tempo e di frequenze, CDMA assegna loro un codice. Ciascun nodo, usa il proprio codice univoco per codificare i dati inviati. Se i codici sono scelti accuratamente, le reti CDMA avranno un'eccellente proprietà: consentiranno a nodi differenti di trasmettere simultaneamente e ai rispettivi destinatari di ricevere correttamente i bit dei dati codificati (assumendo che il ricevente conosca il codice) nonostante le interferenze derivanti dalle trasmissioni degli altri nodi. CDMA trova largo impiego nella telefonia cellulare. CDMA è molto legato ai canali wireless, ed i codici CDMA, come gli slot TDM e le frequenze FDM, possono essere allocati agli utenti del canale ad accesso multiplo.

### PROTOCOLLI AD ACCESSO CASUALE

I protocolli ad accesso casuale sono quelli attualmente più utilizzati. Una stazione trasmette sul canale se e solo se è libero, mentre se rileva una collisione durante la trasmissione ritrasmette il messaggio dopo un certo periodo di tempo pseudo-casuale (da qui il nome ad accesso casuale). Tra questi protocolli il più importante (che è alla base di molte tecnologie di oggi come Ethernet e WLAN 802.11) è CSMA.

### PROTOCOLLO CSMA/CD

Il protocollo CSMA/CD (Carrier Sense Multiple Access with Collision Detection) è un protocollo ad accesso casuale con rilevamento di portante e di collisione.

- Carrier Sense – Rilevamento portante: il protocollo trasmette sul canale solo se questo è libero, cioè soltanto se nessun altro sta già trasmettendo;
- Collision Detection – Rilevamento di collisione: dopo che è stato trasmesso sul canale il frame, il protocollo resta comunque in ascolto della portante in modo da poter rilevare eventuali collisioni. Se questo accade, il protocollo blocca subito la trasmissione.

Quindi anche se una stazione A ha trovato il canale libero, non è detto durante la trasmissione non avvenga una collisione. Questo è dovuto al ritardo di propagazione: quando A comincia a trasmettere, è possibile che lo faccia anche B, ma nell'istante in cui entrambi hanno interrogato il canale questi risultava libero.

### ALGORITMO CSMA/CD

- 1) La NIC ottiene il datagramma dal livello 3, lo incapsula in un frame e lo alloca nel suo buffer di invio;
- 2) Se il canale è libero il frame viene trasmesso, altrimenti si attende che il canale si liberi;
- 3) La scheda di rete resta in ascolto del canale anche se il frame è stato trasmesso e se non rileva collisioni dopo un certo lasso di tempo assume che tutto sia filato liscio;
- 4) Se viene rilevata una collisione durante la trasmissione, la scheda interrompe la trasmissione andando in modalità EXPONENTIAL BACKOFF.
- 5) La scheda di rete attende un tempo calcolato prima di effettuare la ritrasmissione, tempo noto come TEMPO DI BACKOFF, e dipende dal numero di tentativi di ritrasmissione effettuati: sia  $n$  la  $n$ -esima collisione, allora il tempo di backoff  $t$  è uguale a  $K \cdot 512$ , con  $K \in \{0, 1, \dots, 2^{n-1}\}$ , con  $K$  scelto casualmente da questo insieme. Perciò più è grande il numero di tentativi più il tempo massimo di backoff

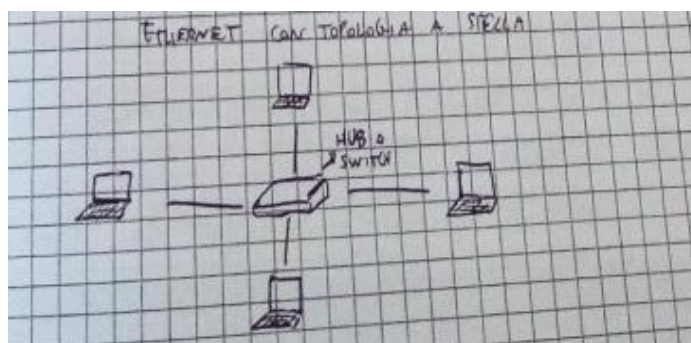
aumenta in modo esponenziale facendo diminuire la possibilità di collisione (più si aspetta, meno è probabile una collisione).

Quando CSMA/CD rileva una collisione blocca la collisione inviando sul canale un particolare segnale noto come SEGNALE SPORCO (JAM) che comunica a tutte le stazioni in ascolto che si è verificata una collisione.

Nonostante CSMA/CD adotti un meccanismo di temporizzazione attraverso il quale cerca di diminuire sempre più la possibilità che il suo frame collida, non si tratta di un protocollo di trasferimento dati affidabile. Se il canale in un certo istante risulta particolarmente intasato, dopo un certo numero di tentativi il protocollo solleverà una segnalazione di errore ai livelli superiori, che può essere interpretata come pacchetto perso. Se gli interlocutori dialogano con TCP, allora qualche tempo dopo la NIC in questione vedrà di nuovo lo stesso frame perché il mittente lo ha ritrasmesso.

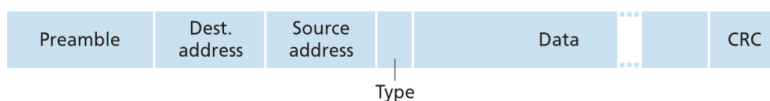
### TECNOLOGIA ETHERNET (102.3)

La tecnologia Ethernet è di gran lunga la tecnologia LAN più diffusa per via delle sue ottime prestazioni al fronte di costi ridotti. Si tratta di una tecnologia in quanto è sia una specifica di livello fisico che una specifica di livello collegamento, grazie al suo utilizzo con una grande varietà di mezzi fisici. Ethernet nasce negli anni '70 come la prima LAN, con cavo coassiale condiviso su cui affacciavano i nodi della rete. Questa topologia era nota come Ethernet a bus condiviso e aveva una velocità di 10 Mb/s. Negli anni '90 si passò da una rete broadcast ad una topologia a stella, grazie all'introduzione di un dispositivo noto come hub. In questa particolare topologia, ogni host era collegato ad un hub da un canale punto-punto (un doppino di rame intrecciato ad esempio). Inoltre in questi anni si passò ai 100 Mb/s. Negli anni 2000 un'ulteriore evoluzione portò la velocità Ethernet a 1 Gb/s e all'introduzione di un apparecchio più sofisticato dell'hub noto come switch. Nel 2006 l'ultima evoluzione portò le Ethernet a 10Gb/s.



Un ethernet con topologia a stella.

Oltre ad essere una tecnologia, Ethernet è anche un protocollo, ed in quanto tale presenta un formato per i suoi frame:



- Source Address e Destination Address: specificano l'indirizzo sorgente e quello destinatario di livello 2. Questo indirizzo è noto come indirizzo MAC;
- CRC: indica il risultato che controllo di errore CRC fatto dal mittente;

- TYPE: indica il tipo di protocollo di livello 3 (visto che non esiste solo IP). E' come il campo PROTOCOL per IP e numero porta per TCP e UDP.
- Preambolo: contiene 8 byte del tipo 10101010 tranne l'ultimo 10101011 che servono per sincronizzare i due clock delle schede di rete interlocutrici e per comunicare alla ricevente l'inizio del frame (1011).

### HUB VS SWITCH

Un HUB è un dispositivo di livello fisico che ha lo scopo di leggere i dati in entrata da un'interfaccia e duplicarli uguali a tutte le interfacce di uscita tranne quella di arrivo. E' un dispositivo molto semplice che ha sostituito il canale condiviso broadcast delle topologie a bus condiviso.

Uno SWITCH è un dispositivo più complesso: è un commutatore di pacchetti di livello 2 (quasi come un router ma a livello 2) che smista il traffico dalle interfacce di ingressi a quelle di uscita, con la sostanziale differenza che ha una memoria che gli consente di registrare da quali interfacce sono posti gli host adiacenti. In questi termini lo SWITCH rappresenta una rivoluzione in quanto consente di smistare sui canali corretti il traffico in arrivo (e non in broadcast) diminuendo così anche il traffico sugli altri canali e quindi la probabilità di collisioni. Lo SWITCH inoltre è PLUG-N-PLAY, vale a dire che tutto quello che impara lo impara da se. Lo SWITCH possiede una tabella nota come tabella di switch contenente:

**INDIRIZZO MAC**

**INTERFACCIA**

**TEMPO**

Ovvero l'indirizzo mac del nodo raggiungibile attraverso quella interfaccia e il tempo in cui questa riga è stata memorizzata.

### FUNZIONAMENTO DI UNO SWITCH

Uno switch lavora nel seguente modo:

- 1) Legge il frame in arrivo e prende il suo MAC DESTINATARIO, consulta la tabella e verifica la presenza della riga riguardante quel MAC;
- 2) Se il MAC non si trova nella tabella, allora lo switch si comporta con un hub con la differenza che memorizza nella tabella una nuova riga, trasmettendo in broadcast il frame a tutte le interfacce;
- 3) Se il MAC è presente nella tabella verifica che l'interfaccia di arrivo del frame non sia quella presente nella tabella: in questo caso significa che l'indirizzo è su quella stessa interfaccia e scarta il frame;
- 4) In caso contrario invia il frame sull'interfaccia di uscita indicata nella tabella.

Il meccanismo di PLUG-N-PLAY dello switch è garantito dal punto 2), grazie al quale, in mancanza dell'informazione in tabella, la memorizza autonomamente. Il campo TEMPO della tabella serve a controllare che le righe non stiano in tabella per più di un certo tempo, in successione del quale vengono cancellate per garantire gli aggiornamenti.

## INDIRIZZI MAC

Gli indirizzi MAC sono gli indirizzi con i quali è possibile identificare la scheda di rete di un dispositivo nelle comunicazioni di livello applicativo. Un indirizzo MAC ha 48 bit e viene scritto in formato esadecimale:

00: 30: f1: 0d: 19: db      6 byte in esadecimale

Un indirizzo MAC identifica ogni singolo host all'interno della rete ed è piatto e univoco per ognuna di esse. In sostanza in qualsiasi rete venga messa un dispositivo, l'indirizzo MAC di una scheda di rete, a differenza dell'indirizzo IP, resta sempre lo stesso. L'assegnazione del MAC address avviene in fase di fabbricazione della NIC e viene gestito dalla IEEE (Institute of Electrical and Electrotechnical Engineering). Gli indirizzi MAC sono importanti nella comunicazione di livello 2 in quanto indicano l'host subito adiacente al link di trasferimento. Così come in IP esiste un MAC broadcast che è

ff: ff: ff: ff: ff: ff      6 byte e 1 in esadecimale

Per capire come vengono usati i MAC address in una comunicazione di livello 2 dai nodi in gioco nella rete, occorre conoscere il protocollo ARP.

## PROTOCOLLO ARP

Il protocollo ARP è un protocollo di servizio di livello 2 che si interfaccia con il livello 3 nella traduzione tra indirizzi IP e indirizzi MAC. E' un po' come DNS tra applicazione e rete o trasporto e rete. E' molto importante perché garantisce il dialogo corretto tra i nodi al livello di collegamento. Quando un nodo A vuole inviare il datagramma IP ad un nodo B (in questo caso A e B sono collegamenti punto-punto) allora lo invia alla NIC che lo incapsula in un frame con mac mittente quello di A, ma che mac mette per il destinatario? Qui entra in gioco il protocollo ARP. A manda in broadcast un frame di tipo ARP REQUEST in cui chiede il MAC address corrispondente all'indirizzo IP di B. Solo B riconosce il suo IP e risponde in unicast direttamente ad A con il suo MAC address con un frame di tipo ARP-REPLY. Ora A possiede il mac address di B e può procedere alla spedizione sul canale del pacchetto. Ogni interfaccia di rete possiede tuttavia una memoria nota come ARP-TABLE che memorizza i seguenti campi:

**IP ADDRESS**

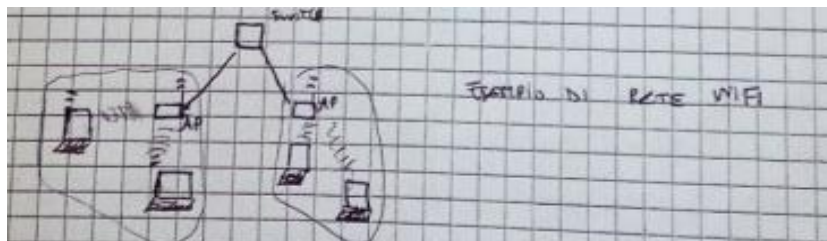
**MAC ADDRESS**

**TTL**

Questo consente al mittente di limitare il numero di trasmissioni ARP request e ARP reply che, a maggior ragione in perché sono in broadcast, potrebbero intasare le vie di comunicazione. Un altro possibile scenario è quello in cui A vuole inviare un datagramma a B che si trova in una sottorete diversa intervallata da un router. In questo caso, grazie alla sua tabella di routing, A capisce che B si trova dietro il gateway del router e quindi invia un ARP request per conoscere il MAC di quella interfaccia del router. Invia il frame e all'rivo su R l'IP di destinazione è ancora B, mentre il MAC destinatario è l'interfaccia di R. Il procedimento viene poi ripetuto. R grazie alla sua forwarding table vede che B si trova dietro la sua interfaccia e manda un ARP request a B e così via. E' importante notare che R possiede due ARP table, una per ogni interfaccia. Il protocollo ARP è un protocollo PLUG-N-PLAY.

## ARCHITETTURA W-LAN

Nella panoramica delle reti wireless si introduce un nuovo dispositivo noto come AP (Access Point) che ha lo scopo di effettuare una connessione con le schede di wireless della rete limitrofa e connetterli al mondo esterno. Tipicamente in una rete WI-FI abbiamo i dispositivi, un AP per ogni dominio di collisione che è collegato per esempio via ethernet ad un router o ad uno switch.



*Un esempio di rete wi-fi. Gli AP vengono collegati ad uno switch (in alto) e permettono la connessione alla rete da parte di diversi dispositivi in diversi domini di collisione.*

Nelle reti wireless il mezzo trasmissivo è l'aria, quindi si tratta la wi-fi di una rete ad accesso multiplo su canale condiviso. Ogni scheda WI-FI emette onde elettromagnetiche che devono rigettare all'interno di fasce di frequenza stabilite dagli standard. 802.11 lavora su fasce di frequenza che vanno da 2,4 GHz fino a 2,4856 GHz, quindi abbiamo 85 Mhz per il canale. Questa banda viene suddivisa in 11 canali di ampiezza 22Mhz che si sovrappongono, tranne l'1, il 6 e l'11. Le trasmissioni vengono suddivise all'interno di questi canali.

## PROTOCOLLO CSMA/CA

Il protocollo di accesso al canale trasmissivo per la Wlan 802.11 è un protocollo ad accesso casuale appartenente a CSMA noto come CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance), che sfrutta rilevazione di portante che evita le collisioni.

- Carrier Sense – Rilevazione di portante: CSMA/CA non trasmette sul canale se verifica che è utilizzato (proprio come CSMA/CD);
- Collision Avoidance – Rilevazione di collisioni: a differenza di CSMA/CD, /CA evita che avvengano collisioni sul canale.

CSMA/CD non sarebbe utile nelle 802.11 perché nel wi-fi è difficile implementare un sistema di rilevazione delle collisioni a causa delle attenuazioni di segnale, e perché a causa dei ritardi di propagazione e dell'influenza delle distanze è possibile che qualche stazione non si accorga delle collisioni. Per questi motivi CSMA/CA adotta un meccanismo più cauto rispetto a /CD e possiede un meccanismo di avvenuta ricezione dei messaggi noto come RTS/CTS.

## ALGORITMO CSMA/CA

- 1) Se il canale è libero, la stazione trasmette il frame;
- 2) Se il canale è occupato, la stazione calcola un tempo casuale di attesa (tempo di backoff), che verrà usato diversamente rispetto a come avveniva con /CD. In particolare:
  - a. Se il canale risulta inattivo CSMA/CA decrementa questo tempo ma non trasmette;
  - b. Se il canale risulta attivo CSMA/CA ferma il tempo e lo lascia invariato;

- 3) Quando il timer arriva a zero (accadrà solo quando il canale è inattivo) trasmette il frame;
- 4) Se riceve la conferma allora torna al passo 2). Se entro un dato tempo non riceve conferma, ritrasmette ma a partire da un tempo più ampio.

Utilizzare in questo modo il tempo di backoff serve ad evitare collisioni e sprecare la banda: se due stazioni volessero trasmettere, ma una terza occupasse il canale, le due stazioni aspetterebbero l'avvenuta liberazione del canale da parte della terza stazione. Appena questo succede i due nodi trasmettono producendo una costosa collisione, ma dato che CSMA/CA non blocca la trasmissione se avviene una collisione, questo tempo sarebbe tutta banda sprecata.

### MECCANISMO RTS/CTS

Questo meccanismo è implementato (ma opzionale) da 802.11 per evitare il problema dei terminali nascosti: due terminali si trovano ad una distanza tale da non riuscire a rilevare i propri segnali; in mezzo vi è un terzo terminale che rileva bene i due segnali e le collisioni che ne derivano. Questo meccanismo prevede l'invio di alcuni piccoli frame per prenotare il canale prima della trasmissione dei dati; questi frame sono noti come RTS (Request To Send) e CTS (Clear To Send). Quando un terminale vuole trasmettere invia un RTS all'AP. Se avviene una collisione allora si ritrasmette fin quando l'AP non risponde con un CTS, cioè un messaggio che segnala l'avvenuta prenotazione del canale. Il messaggio di CTS è ascoltato da tutte le stazioni, per cui tutte tranne quella che ha il permesso non possono inviare dati sul canale e A può trasmettere i suoi dati. Una volta trasmessi l'AP invia un ACK che sblocca tutti i terminali che ricominceranno quindi a chiedere RTS per prenotare il canale. Questo meccanismo è molto buono perché evita numerose collisioni. Tuttavia aggiunge dei ritardi di trasmissione dovuti allo scambio di questi speciali frame, anche se 802.11 può essere configurato per usare RTS/CTS solo per frame con un payload superiore ad una certa soglia.



## LIVELLO FISICO

Il livello fisico si occupa dell'implementazione fisica di link, modalità di trasmissione, standardizzazione dei mezzi trasmissivi, codifica dei segnali. Ogni segnale ha un suo spettro, che dipende dalle frequenze in gioco nel segnale. Ad esempio voci acute hanno uno spettro spostato sulle frequenze alte, le voci basse hanno uno spettro spostato sulle voci basse. Questo vale anche per i segnali digitali maggiore è la banda in bit/sec, più ampio è lo spettro. Il limite teorico dato dal teorema di Nyquist è  $2 S \log_2 H$ . Ma più è alto H, più il rapporto segnale rumore è alto!

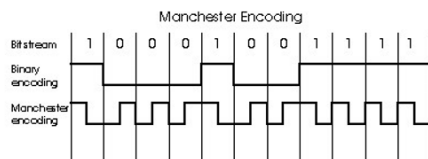
### CODIFICA MANCHESTER

Si usa nelle reti ethernet. Si trasforma il segnale digitale in uno *analogico* dove ogni segnale (0,1) viene confrontato con un alternanza di tensione che va da -5V a 5V in modo che il ricevente possa veder anche segnali adiacenti uguali (stringhe di 0 o 1).

Oltre alla Manchester ci sono varie modulazioni:

- Modulazioni di ampiezza (AM), con 0 ed 1;
- Modulazione di frequenza (FM), con 0 ed 1;
- Modulazione di fase in cui ogni simbolo (bound) è codificato con un certo tipo di segnale analogico.

### Manchester encoding



La codifica Manchester.

### TEOREMA DI NYQUIST

Problema: avendo H simboli e una banda B, a quanti Mb/s posso trasmettere ogni simbolo?

Risposta:  $2B * \lg_2 H$ , con  $B = 1\text{Mb}$ .

ESEMPIO: con 2 simboli ho 2Mbit/s, cioè 2 milioni di simboli al secondo. Con 4 ho  $2B \lg_2 4 = 2B*2 = 4\text{ Mbit}$  cioè 4 milioni di simboli /s.

## LINK FISICI

I mezzi fisici si dividono in due categorie: i mezzi vincolati (guided media) e quelli non vincolati (unguided media). Nei primi, le onde vengono contenute in un mezzo fisico, quale un cavo in fibra ottica, un filo di rame o un cavo coassiale. Nei secondi, le onde si propagano nell'atmosfera e nello spazio esterno, come avviene nelle LAN wireless o nei canali digitali satellitari.

**Doppino di rame intrecciato:** Il mezzo trasmissivo vincolato meno costoso e più utilizzato è il doppino di rame intrecciato, usato nelle reti telefoniche e comunemente presente nelle case e negli ambienti di lavoro. Il doppino intrecciato è costituito da due fili di rame distinti, ciascuno spesso meno di 1 mm, disposti a spirale regolare. I fili vengono intrecciati assieme per ridurre l'interferenza elettrica generata da altre coppie presenti nelle vicinanze. Di regola, un certo numero di doppini viene riunito e avvolto in uno schermo protettivo a formare un cavo. Una coppia di fili costituisce un singolo collegamento di comunicazione. Il doppino intrecciato non schermato (UTP, unshielded twisted pair) viene comunemente utilizzato per le reti all'interno di un edificio, cioè per le LAN. Le odierne velocità trasmissive di una rete locale che utilizza il doppino variano tra 10 Mbps e 10 Gbps. Tali velocità dipendono dallo spessore del filo e dalla distanza che separa trasmettitore e ricevitore. La tecnologia attuale del doppino intrecciato, in particolare la categoria 6a, può raggiungere velocità trasmissive di 10 Gbps per distanze inferiori a un centinaio di metri, perciò il doppino intrecciato rappresenta la soluzione migliore per le LAN ad alta velocità. Il doppino intrecciato viene comunemente usato per l'accesso a Internet residenziale.

### Doppino Intrecciato



(a)



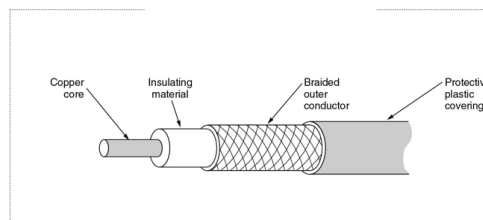
(b)

(a) Category 3 UTP.  
(b) Category 5 UTP.

*Doppino intrecciato*

**Cavo coassiale:** è costituito da due conduttori di rame, ma questi sono concentrici anziché paralleli. Con questa struttura, e grazie a uno speciale isolamento e schermatura, il cavo coassiale può raggiungere alte frequenze di trasmissione. Il suo impiego è piuttosto comune nei sistemi televisivi via cavo. E' peggiore del doppino perché non può essere intrecciato e le distanze lo influenzano molto.

### Cavo Coassiale

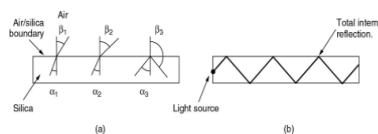


*Cavo coassiale*

**Fibra ottica:** La fibra ottica è un mezzo sottile e flessibile che conduce impulsi di luce, ciascuno dei quali rappresenta un bit. Sono fili di vetro, conduttori ottici composti da vetro interno e uno esterno. La luce passa nel vetro interno e ha un indice di rifrazione tale per cui il segnale si propaga senza perdita di intensità per lunghissime distanze. Una singola fibra ottica può supportare enormi velocità trasmissive, fino a decine o centinaia di gigabit al secondo. E' immune all'interferenza elettromagnetica, presenta attenuazione di segnale molto bassa nel raggio di 100 chilometri ed è molto difficile da intercettare. Queste caratteristiche hanno reso la fibra ottica il mezzo trasmissivo vincolato a lungo raggio favorito dagli operatori, in particolare per i collegamenti intercontinentali. Esistono due tipi di fibra:

- **Fibra mono nodo:** si parla con un solo colore (raggiunge distanze più elevate);
- **Fibra multi nodo:** si parla con più colori.

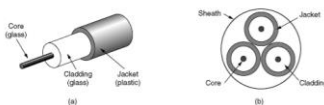
### Fibra Ottica



La luce è intrappolata nella fibra

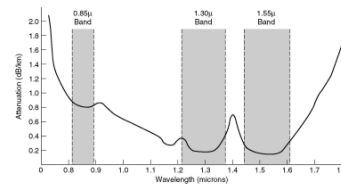
### Fiber Cables

- (a) Vista di una fibra singola  
(b) Una guaina con tre fibre



### Trasmissione della luce

Attenuazione della fibra  
Lunghezza d'onda = frequenza \* velocità del segnale (300'000km/sec)  
 $dB = 20 \log_{10} A$



### ADSL

Sul doppino passa su un piccolo canale la parte della voce. Da 25-110Mhz passano le informazioni di comunicazione. Il modem ha una interfaccia di rete che dialoga con moduli di 44 Mhz. Una parte del canale è destinato all'upload, una parte al download. Viene riservato più spazio di frequenza ai canali alti (download). Bisogna tuttavia sapere che a frequenze alte, l'attenuazione su lunghe distanze è maggiore. Il risultato è che deve avvicinare le centraline vicino casa (DSLAM). Questo è quello che si sta facendo, cioè portare la fibra ottica on the street, cercando di avvicinare DSLAM con collegamenti in fibra ottica.

## ***Processi, Thread, Multitasking, Multithreading.***

Su qualsiasi SO, ogni processo, può essere visualizzato tramite la gestione delle attività, attraverso il quale si potranno notare due colonne: PID e CPU (colonna dettagli).

- PID, se esso ha un valore pari a 0 indica che il processo non è in esecuzione
- CPU, indica la percentuale di inattività della CPU per quel processo.

Su Windows la somma totale di utilizzo della CPU, dei processi non andrà mai oltre il 100%, su Linux invece questa somma potremmo trovarla maggiore del 100%.

- La differenza sostanziale è che in Linux se troviamo un processo al 100%, indica che quel processo sta occupando interamente un core della CPU.
- In Windows invece, se un processo occupa il 25% della CPU totale, significa che sta utilizzando un intero core della CPU, quindi quando in Windows si arriva al 100% significa che la CPU è completamente saturata.

Infatti sempre supponendo una CPU quad-core, se lanciassimo l'esecuzione di un `while true`:

- ° Su Windows, vedremmo la CPU al 25% (occupa un core)
- ° Su Linux, vedremmo la CPU al 100% (occupa un core)

Se invece dovessimo lanciarne 4 di esecuzioni di `while true`, su Windows andremmo al 100% mentre su Linux al 400% (supponendo di avere 4 core a disposizione).

### ***Differenze con la macchina spiegata da Rullo e quella di Ianni.***

1. Non vi è soltanto il registro Accumulatore, ma ve ne sono presenti molti altri.
2. Le CPU reali si interfacciano anche con meccanismi esterni e non solo con la memoria RAM come visto con Rullo.
3. **INTERRUPT-CONTROLLER**: (è la differenza sostanziale), la sua funzione principale è quella di interrompere il ciclo di vita di un processo, da questo registro, parte un "filo" detto **INTR (Int-Request)**, che viene posto ad 1, per indicare che vi è un'interruzione dell'esecuzione, per far posto a qualcos'altro generato esternamente, interrompendo l'esecuzione del processo corrente.

Questo registro è utile per la gestione degli eventi, infatti se si preme un qualsiasi tasto, viene generata un'interruzione per poter gestire l'evento, ciò che avviene in modo particolare è porre l'INTR ad 1, successivamente il processore porrà l'**INTA (int-acknowledge)** ad 1 per indicare appunto che la CPU ha capito che dovrà eseguire un'interruzione dell'esecuzione, infatti ciò che si trova all'interno del Program Counter (PC) verrà salvato altrove, e al suo interno verrà trascritto il codice specifico da eseguire per la gestione dell'evento per cui si è generata l'interruzione (Event Handler).

## **PROCESSI e THREAD**

*Un Processo viene definito come un unico Main, tuttavia da esso possono partire tanti flussi, questi flussi sono i Thread del Processo.*

### **PROCESSI:**

- Sono una sorta di Software, i quali sono associati ad un unico file binario eseguibile
- Associato ad un insieme di Thread, i quali condividono la stessa Memoria (RAM) del processo.
- I processi non vedranno mai direttamente la memoria e le variabili di altri processi (es. Chrome.exe non potrà vedere i dati e i file di Word.exe), se si volesse garantire la comunicazione tra processi, bisognerebbe farli comunicare tramite file ai quali possono accedere entrambi, oppure tramite rete con protocolli TCP-IP.
- Infatti ogni Processo vede SOLO ed esclusivamente la memoria RAM che gli è stata riservata.

### **THREAD:**

- Sono un Flusso d'esecuzione
- Girano all'interno di un processo padre, infatti per un processo potranno esserci più Thread
- Ciascun Thread potrà accedere alla stessa Memoria RAM e ai dati del relativo processo
- Condividono tra di loro le stesse risorse e strutture dati.

## **MULTITASKING COLLABORATIVO**

Modello con cui funzionava la gestione dei processi di Windows. Questa metodologia tuttavia è ancora in uso per la gestione degli eventi di Java.

Una risorsa prende il controllo della CPU, e la occupa fin tanto che non ha finito la sua esecuzione, quando la rilascia, prende il controllo la successiva risorsa presente nella coda.

Es. **While(true){** //è il while del sistema operativo

**Evento e= codaEventi.take();** // è la coda secondo una struttura FIFO degli eventi che devono essere eseguiti.

**call(e.destinatario.winProc);** //il destinatario sarà colui che beneficerà della gestione dell'evento e, infatti WinProc, si occupa di gestire il codice specifico di quell'evento.

**}**

Con il multitasking collaborativo se un qualsiasi processo impiega troppo tempo ad essere eseguito, o se per sbaglio vi sono degli errori implementativi come dei while(true) che non si fermano mai, la coda si riempirà all'infinito, in quanto nessun processo verrà più eseguito e gestito oltre quello che già è in esecuzione.

I Thread in questo sistema passano dagli stati:

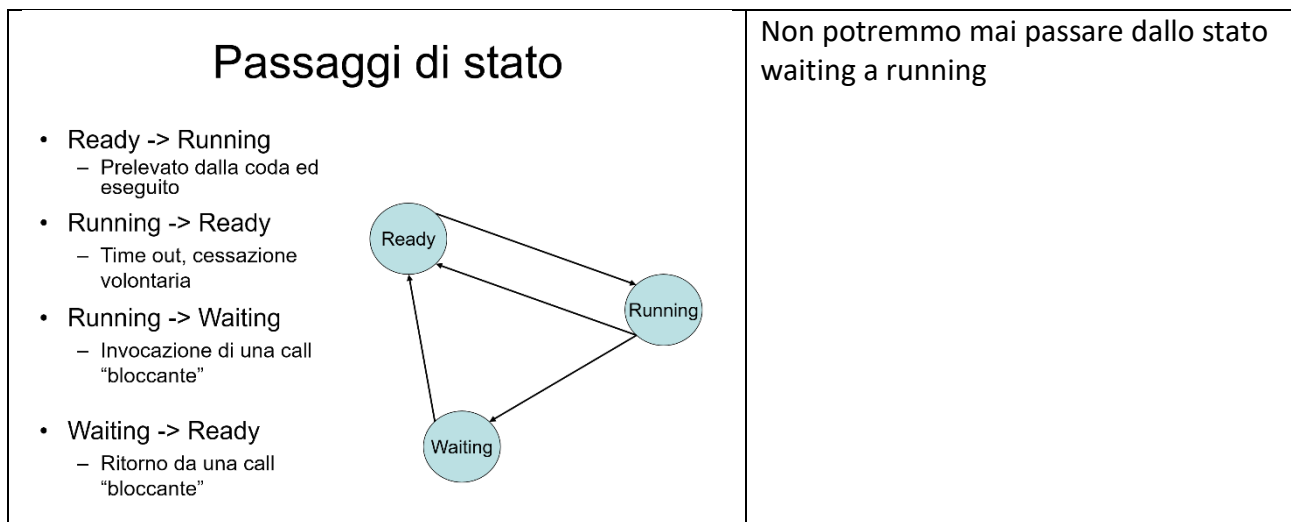
- Ready: ovvero quando sono pronti ad essere eseguiti
- Running: quando il Thread è in esecuzione
- Waiting: quando è in attesa di input esterni.
- 

Tuttavia per superare i limiti del multitasking collaborativo, si è creato lo scheduler dei thread.

## MULTITASKING NON-COLLABORATIVO

*“Non possiamo tenere la CPU impegnata per tutto il tempo che si vuole”*

Si avrà un tempo entro il quale il processo verrà interrotto a prescindere dallo stato in cui esso si trova. Lo scheduler guarderà alla coda dei processi prendendone uno tra quelli in stato di Ready e lo fa eseguire per un determinato lasso temporale, quando questo tempo scade, se il Thread non ha terminato la sua esecuzione verrà reinserto in coda in di Ready, e ne prenderà un altro, quando il Thread precedentemente interrotto ritorna in esecuzione riprenderà dallo stato in cui era stato interrotto. Tuttavia un Thread può andare direttamente in stato di waiting se termina la sua esecuzione in quanto non ha necessità di continuare e resterà in attesa di eventi esterni.



Es. di multitasking nn-cllbrtv

```
While(true){  
    Thread t= codaThread.take();  
    t.load();  
    impostaTimer();  
    t.exec();  
    t.save();  
}
```

Le operazioni Load e Save:

- Save(): scatta una fotografia del Thread nel momento in cui si è sospeso e la salva in memoria (TSS (Task State Statement) in Intel x86)
- Load(): carica “la fotografia” del Thread ripristinando lo stato in cui era stato salvato per fargli riprendere l’esecuzione.

**THREAD CPU-BOUND:** Sta sempre in stato di Ready, in quanto è sempre pronto ad eseguire

**THREAD IO-BOUND:** Legati principalmente alle interfacce grafiche in attesa di eventi e quasi sempre in stato wait.

Le operazioni Load e Save sono operazioni di Context-Switch.

## SCHEDULER-WINDOWS

Ha 32 code, ovvero ciascuna coda rappresenta un livello di priorità (0-31), più è alta la priorità e prima il processo verrà eseguito, rispetto a processi con priorità inferiore

Lo scheduler inizia dalla cella più alta (31) e valuta se all'interno di quella cella vi sono dei processi, allora li mette in stato di Ready, se non vi è nulla scende al livello più basso e così via. I processi utente non avranno mai priorità >26, mentre quelli di sistema avranno solo priorità >=27, i processi Real Time sono quelli che hanno la necessità di essere eseguiti in tempo reale (riproduzione video VLC) sotto i Real time troviamo le varie fasce che vanno dalla alta alla più bassa. Un processo solitamente nasce e muore nella fascia normale. Tuttavia agli utenti esterni è consentito modificare la priorità soltanto tra la Fascia Alta e la più Basso.

Quando un Thread supera un determinato limite di non esecuzione, vi sarà un processo di sistema che porterà la priorità del suddetto a 9, ovvero normale in modo tale che esso possa essere eseguito (ma non è detto). Il quanto di tempo per ogni Thread, è suddiviso in 6 blocchi, e sarà lo scheduler stesso a decidere quanto tempo assegnare a un processo, e quando interrompere la sua esecuzione.

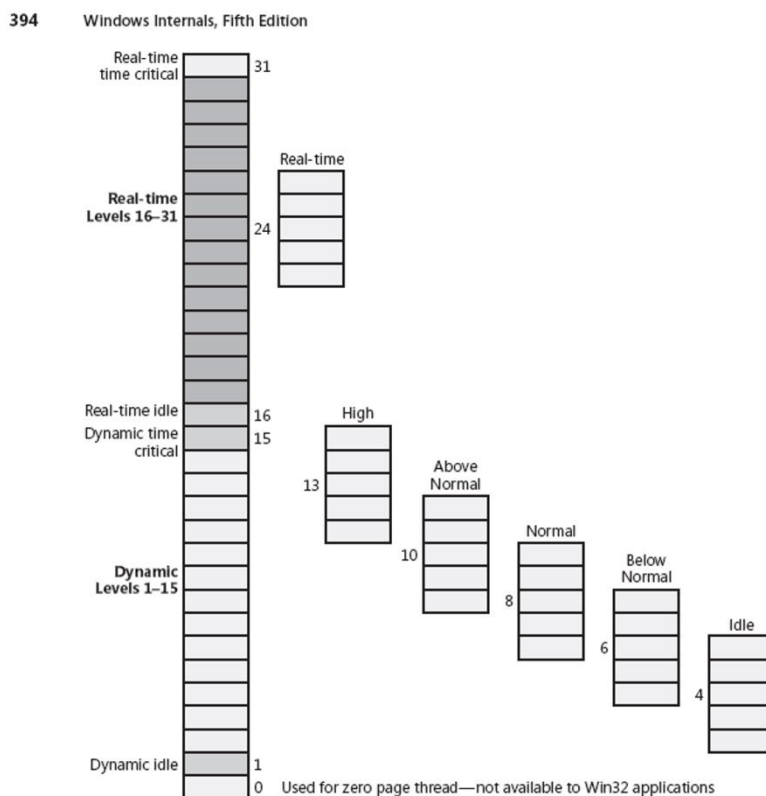


FIGURE 5-13 Mapping of Windows kernel priorities to the Windows API

## **SCHEDULER-LINUX**

Vi sono 3 modalità di scheduler.

Anche qui vi sono le priorità ma non come in Windows, poiché con Linux è il SO a dare le priorità ai processi.

In Linux la gestione è più semplificata, infatti, i Processi:

IO-BOUND: hanno priorità alta in quanto vogliono subito le risorse per l'utilizzo della CPU, e man mano la priorità sale

CPU-BOUND. Solitamente processi di sistema, man mano la priorità scende.

In Linux la priorità viene assegnata tenendo conto della percentuale di CPU usata precedentemente.

***N.B: Sia lo scheduler di Windows, che quello di Linux, lavorano su base Thread e non sui processi, infatti se un processo ha 10 Thread, allora in entrambi gli scheduler presenteranno 10 istanze di quel processo.***

## **PROBLEMI D'INCOSISTENZA**

Il codice che scriveremo dovrà esser pensato per sistemi multithread in cui molti client effettuano operazioni in modo concorrente, e ovviamente non possiamo permetterci di andare in contro a problemi d'inconsistenza dovuti ad una mal progettazione del sistema.

Infatti se per esempio si analizza il codice di esempio:

```
int posto[100];  
int allocaposto(int p, int codiceutente)  
{  
    if (!posto[p])  
        return posto[p] = codiceutente;  
    else  
        return 0;  
}
```

Questo codice potrà andar bene se il programma è pensato per un sistema mono Thread, ma per un sistema multi Thread è un suicidio, in quanto, più Thread potrebbero lavorare su questo processo contemporaneamente, inoltre sappiamo che potrebbero interrompersi in qualsiasi parte di questo codice poiché è lo scheduler a gestire il tempo e le richieste di ciascun Thread.

In questo modo potrebbe accadere che il posto **p** venga richiesto da più persone, e siccome non vi è un meccanismo che regola l'accesso al codice, e poiché i Thread potrebbero interrompersi in qualsiasi momento durante l'esecuzione, questo posto potrà esser assegnato anche a più persone, poiché la variabile verrebbe sovrascritta (vedi slide per esempio grafico).



## Metodi principali classe Thread

**.start():** È un metodo predefinito della classe Thread, prima crea e poi mette a Ready il Thread, la prima operazione che esegue è quella salvata nel Program Counter e cioè mandare l'esecuzione subito al metodo run, eseguendo il codice che vi è al suo interno.

**.run():** l'effetto è quello che è come se si lavorasse in un solo main, impostando il flusso primario all'interno del metodo ed eseguendovi il codice.

**.sleep(millis):** metodo della classe Thread, manda il thread in stato di wait per il tempo indicato come parametro, potremmo uscire da questo stato di wait, o perché è scaduto il tempo o perché il thread è stato stoppato.

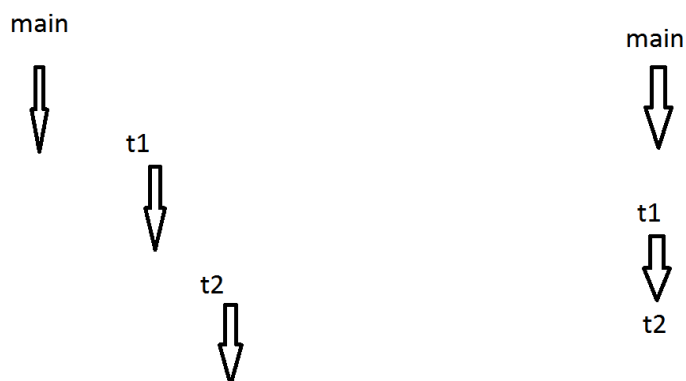
## Differenze tra il metodo .start() e .run() della classe Thread.

Vi sono delle differenze a seconda se noi facciamo partire il Thread con uno dei due metodi. Infatti supponendo di avere i thread t1 e t2:

Start

Run

|                                                                                                                                         |                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Se eseguiamo il codice:<br>t1.start();<br>t2.start();                                                                                   | Se eseguiamo il codice<br>t1.run();<br>t2.run();                                                                   |
| In questo modo se correttamente gestito stiamo effettuando il multithread, in quanto sarà lo scheduler di sistema a gestire i processi. | In questo modo non stiamo effettuando il multithread, ma il codice eseguirà prima t1 e poi t2 in modo sequenziale. |



Quindi abbiamo capito che i thread potrebbero accedere contemporaneamente a variabili globali, risorse, che dovranno essere disciplinate da appositi meccanismi, poiché queste richieste effettuate contemporaneamente sono imprevedibili e difficili da debuggare.

Dovremmo garantire una sorta di accesso esclusivo per evitare dei problemi d'inconsistenza, questo verrà reso possibile attraverso dei meccanismi di MUTA-ESCLUSIONE (accesso esclusivo a variabili globali), si utilizzeranno LOCK e metodi/blocchi SYNCHRONIZED.

## LOCK

Si individua il blocco di codice che viene eseguito in contemporanea e lo racchiudiamo all'interno del Lock.

```
Lock lck= new ...
Method(){
    Lck.lock();
...
... // qui dentro ci sarà il codice a cui garantire l'accesso esclusivo.
...
    Lck.unlock();
}
```

Se qualsiasi Thread arriva al LCK.lock() e la risorsa risulta essere impegnata già da un altro Thread, quest'ultimo si fermerà e attenderà il suo rilascio in quanto lck.lock() **è un metodo bloccante**, evitando così problemi di inconsistenza, poiché quando un Thread prende la risorsa siamo sicuri che vi potrà accedere e modificarla soltanto lui, e tutti gli altri successivamente vedranno le modifiche effettuate dal Thread che aveva l'accesso alla risorsa. Col .unlock() rilasciamo la risorsa.

Cosa succede:

Se per esempio abbiamo 3 Thread t1, t2, t3

T1 è il primo tra quelli in stato di ready che viene preso dallo scheduler, trova il lock() libero, e inizia ad eseguire il codice all'interno, poi arriva t2 e trova la risorsa occupata da t1 che detiene il lock, quindi t2 andrà in wait, così come t3 che arriva e la trova occupata, quando t1 rilascia, lo scheduler prenderà un Thread a caso tra quelli in stato di wait e il processo riparte questo nel caso in cui t1 riesce ad eseguire il codice durante il suo quanto di tempo.

Nel caso in cui il tempo non dovesse bastare e t1 viene bloccato all'interno del blocco critico, comunque sia tutti i Thread che vi arrivano dopo troveranno la risorsa occupata da t1 anche se egli non sta eseguendo e nessuno potrà accedere alla risorsa, quindi quando t1 verrà ripreso dallo scheduler, dallo stato ready ritorna a running e riprenderà l'esecuzione del blocco dal punto in cui era stato interrotto poiché è lui il proprietario del lock.

**ReentrantLock**-> Particolare tipo di Lock detto rientrante, ovvero se un Thread non termina l'esecuzione all'interno del Blocco in cui ha il lock, questo particolare oggetto lo farà riprendere da dove ha interrotto, se nel costruttore passiamo True es: **new ReentrantLock(true)**, il Lock sarà Fair, **Fair**-> ovvero tutti i coloro i quali accedono alla risorsa ma la trovano occupata verranno messi in wait e richiamati secondo l'ordine con cui hanno provato ad accedere **UnFair**-> vengono richiamati a caso dal pool in wait

## SYNCHRONIZED

In Java ogni Oggetto ha un lock nascosto accessibile attraverso la parola **synchronized**

Synchronized(this) si riferisce al lock nascosto della classe//

Se scriviamo un blocco:

Synchronized(this) { //equivale: thi.lockNascosto.lock()

//equivale this.lockNascosto.unlock();

### **Lock e blocchi synchronized**

#### • **Synchronized** ⇨ **Lock implicito**

- 1. Un lock può essere posseduto da un thread alla volta;**
- 2. Ogni thread che cerchi di prendere il possesso di un lock già occupato viene posto in stato di Wait;**
- 3. Un thread che possiede un lock può riacquisirlo quante volte vuole senza bloccarsi (lock rientrante);**
- 4. Quando un lock viene liberato, uno tra i thread in Wait sullo stesso lock viene svegliato e prende il possesso del lock;**
- 5. Ogni ISTANZA di classe possiede un UNICO lock nascosto usabile facendo uso di metodi synchronized o blocchi synchronized;**

Solitamente nei programmi che si pensano per il multithread vi è una struttura da dover “proteggere”, la classe in cui risiederà tale struttura, sarà progettata con diversi metodi pubblici, e dovremmo disciplinare l’accesso alle variabili in modo da evitare problemi di inconsistenza, e quindi rendere la classe Thread Safe. I vari Thread del programma, potranno accedere alla struttura soltanto tramite il proprio metodo run().

Le operazioni di Test and Set devono essere fatte tutte in una volta ovvero in un unico metodo per non aver problemi di inconsistenza poiché per esempio:

```
while(classeConStrutturaDaProteggere.testVariable()){  
    classeConStrutturaDaProteggere.setVariable();}
```

Sarebbe un errore gravissimo in quanto supponendo che entrambi i metodi di quella classe hanno il lock al loro interno dopo che abbiamo valutato e testato la variabile nel primo metodo potrebbe accadere che non riusciamo ad accedere subito nel metodo all’interno del while in quanto un altro thread ci ruba il lock ed effettua modifiche e soltanto dopo quando ritorniamo all’interno del while, ovvero da dove si è interrotta l’esecuzione, può capitare che ciò che abbiamo valutato prima sia cambiato e modifichiamo dati incongruenti, quindi per risolvere dobbiamo:

```
while(classeConStrutturaDaProteggere.testAndSetVariable()){}
```

Tuttavia anche se ci troviamo a dover affrontare soltanto operazioni di Test, dobbiamo renderle anch’esse Thread Safe. Es:

```
public synchronized void getNumber(){  
    return number;  
}
```

In questo modo saremmo sicuri che i Thread che vi accedono a tale valore ne leggeranno il contenuto esatto, senza che altri lo modifichino.

**N.B Il lock dovrà esser rilasciato come ultima cosa, non possiamo lasciare spezzoni di codice non protetti, in caso di metodi con ritorno si utilizzerà il blocco try{}finally{lock.unlock()} che ci consente di rilasciare il lock dopo che il metodo ha restituito il valore.**

## CONDITION

Bisogna evitare i Thread che fanno operazioni soltanto se vi sono determinate modifiche, in quanto questi thread prima prendono il lock, verificano in caso eseguono altrimenti lasciano il lock, questo prendere e lasciare lock, definito come POLLING, va evitato, poiché stiamo impiegando risorse e CPU inutilmente se non possiamo effettuare le operazioni in quanto la condizione non è cambiata, ci serve un meccanismo che consente di mandare in wait il Thread (così non viene schedato) e che lo risveglia soltanto quando vi sono modifiche effettive. Questo costrutto che ci consente di fare ciò sono le **CONDITION**.

Le **CONDITION**, sono definite come una sorta di allarme, devono essere abbinate ad un LOCK

Lock l -> su questo lock potremmo dichiarare quante Condition vogliamo.

Le operazioni che è possibile effettuare su di una generica Condition C, sono:

**C.await();** - > il suo effetto è quello di mettere in wait il thread e rilascia il lock

**C.signal();** - > il suo effetto è risvegliare un Thread scelto a caso tra quelli presenti nello stato di wait della condition.

**C.signalAll();** -> il suo effetto è quello di risvegliare tutti i Thread presenti nello stato di wait della condition.

Una condition può essere dichiarata soltanto su un Lock:

```
final Lock lock = new ReentrantLock();  
final Condition cond = lock.newCondition();
```

Possiamo dichiarare quante condition vogliamo per quel lock, ma una condition deve per forza riferirsi ad un lock.

Ogni condition possiede la sua sala d'attesa dei thread che vengono posti in wait ->  
**nomecondition.await();**

Dove inseriamo l'await dobbiamo già sapere in quale parte del codice ci sarà il signal, Infatti l'obiettivo di signal è proprio quello di svegliare e mettere nella sala d'attesa per il possesso del lock, anche se il possesso del lock resta a colui che ha fatto l'operazione di signal fin tanto che non lo rilascia.

In questo modo l'effetto che avremo è che lo scheduler userà il Thread soltanto quando siamo noi a decidere di porlo ready, per poter farlo andare in esecuzione, e non più quando lo scheduler decide di chiamarlo a suo piacimento anche se non ci serve quell'esecuzione.

N.B LE CONDITION POSSONO ESSERE UTILIZZATE SOLTANTO SE ABBIAMO PRESO POSSESSO DEL LOCK:

|                                                                                                                                                     |                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>condition.await(); lock.lock(); . . . Lock.unlock()</pre> <p><b>QUESTO RAPPRESENTA UN ERRORE GRAVISSIMO CHE SCATURISCE IN UN'ECCEZIONE</b></p> | <pre>Lock.lock();  condition.await();  Lock.unlock();</pre> <p><b>QUESTO È IL MODO CORRETTO DA DOVER SEGUIRE IN QUANTO NON GENERA NESSUN TIPO DI PROBLEMA.</b></p> |
|                                                                                                                                                     |                                                                                                                                                                    |

Dopo che facciamo signal, l'effetto è che il Thread che è stato messo nella wait della condition andrà nella wait del Lock. Il Thread prenderà il possesso di quest'ultimo quando sarà lo scheduler a deciderlo, riprendendo la sua esecuzione dal passo immediatamente successivo al condition.await();

La condizione che ci consente di andare avanti dev'essere testata sempre con un while al fine di evitare casi di **SPURIUS WAKE-UP**, ovvero un Thread può svegliarsi senza una signal. Proteggiamo quindi con un while, se mettessimo l'if andiamo direttamente dopo l'await anche se magari non dovevamo, poiché pensiamo che la condizione sia stata cambiata, utilizzando il while, invece ricontrolliamo continuamente la condizione dopo la signal.

|                                                                                                                                     |                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>While(!condizione) { Cond.await(); } . . .</pre> <p><b>QUESTO È CORRETTO perché DOPO IL SIGNAL RICONTROLLIAMO IL WHILE</b></p> | <pre>If(!condizioone) { Cond.await(); } .</pre> <p><b>QUESTO NON È CORRETTO IN QUANTO DOPO LA SIGNAL RIPRENDIAMO IMMEDIATAMENTE DOPO L 'IF</b></p> |
|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|

Da ricordare che l'effetto della signal è quello di far riprendere l'esecuzione da dopo l'await ().

## CODE BLOCCANTI – BLOCKING QUEUE

Coda FIFO Thread Safe, può risolvere problemi di sincronizzazione.

Da un lato inseriamo nel Buffer dall'altro Preleviamo.

Esempio:

Quando si masterizza un CD/DVD si utilizza il Buffer:

- La CPU prende i dati e li inseriva nel buffer
- Il Masterizzatore prende i dati dal Buffer e li scriveva sul CD

Le pizzerie sono un altro esempio di Buffer, infatti:

Da un lato arrivano le ordinazioni al pizzaiolo che vengono prese in sala,  
Dall'altro vi è il pizzaiolo che smaltisce gli ordini e soltanto quando essi saranno pronti,  
Verranno inseriti in un altro Buffer degli ordini pronti, da cui poi verranno prelevati da chi deve consegnare l'ordine

|                     |  |  |  |  |  |  |  |  |                                  |
|---------------------|--|--|--|--|--|--|--|--|----------------------------------|
| Inserisco<br>Ordine |  |  |  |  |  |  |  |  | Pizzaiolo<br>Smaltisce<br>Ordine |
|---------------------|--|--|--|--|--|--|--|--|----------------------------------|

|                                            |  |  |  |  |  |  |  |  |                                            |
|--------------------------------------------|--|--|--|--|--|--|--|--|--------------------------------------------|
| Pizzaiolo<br>Inserisce<br>Ordine<br>Pronto |  |  |  |  |  |  |  |  | Cameriere<br>preleva<br>l'ordine<br>pronto |
|--------------------------------------------|--|--|--|--|--|--|--|--|--------------------------------------------|

Quando tutto è vuoto la struttura va in wait.

Il Buffer dovrà essere implementato con un'array in modo da essere il più versatile possibile, e contenere qualsiasi tipo di Oggetto T.

Dovranno esser implementate due operazioni Principali:

- **PUT** (Inserimento nel Buffer): il cui prototipo sarà: **void put (T palluzza)**
- **TAKE** (estrazione dal Buffer): il cui prototipo sarà: **T take();**

Per l'operazione **put** verrà inserita una variabile che ci dirà qual è la posizione in cui dobbiamo inserire (**variabile IN**)

Pel l'operazione **take** verrà inserita una variabile che ci dice da quale posizione devo estrarre. (**variabile OUT**)

La variabile OUT seguirà la variabile IN.

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|

OUT

IN

Quando ci troviamo in questa situazione, si verificherà un ***ArrayOutOfBoundException***, poiché la variabile IN non punta ad elementi del Buffer. Per questo motivo per rendere il codice più versatile e generico possibile utilizziamo l'operatore % in modo tale da restare sempre all'interno del Buffer, e quando si raggiunge la **size** del buffer, ritornare da capo.

**In = (in + 1) % N;**

La questione principale legata al Buffer, riguarda i casi in cui IN e OUT dovessero sovrapporsi, quindi bisogna gestire due casi.

- **Buffer tutto vuoto:** Non posso prelevare nulla dal Buffer, **EMPTY**
- **Buffer tutto pieno:** Non posso inserire nulla nel Buffer, **FULL**

Per questo motivo il **Buffer Circolare** (Prende questo nome per via degli indici IN e OUT), dev'essere reso Thread Safe, introducendo un Lock per l'accesso in MUTA ESCLUSIONE, e due Condition EMPTY e FULL per la gestione dei due casi soprelencati.

Per evitare di non sapere se il Buffer è pieno o è vuoto verrà introdotta una variabile SlotPieni che sarà incrementata nel momento in cui inseriamo, e decrementata quando preleviamo.

Classe BlockingQueue fatta dal Prof.

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
```

```
public class BlockingQueue2016<T> {
    // la rendiamo di tipo generico per utilizzarla con tutti i tipi di oggetti.

    private T theBuffer[]; // sarà il nostro buffer
    private int in, out, slotPieni; // le variabili per vedere da dove inserire
    // prelevare, e tener conto degli slotPieni

    private Lock lock = new ReentrantLock();
    private Condition bufferEmpty = lock.newCondition(); // la utilizziamo per
    vedere quando il Buffer sarà tutto vuoto

    private Condition bufferFull = lock.newCondition(); // la utilizziamo per
    vedere quando il Buffer sarà tutto pieno

    @SuppressWarnings("unchecked")
    public BlockingQueue2016(final int dimensionOfBuffer) {
        in = 0;
        out = 0;
        slotPieni = 0;
        theBuffer = (T[]) new Object[dimensionOfBuffer]; // forziamo il cast a T in quanto
obj non i tipigenerici
    }
```

```

    }

    public void put(T elem) {
        lock.lock();
        while (slotPieni == theBuffer.length) {
            try {
                bufferFull.await();
            } catch (InterruptedException e) {

                e.printStackTrace();
            }
        }

        slotPieni++;
        theBuffer[in] = elem;
        in = (in + 1) % theBuffer.length;

        if(slotPieni == 1)
            bufferEmpty.signalAll();

        lock.unlock();
    }

    public T take() {
        lock.lock();
        try {
            while (slotPieni == 0) {
                try {
                    bufferEmpty.await();
                } catch (InterruptedException e) {

                    e.printStackTrace();
                }
            }

            slotPieni--;

            if (slotPieni == theBuffer.length - 1)
                bufferFull.signalAll();
            // svegluo soltanto se il buffer era tutto pieno e ho liberato un
            // posto
            return theBuffer[out];
        } finally {
            out = (out + 1) % theBuffer.length;

            lock.unlock();
        }
    }
}

```



L'implementazione di Java che fa esattamente quanto visto sopra, invece si riferisce all'interfaccia `BlockingQueue<E>`, di cui troviamo diverse implementazioni, la principale che possiamo utilizzare, è `ArrayBlockingQueue`, che offre i metodi:

| Constructor and Description                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><code>ArrayBlockingQueue</code></b> (int capacity)<br>Creates an <code>ArrayBlockingQueue</code> with the given (fixed) capacity and default access policy.                     |
| <b><code>ArrayBlockingQueue</code></b> (int capacity, boolean fair)<br>Creates an <code>ArrayBlockingQueue</code> with the given (fixed) capacity and the specified access policy. |

`void put(E e) throws InterruptedException`

Inserts the specified element into this queue, waiting if necessary for space to become available.

`E take() throws InterruptedException`

Retrieves and removes the head of this queue, waiting if necessary until an element becomes available.

`public int size()`

Returns the number of elements in this queue.

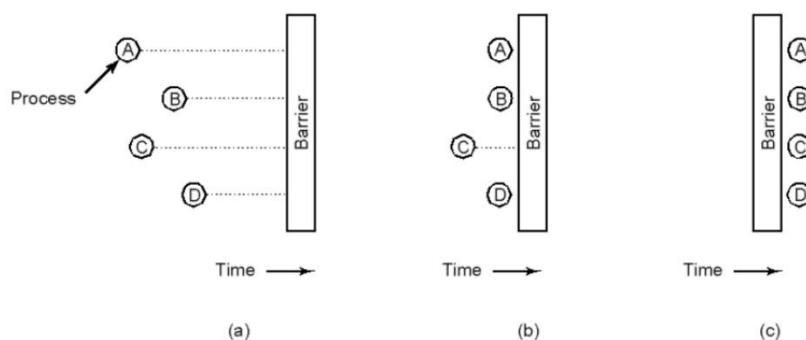
`public void clear()`

Automatically removes all of the elements from this queue. The queue will be empty after this call returns.

```
final BlockingQueue <E> b_queue = new ArrayBlockingQueue<E>(capacity);
```

## BARRIERA

Quando bisogna effettuare delle operazioni principalmente di calcolo, (per esempio vogliamo effettuare la somma degli elementi di un array di lunghezza N), potremmo decidere di voler velocizzare quest'operazione, una soluzione potrebbe essere quella di prendere il nostro array di lunghezza N, e suddividerlo in parti uguali, queste parti per esempio possono essere il numero di core del nostro pc, quindi  $N/\text{core del PC}$ , e creare appunto tanti thread quanti sono i core del pc, ciascuno dei quali si occuperà di sommare una sola porzione di array, e soltanto quando tutti i thread avranno terminato si metteranno insieme i risultati dei vari worker (per noi i nostri thread) e otterremo così il risultato finale, la barriera serve appunto per determinare "quando tutti i thread hanno terminato", infatti la barriera è un meccanismo di sincronizzazione che forza tutti i thread ad aspettare finché tutti non raggiungono un determinato punto.



Nell'esempio dei numeri Primi, la barriera ricorre in nostro aiuto, fin tanto che qualcuno sta calcolando resta tutto in attesa (**barriera.await()**), e soltanto quando ogni worker avrà detto alla barriera di aver finito le sue operazioni di calcolo, (**barriera.done()**), si potranno prendere i risultati, **for (Worker w), sum += w.takeresult()**.

*Ovviamente è di fondamentale importanza che i worker non siano soggetti al lockaggio delle risorse, poiché loro devono occuparsi di effettuare i calcoli, e se il nostro intento è quello di finire prima, dobbiamo garantire un parallelismo di calcolo tra i vari worker, la sequenzialità sarà introdotta soltanto quando verrà richiamata la barriera.*

**Barriera fatta dal Prof Ianni:**

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class Barriera {

    Lock lock = new ReentrantLock();
    Condition condition = lock.newCondition();
    private int sogliaTotale;
    private int workerTerminati = 0;

    public Barriera(final int soglia) {
        this.sogliaTotale = soglia;
    }
}
```

```

    public void done() {
        lock.lock();
        workerTerminati++;

        if (workerTerminati == sogliaTotale)
            condition.signalAll();

        lock.unlock();
    }

    public void await() {
        lock.lock();
        while (workerFiniti < sogliaTotale)
            try {
                condition.await();
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        lock.unlock();
    }
}

```

Il metodo `done()`, verrà chiamato dai worker soltanto quando si è terminato il processo di calcolo. Il metodo `await()`, invece verrà chiamato dall'oggetto che attende che tutti i worker terminano le loro operazioni per poter successivamente raccogliere i risultati.

Se non volessimo usare la Tipologia di Barriera fatta dal Prof, possiamo usare la `CyclicBarrier` di java, la quale è provvista del solo metodo `await()`, e quando dichiariamo la soglia massima dovremmo passare la soglia massima più 1, l'1 sarebbe il solver che attende che tutti siano terminati il quale deve mettersi a dormire anche lui nell'attesa che tutti finiscano!

## Cyclic Barrier

| Constructor and Description                                                                                                                  |                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>CyclicBarrier</b> (int parties)                                                                                                           |                                                                                 |
| Creates a new <code>CyclicBarrier</code> that will trip when the given number of parties (threads) are waiting upon it, and does not perform |                                                                                 |
| Int                                                                                                                                          | <b>await</b> ()                                                                 |
|                                                                                                                                              | Waits until all <b>parties</b> have invoked <code>await</code> on this barrier. |

## ***DEADLOCK***

Il Deadlock si verifica quando due o più Thread, sono bloccati nell'attesa di ottenere il lock per accedere a delle risorse che sono bloccate da altri Thread in Deadlock.

Per esempio se il Thread 1 ha il lock sulla risorsa A, e vuole prenderlo per la risorsa B, ed il Thread 2 ha il lock per la risorsa B, e vuole prenderlo per la risorsa A, in questo caso il Thread 1 non prenderà mai il lock per B e il Thread 2 mai per A, rimanendo bloccati per sempre perché nessuno dei due si sbloccherà, questa situazione è il Deadlock.

Thread 1 lock A, wait for B

Thread 2 lock B, wait for A.

Per prevenire il tutto, una soluzione potrebbe essere quella di ordinare il lockaggio delle risorse, ovvero se si individuano potenziali situazioni di Deadlock, come quella sopra descritta, dobbiamo far in modo che il thread 1 acquisisca contemporaneamente il lock per le risorse A e B, e il thread 2, deve attendere che termini il Thread A per ottenere le risorse.

Thread 1:

lock A, B

Thread 2:

wait A,B

Nell'esempio fatto dal Prof, relativo ai filosofi, quando un filosofo vuole mangiare deve prendere il controllo sia sulla bacchetta alla sua sinistra sia su quella alla sua destra contemporaneamente al fine di evitare situazioni di Deadlock!

# ***STARVATION***

Quando ad un Thread non viene concesso la CPU poiché altri thread la stanno occupando tutta e la sua esecuzione non avverrà mai, questa viene chiamata “starvation”, il Thread non farà la sua esecuzione proprio perché gli altri Thread non gli consentono di occupare la CPU. La soluzione alla starvation è quella di garantire meccanismi di equità.

Le principali cause di Starvation:

- I thread con una priorità alta occuperanno quasi sempre la CPU rispetto a thread con priorità bassa.
- Thread bloccati in tempo indefinito all'interno di un blocco synchronized, perché altri thread vi accedono costantemente prima di lui

Per garantire l'equità e quindi evitare il fenomeno della starvation una soluzione potrebbe essere quella di stabilire l'accesso ai blocchi critici solo per un determinato periodo o per un determinato numero di volte, raggiunto questo limite anche gli altri thread se in wait devono avere la possibilità di accedere alle risorse.

Vedi: Filosofi con Starvation.

## ***READ / WRITE LOCK***

I Read/ Write lock, sono più sofisticati rispetto ai lock semplici, se abbiamo un'applicazione che deve leggere e scrivere qualche risorsa, e che la scrittura non viene consentita fin tanto che qualcuno sta leggendo, se due o più Thread, leggono la stessa risorsa, non causano problemi agli altri e ne li causano tra di loro, in questo modo più Thread potranno leggere la risorsa contemporaneamente! Ma se abbiamo un singolo Thread che vuole scrivere la risorsa, nessun altro thread lettore o scrittore che esso sia deve risultare in esecuzione nel momento in cui voglio scrivere la risorsa.

Inizialmente bisogna definire quando garantire l'accesso in lettura o in scrittura.

Read Access: Soltanto quando non ci sono scrittori e quando nessun thread ha fatto richiesta per scrivere

Write Access: Soltanto quando non ci sono né lettori o scrittori.

Implementazione di lanni:

- Tanti lettori
- Un solo scrittore

Possono leggere più Thread contemporaneamente, ma lo scrittore sarà unico.

```
import java.util.concurrent.locks.Condition;  
import java.util.concurrent.locks.Lock;  
import java.util.concurrent.locks.ReentrantLock;  
  
public class DatoCondiviso {  
    private int dato;  
    private boolean ciSonoScrittori = false;  
    private int numLettori = 0;  
    private Lock lock = new ReentrantLock();  
    private Condition condition = lock.newCondition();  
  
    public int getDato() {  
        return dato;  
    }  
  
    public void setDato(int datoNuovo) {  
        this.dato = datoNuovo;  
    }  
  
    public void takeWriteLock() throws InterruptedException {  
        lock.lock();  
        while (ciSonoScrittori || numLettori > 0)  
            condition.await();  
        ciSonoScrittori = true;  
        lock.unlock();  
    }
```

```

    }

    public void leaveWriteLock() {
        lock.lock();
        ciSonoScrittori = false;
        condition.signalAll();
        lock.unlock();
    }

    public void takeReadLock() throws InterruptedException {
        lock.lock();
        while (ciSonoScrittori)
            condition.await();
        numLettori++;
        lock.unlock();
    }

    public void leaveReadLock() {
        lock.lock();
        numLettori--;
        condition.signalAll();
        lock.unlock();
    }
}

```

Il Read Write Lock ha due metodi di lock e due di unlock, un lock e unlock per l'accesso e il rilascio della risorsa in lettura e un lock e unlock per la modifica e il rilascio della risorsa in scrittura.

In questo modo però possono verificarsi casi di Starvation, ovvero se abbiamo tantissimi lettori e un solo scrittore, potrebbe capitare che lo scrittore non acceda mai alla risorsa e non riesca mai a modificarla.

Una soluzione potrebbe essere quella di rendere il lock FAIR.

Un'altra alternativa è quella di utilizzare dei contatori ausiliari per conteggiare gli accessi consecutivi dei lettori o di limitare gli accessi simultanei dei lettori o contare gli scrittori in attesa.

```

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class DatoCondivisoSenzaStarvation {

    private final int SOGLIA = 5;

    private int dato;
    private boolean ciSonoScrittori = false;
    private int numLettori = 0;
    private int numGiriSenzaScrittori = 0;
    private int numScrittoriInAttesa = 0;

    private Lock lock = new ReentrantLock();
    private Condition condition = lock.newCondition();
}

```

```

public int getDato() {
    return dato;
}

public void setDato(int datoNuovo) {
    this.dato = datoNuovo;
}

public void takeWriteLock() throws InterruptedException {
    lock.lock();
    numScrittoriInAttesa++;
    while (ciSonoScrittori || numLettori > 0)
        condition.await();
    ciSonoScrittori = true;
    numGiriSenzaScrittori = 0;
    numScrittoriInAttesa--;
    lock.unlock();
}

public void leaveWriteLock() {
    lock.lock();
    ciSonoScrittori = false;
    condition.signalAll();
    lock.unlock();
}

public void takeReadLock() throws InterruptedException {
    lock.lock();
    while (ciSonoScrittori || numGiriSenzaScrittori > SOGLIA || numScrittoriInAttesa > 0)
        condition.await();
    numLettori++;
    numGiriSenzaScrittori++;
    lock.unlock();
}

public void leaveReadLock() {
    lock.lock();
    numLettori--;
    condition.signalAll();
    lock.unlock();
}

```

Se non volessimo utilizzare l'implementazione del Prof.Ianni,  
 Quella di Java è così definita dall'interfaccia ReentrantReadWriteLock e dall'implementazione  
 ReentrantReadWriteLock,

| Constructor and Description                                                                                          |
|----------------------------------------------------------------------------------------------------------------------|
| <b>ReentrantReadWriteLock ()</b><br>Creates a new ReentrantReadWriteLock with default (nonfair) ordering properties. |
| <b>ReentrantReadWriteLock (boolean fair)</b><br>Creates a new ReentrantReadWriteLock with the given fairness policy. |



*final ReentrantReadWriteLock rwl = new ReentrantReadWriteLock();*

successivamente bisognerà dichiarare il lock in lettura e il lock in scrittura:

```
private final Lock r = rwl.readLock();  
private final Lock w = rwl.writeLock();
```

dopodichè essendo gestito automaticamente al suo interno come abbiamo precedentemente visto:

```
public Data get(String key) {  
    r.lock();  
    try { return m.get(key); }  
    finally { r.unlock(); }  
}  
public void put( Data value) {  
    w.lock();  
    put(value);  
    w.unlock();  
}
```

Possiamo tranquillamente utilizzare il lock sia in lettura che in scrittura.

# SHELL DI LINUX

## USEFUL COMMAND

**exit**: close Terminal

**ls**: Listing of directory and files of working directory

***pwd (print working directory)***: print output the full path of working directory

Every bash command is combined with options

***ls -l***:

***ls -l -h***

***ls -l -h -s*** is similar to write ***ls -lhs***

- Only one – are the SHORT OPTION
- More - are LONG OPTION

### LS OPTION:

- **-a** - lists all contents, including hidden files and directories
- **-l** - lists all contents of a directory in long format

```
$ ls -l
drwxr-xr-x 5 cc eng 4096 Jun 24 16:51 action
drwxr-xr-x 4 cc eng 4096 Jun 24 16:51 comedy
drwxr-xr-x 6 cc eng 4096 Jun 24 16:51 drama
-rw-r--r-- 1 cc eng 0 Jun 24 16:51 genres.txt
```

The **-l** option lists files and directories as a table. Here there are four rows, with seven columns separated by spaces. Here's what each column means:

1. Access rights. These are actions that are permitted on a file or directory.
2. Number of hard links. This number counts the number of child directories and files. This number includes the parent directory link (..) and current directory link (.).
3. The username of the file's owner. Here the username is **cc**.
4. The name of the group that owns the file. Here the group name is **eng**.
5. The size of the file in bytes.
6. The date & time that the file was last modified.
7. The name of the file or directory.

- **-t** - order files and directories by the time they were last modified.

## **DOCUMENTAZIONE DEI COMANDI**

**ls - - help**: offer summary explanation of ls command

**man ls**: offer detailed explanation of ls command

inside *man* we can find a command typing: **/command**

**q**: to exit man

**find**

**find - type d**

We can stop every command execution typing **CTRL+C**

**In linux TUTTO è UN FILE, driver, directory, eseguibili, ecc....**

**ls -l /proc/net**: i'm specifying to run ls command on directory proc/net

**PATH ASSOLUTO**: parto dalla radice es. /home/ianni

**PATH RELATIVO**: partendo dalla cartella corrente es: ../ianni)

**tail** fornisce la parte finale dei file in ingresso.

Se non viene specificato altrimenti l'ingresso viene considerato semplice testo e ne vengono date le ultime 10 righe.

**\$tail-n filename**(mostra le ultime n linee di filename. Se n non è specificato, di default è 10).

wc

Fornisce il "numero di parole (word count)" presenti in un file o in un flusso I/O

**\$wc file**

wc:opzioni

-w fornisce solo il numero delle parole.

-l fornisce solo il numero di righe.

### **Altri Comandi:**

- **cd (change directory)**: change working directory
- **cat**: show on terminal content file, or it is used to concat more file. Eg: cat file1 file2 concat, file1 and file2 then show the result
- **cp**: copy file/directory.
  - o cp must be received to the first argument the source file to copy, and at the second argument the destination directory or file, where the source file must be copy. Follow an

example, to copy one single file, and to copy two or more file, the first n argument are the source files, and the last argument is always the destination file/directory.

- `cp biopic/cleopatra.txt historical/`

To copy a file into a directory, use `cp` with the source file as the first argument and the destination directory as the second argument. Here, we copy the file **biopic/cleopatra.txt** and place it in the **historical/** directory.

- `cp biopic/ray.txt biopic/notorious.txt historical/`

To copy multiple files into a directory, use `cp` with a list of source files as the first arguments, and the destination directory as the last argument. Here, we copy the files **biopic/ray.txt** and **biopic/notorious.txt** into the **historical/** directory.

- **mv**: The mv command, moves file or directory, it is similar to cp command in its usage. The mv command is also used to rename file. The principal use, it's to move file or directory, to do this, we can pass to the first argument the file/directory we want to move, and at the second argument the destination directory. Like cp command, mv command can move more file or directory, in fact the first n argument are the directory or file we want to move and the last argument is the destination directory. Mv is used to rename file, at the first argument there is the file we want to rename, and at the second argument the new name and extension for the source file.

- `mv superman.txt superhero/`

To move a file into a directory, use `mv` with the source file as the first argument and the destination directory as the second argument. Here we move **superman.txt** into **superhero/**.

- `mv wonderwoman.txt batman.txt superhero/`

To move multiple files into a directory, use `mv` with a list of source files as the first arguments, and the destination directory as the last argument. Here, we move **wonderwoman.txt** and **batman.txt** into **superhero/**.

- `mv batman.txt spiderman.txt`

To rename a file, use `mv` with the old file as the first argument and the new file as the second argument. By moving **batman.txt** into **spiderman.txt**, we rename the file as **spiderman.txt**.

- **rm**: delete file/directory

- `rm waterboy.txt`

The `rm` command deletes files and directories. Here we remove the file **waterboy.txt** from the filesystem.

- `rm -r comedy`

The `-r` is an option that modifies the behavior of the `rm` command. The `-r` stands for "recursive," and it's used to delete a directory and all of its child directories.

Be careful when you use `rm`! It deletes files and directories permanently. There isn't an undelete command, so once you delete a file or directory with `rm`, it's gone.

- **less**: visualizza file di una pagina scaricata in formato testuale

- **file**: analisi file
- **mkdir (make directory)**: create new directory, with `-r` options we can create a tree-directory structure
- **touch**: to create an empty file

In Linux per i percorsi utilizziamo lo slash /

In Windows per i percorsi utilizziamo il back-slash \

### CANALI DI OUTPUT

1. Linea di comando (@argv, args[])
2. Standard Input(stdin, cin)
3. Standard output (stdout, cout)
4. Standard error (cerr)

1,2 sono canali di input, mentre 3,4 canali di output

```
$ sort lakes.txt
```

`sort` takes the standard input and orders it alphabetically for the standard output. Here, the lakes in `sort lakes.txt` are listed in alphabetical order.

```
$ uniq deserts.txt
```

`uniq` stands for "unique" and filters out adjacent, duplicate lines in a file

```
$ sort deserts.txt | uniq
```

A more effective way to call `uniq` is to call `sort` to alphabetize a file, and "pipe" the standard output to `uniq`.

### OPERATORI DI REDIREZIONE

- **>**: redirects standard output of a command to a file, overwriting previous content.
- **>>**: redirects standard output of a command to a file, appending new content to old content.
- **<**: redirects standard input to a command.
- **2>**: Redirects Standar error output of a command to a file
- **|**: Redirects standard output of a command to another command.

es: `ls /ianni/ r> prova.txt`: redirect standard error on file prova.txt

***find /var/log > listafiles.txt r> errorfind.txt*** redirect find result on listafiles.txt and then redirect standard error of listafiles.txt on new file errorfind.txt

## REDIREZIONE E PIPING

```
$ echo "Hello" Hello
```

The `echo` command accepts the string "Hello" as *standard input*, and echoes the string "Hello" back to the terminal as *standard output*.

Let's learn more about standard input, standard output, and standard error:

1. *standard input*, abbreviated as `stdin`, is information inputted into the terminal through the keyboard or input device.
2. *standard output*, abbreviated as `stdout`, is the information outputted after a process is run.
3. *standard error*, abbreviated as `stderr`, is an error message outputted by a failed process.

How does redirection work?

```
$ echo "Hello" > hello.txt
```

The `>` command redirects the standard output to a file. Here, "Hello" is entered as the standard input. The standard output "Hello" is redirected by `>` to the file **hello.txt**.

```
$ cat hello.txt
```

The `cat` command outputs the contents of a file to the terminal. When you type `cat hello.txt`, the contents of **hello.txt** are displayed.

```
$ cat oceans.txt > continents.txt
```

`>` takes the standard output of the command on the left, and redirects it to the file on the right. Here the standard output of `cat oceans.txt` is redirected to **continents.txt**.

Note that `>` overwrites all original content in **continents.txt**. When you view the output data by typing `cat` on **continents.txt**, you will see only the contents of **oceans.txt**.

If we don't overwrite the content, we can use `>>`:

```
$ cat glaciers.txt >> rivers.txt
```

`>>` takes the standard output of the command on the left and *appends* (adds) it to the file on the right. You can view the output data of the file with `cat` and the filename.

Here, the the output data of **rivers.txt** will contain the original contents of **rivers.txt** with the content of **glaciers.txt** appended to it.

```
$ cat < lakes.txt
```

`<` takes the standard input from the file on the right and inputs it into the program on the left. Here, **lakes.txt** is the standard input for the **cat** command. The standard output appears in the terminal.

**Pipe "I", It allows to run chained command, in italiano detto tubo.**

```
$ cat volcanoes.txt | wc
```

`|` is a "pipe". The `|` takes the standard output of the command on the left, and *pipes* it as standard input to the command on the right. You can think of this as "command to command" redirection. Here the output of **cat volcanoes.txt** is the standard input of **wc**. In turn, the **wc** command outputs the number of lines, words, and characters in **volcanoes.txt**, respectively.

```
$ cat volcanoes.txt | wc | cat > islands.txt
```

Multiple `|`s can be chained together. Here the standard output of **cat volcanoes.txt** is "piped" to the **wc** command. The standard output of **wc** is then "piped" to **cat**. Finally, the standard output of **cat** is redirected to **islands.txt**.

Es: **cat/etc/services | sort**, mostra il contenuto di **etc/services** dopo averlo ordinato

**grep**: regular expression di Bash, (consente il filtraggio dei file di testo)

Es. **grep/ianni/etc/passwd**

Vedere tutti gli utenti che usano Bash: **grep bash < etc/passwd**

Vedere tutti gli utenti che non usano Bash: **grep -v bash < etc/passwd**

**grep** stands for "global regular expression print". It searches files for lines that match a pattern and returns the results

```
$ grep Mount mountains.txt
```

It is also case sensitive. Here, **grep** searches for "Mount" in **mountains.txt**.

```
$ grep -i Mount mountains.txt
```

**grep -i** enables the command to be case insensitive. Here, **grep** searches for capital or lowercase strings that match **Mount** in **mountains.txt**.

The above commands are a great way to get started with **grep**. If you are familiar with regular expressions, you can use regular expressions to search for patterns in files.

```
$ sed 's/snow/rain/' forests.txt
```

**sed** stands for "stream editor". It accepts standard input and modifies it based on an *expression*, before displaying it as output data. It is similar to "find and replace".

Let's look at the expression **'s/snow/rain/'**:

- **s**: stands for "substitution". it is *always* used when using **sed** for substitution.
- **snow**: the search string, the text to find.
- **rain**: the replacement string, the text to add in place.

In this case, `sed` searches **forests.txt** for the word "snow" and replaces it with "rain." Importantly, the above command will only replace the first instance of "snow" on a line.

```
$ sed 's/snow/rain/g' forests.txt
```

The above command uses the `g` expression, meaning "global". Here `sed` searches **forests.txt** for the word "snow" and replaces it with "rain", *globally*. All instances of "snow" on a line will be turned to "rain".

A number of other commands are powerful when combined with redirection commands:

- `sort`: sorts lines of text alphabetically.
- `uniq`: filters duplicate, adjacent lines of text.
- `grep`: searches for a text pattern and outputs it.
- `sed`: searches for a text pattern, modifies it, and outputs it.

Sulla console -> Java CommandLine parola-0 [argv[0]] parola-1[argv[1]]

- parola-0
- parola-1 //vengono entrambe stampate a video dopo il comando

Command Line Options di Eclipse

- Run Configuration ed imposto args come canale di input

### **APPROFONDIMENTO SULLE PIPE**

- **ls | sort**: ordina lessicograficamente il risultato di ls
- **ls | head**: stampa per prime 10 righe di ls
- **ls | sort -r | head -n 5**: stampa le prime 5 righe di ls ordinate lessicograficamente in ordine inverso

### **CARATTERI JOLLY**

- **\***: zero o più caratteri qualsiasi
- **?**: 1 carattere qualsiasi
- **[]**: range di caratteri
- **!**: negazione:

Per quanto riguarda questi caratteri jolly prima dell'esecuzione di un programma, c'è un preprocessamento e poi i caratteri vengono "esposti" e solo successivamente si esegue, il programmatore non si deve preoccupare di questo in ambiente LINUX al contrario invece su ambiente WINDOWS.



## VARIABILI D'AMBIENTE

**env**

The **env** command stands for "environment", and returns a list of the environment variables for the current user. Here, the **env** command returns a number of variables, including **PATH**, **PWD**, **PS1**, and **HOME**.

**env | grep PATH**

**env | grep PATH** is a command that displays the value of a single environment variable. Here the standard output of **env** is "piped" to the **grep** command. **grep** searches for the value of the variable **PATH** and outputs it to the terminal

- **env**: lista delle variabili d'ambiente

**env | grep PATH**: quando viene eseguito un comando da terminale viene cercato tra le cartelle **usr**, se lo trova viene eseguito, altrimenti no.

**Echo \$ PATH**: stampa il contenuto delle variabili d'ambiente **PATH**

## CAMBIARE I PERMESSI DI ACCESSO AD UN FILE

Diamo uno sguardo più da vicino al contenuto di una directory, per esempio, scrivendo il comando **ls -l** (la "l" sta per "long".)

total 42

|            |   |     |       |       |              |                |
|------------|---|-----|-------|-------|--------------|----------------|
| -rwxr-xr-x | 1 | joe | acctg | 23068 | Feb 26 2004  | archive.sh     |
| -rw-rw-r-- | 1 | joe | acctg | 12878 | Jul 24 21:58 | orgchart.gif   |
| -rw-rw-r-- | 1 | joe | acctg | 2645  | Jun 30 08:48 | personnel.txt  |
| -rw-r--r-- | 1 | joe | acctg | 168   | Jul 17 11:51 | publicity.html |
| drwxrwxr-x | 2 | joe | acctg | 1024  | Mar 18 16:27 | sales          |
| -rw-r----- | 1 | joe | acctg | 512   | Sep 1 07:00  | topsecret.inf  |
| -rwxr-xr-x | 1 | joe | acctg | 2645  | Aug 4 11:03  | wordmatic      |
| 1          | 2 | 3   | 4     | 5     | 6            |                |

La riga **total** dice quanti blocchi (di solito 1024 byte per blocco) sono contenuti in questa directory.

- 1- Il primo carattere se è un trattatino indica un file altrimenti se è uguale a **d**, indica una directory, gli altri valori sono i permessi del proprietario, gruppo, others
- 2- Indica il numero di collegamenti di quel file/directory
- 3- Indica il proprietario del file e il gruppo a cui appartiene
- 4- Dimensione in byte

- 5- Data ...
- 6- Nome del file/directory

## Permessi del proprietario

Diamo uno sguardo alla prima colonna. La prima lettera indica se si tratta di un normale file o di una directory. La lettera successiva comunica al sistema che tipo di accesso è **permesso** su questo file.

Nella lista sotto, si è eliminata qualche colonna e si è aggiunto un po' di spazio alla colonna dei permessi per renderla più leggibile.

```
- rwx r-  
xr x joe acctg archive.sh  
- rw- rw-r-  
- joe acctg orgchart.gif  
- rw- rw-r-  
- joe acctg personnel.txt  
- rw- r--r-  
- joe acctg publicity.html  
d rwx r-xr-  
x joe acctg sales  
- rw- r----  
- joe acctg topsecret.inf  
- rwx r-xr-  
x joe acctg wordmatic
```

Il primo gruppo di tre lettere, dopo il tipo di file, indica quali permessi possiede il proprietario del file.

Una **r** nella prima posizione significa che si ha il permesso per leggere (**read**) il file. Una **w** nella seconda posizione significa che si ha il permesso di scrivere (**write**) nel file. Questo include la capacità di poter cancellare il file. Una **x** nella terza posizione significa che si ha il permesso per eseguire (**execute**) il file.

Un trattino in una posizione qualsiasi significa che non si ha quel particolare permesso.

Come si può vedere sopra, joe, l'utente proprietario dell'archivio, può leggere e scrivere tutti i file. Inoltre può anche eseguire lo script shell archive.sh ed il programma wordmatic.

Ma che cosa significa la x sulla directory sales? Una **x** in corrispondenza di una directory assume il significato speciale di "permesso di ispezionare questa directory".

## Permessi del gruppo

Le tre lettere successive a quelle dei permessi del proprietario indicano i permessi del gruppo.

```
-rw x r-x r-x joe acctg archive.sh  
-rw rw- r-- joe acctg orgchart.gif  
-rw rw- r-- joe acctg personnel.txt  
-rw r-- r-- joe acctg publicity.html  
drwx rwx r-x joe acctg sales  
-rw r-- --- joe acctg topsecret.inf
```

```
-rwx r-x r-x joe acctg wordmatic
```

### Permessi degli altri(**other**)

Le ultime tre lettere nella colonna dei permessi indicano ciò che un qualsiasi altro utente, che non sia il proprietario o che non appartenga al gruppo indicato, può fare.

```
-rwxr-x r-x joe acctg archive.sh  
-rw-rw- r-- joe acctg orgchart.gif  
-rw-rw- r-- joe acctg personnel.txt  
-rw-r-- r-- joe acctg publicity.html  
drwxrwx r-x joe acctg sales  
-rw-r-- --- joe acctg topsecret.inf  
-rwxr-x r-x joe acctg wordmatic
```

Gli "altri" in genere hanno condizioni molto ristrette, ad esempio non possono scrivere su nessun file o directory;

### Il comando **chmod**

Il comando chmod si utilizza per cambiare (**change**) il modo (**mode**) di accesso ad un file.

```
chmod who=permissions filename
```

“**who**” specifica i **permessi** per un file di nome **filename**.

“who” è una lista di caratteri che specifica "a chi" dare i permessi sul file. I caratteri possono essere specificati in un ordine qualsiasi.

| Lettera | Significato                                  |
|---------|----------------------------------------------|
| u       | L'utente proprietario del file (ossia "voi") |
| g       | Il gruppo al quale il file appartiene.       |
| o       | Gli altri ( <b>other</b> ) utenti            |
| a       | tutti ( <b>all</b> ) le opzioni precedenti   |

Permessi

Naturalmente, i permessi possono essere quelli elencati di seguito:

|   |                                                                |
|---|----------------------------------------------------------------|
| r | Permesso per leggere ( <b>read</b> ) il file.                  |
| w | Permesso per scrivere ( <b>write</b> ) (o cancellare) il file. |

|   |                                                                                                  |
|---|--------------------------------------------------------------------------------------------------|
| x | Permesso per eseguire ( <b>execute</b> ) il file, o, nel caso della directory, per ispezionarla. |
|---|--------------------------------------------------------------------------------------------------|

Nota: Non inserire spazi vuoti vicino al segno di uguaglianza, altrimenti il comando risulta errato!

## Esempi su chmod

Proviamo a cambiare alcuni dei permessi discussi nelle pagine precedenti. Ecco quali sono inizialmente i permessi dei file:

```
-rwxr-xr-x joe acctg archive.sh
-rw-rw-r-- joe acctg orgchart.gif
-rw-rw-r-- joe acctg personnel.txt
-rw-r--r-- joe acctg publicity.html
drwxrwxr-x joe acctg sales
-rw-r----- joe acctg topsecret.inf
-rwxr-xr-x joe acctg wordmatic
```

Impediamo agli estranei di eseguire archive.sh

|          |                       |
|----------|-----------------------|
| Prima:   | -rwxr-xr-x archive.sh |
| Comando: | chmod o=r archive.sh  |
| Dopo:    | -rwxr-xr-- archive.sh |

Togliamo tutti i permessi per il gruppo sul file topsecret.inf. Otteniamo ciò lasciando vuota la parte del comando che riguarda i permessi.

|          |                                    |
|----------|------------------------------------|
| Prima:   | -rw-r----- topsecret.inf           |
| Comando: | chmod g= topsecret.inf             |
| Dopo:    | -rw- <del>----</del> topsecret.inf |

Rendiamo aperto il file publicity.html cosicchè possa essere letto e scritto da chiunque.

|          |                                   |
|----------|-----------------------------------|
| Prima:   | -rw-r--r-- publicity.html         |
| Comando: | chmod og=rw publicity.html        |
| Dopo:    | -rw- <b>rw-rw-</b> publicity.html |

## ***GESTIONE PROCESSI***

### **Linux ps command**

The ps command on linux is one of the most basic commands for viewing the processes running on the system. It provides a snapshot of the current processes along with detailed information like user id, cpu usage, memory usage, command name etc. It does not display data in real time like top or htop commands. But even though being simpler in features and output it is still an essential process management/monitoring tool that every linux newbie should know about and learn well.

```
ps aux | less
```

An alternative set of options for viewing all the processes running on a system is

```
ps -ef | less
```

#### **GESTIONE PROCESSI E JOB**

**ps:** lista tutti i processi correnti agganciati sullo standard output della console in cui mi trovo. Aggiungendo come opzione di ps **-ax**, ci verranno mostrati tutti i processi del sistema, indipendentemente dalla console in cui mi trovo, ci restituirà quindi un listato completo così composto:

| PID | TTY | STAT | TIME | COMMAND |
|-----|-----|------|------|---------|
|-----|-----|------|------|---------|

|                                         |                                                                                                                                                            |  |                                             |                                                                                                                            |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Indica il PID associato a quel processo | Indica se il processo è in running oppure è fermo, i valori che possiamo trovare sono: <b>s</b> indica che il processo è fermo, oppure <b>+</b> in running |  | Indica da quanto tempo il processo è attivo | Indica il comando che si sta eseguendo, se è racchiuso tra parentesi [] significa che si tratta di un processo di sistema. |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|

**-top:** altro comando utilizzato per visionare i processi, in questo caso lanciando top sulla console uscirà in output una sorta di task manager in formato rigorosamente testuale.

Per ogni processo che a noi interessa è di fondamentale importanza conoscerne il suo pid!

**-kill:** comando utilizzato per killare i processi, viene utilizzato particolarmente in due modi principali:

**-kill PIDPROCESS:** lanciando kill in questo modo il processo viene “gentilmente” invitato a terminare, la sua funzionalità è quella di mandare un segnale al fine di far terminare il processo da solo.

**-kill -9 PIDPROCESS:** In questo modo kill, chiude brutalmente il processo.

Se sulla bash volessimo lanciare un processo, abbiamo due modi per farlo.

- 1) **PROCESSNAME:** in questo modo il processo verrà lanciato dalla bash in cui ci troviamo, dopo un po' si aprirà, tuttavia il controllo della bash lo avrà il processo, quindi non potremmo utilizzarla fin tanto che quel processo termini
- 2) **PROCESSNAME &:** in questo modo verrà comunque lanciato il processo in modo analogo ad 1, tuttavia non perderemo il controllo della bash e potremmo continuare ad utilizzarla normalmente.

Sia 1, che 2, una volta avviati ci daranno in output il pid associato a quel processo.

**-jobs:** Viene utilizzato per visionare i processi inizializzati all'interno della bash associando ad ogni processo un n°ro di jobs.

**-fg NRO\_JOBS:** Porta al primo posto il processo a cui è associato NRO\_JOBS, utilizzando questo comando non potrò utilizzare la shell fin tanto che il processo non viene correttamente chiuso.

## FILE SYSTEM

Un **FILE SYSTEM** (FS), informalmente è un meccanismo con il quale i file sono posizionati e organizzati su un dispositivo di archiviazione o su una memoria di massa, come un disco rigido o un CD-ROM.

Più formalmente, un file system è l'insieme dei tipi di dati astratti necessari per la memorizzazione (scrittura), l'organizzazione gerarchica, la manipolazione, la navigazione, l'accesso e la lettura dei dati.

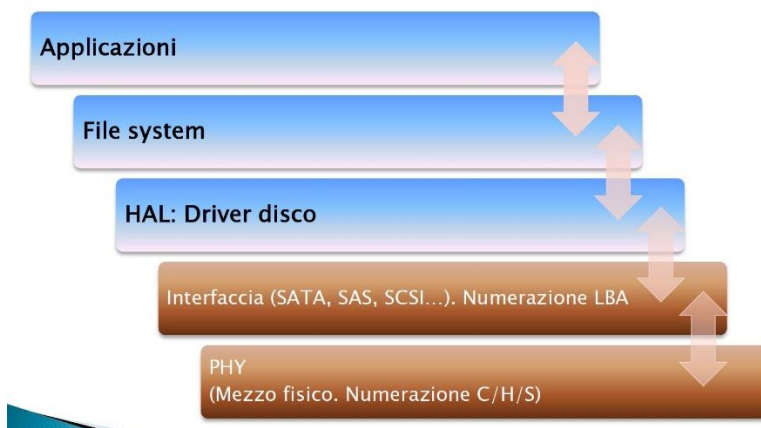
I file system possono essere rappresentati sia graficamente tramite file browser sia testualmente tramite shell. La rappresentazione grafica (GUI) mostra le directory e sottodirectory e/o file, secondo una struttura ad albero.

Il FS, organizza quindi i file e ne gestisce l'accesso ai dati, esso è responsabile:

- Gestione dei File: infatti deve prevedere dei meccanismi per consentirne il salvataggio, la condivisione, la messa in sicurezza(crittografia), ed il riferimento per l'apertura dello stesso.
- Gestione delle memorie ausiliari: infatti deve poter consentire la gestione e il salvataggio di contenuti su dispositivi secondari.
- Meccanismi di integrità dei file: Infatti nel momento del salvataggio è necessario che si occupi di salvare per quel file tutto il contenuto che effettivamente desideriamo vi sia al suo interno, senza inconsistenza o perdita di dati (transazionale).
- Metodi di Accesso: come accedere ai dati salvati.

È Stato concepito principalmente per l'accesso allo spazio della memoria secondaria, in modo particolare il disco fisso. Ha al suo interno diverse funzionalità utilizzate appunto per estrarre dal dato grezzo presente in memoria di massa il file o la directory che ci viene effettivamente fatto/a visualizzare.

***Il FS inoltre è l'unico in grado di consentire la comunicazione e l'interfacciamento tra le applicazioni e la memoria di massa.***



Prima di analizzare il modo in cui lavora un FS, parliamo rapidamente delle MM.

## **LA MEMORIA DI MASSA.**

Componente di fondamentale importanza di un elaboratore, le memorie di massa più utilizzate sono il Disco Meccanico, e le Memorie a Stato Solido.

### **- DISCO MECCANICO.**

È un dispositivo di memoria di massa di tipo magnetico che utilizza uno o più dischi magnetizzati per l'archiviazione dei dati (file, programmi e sistemi operativi).

Il disco rigido è costituito fondamentalmente da uno o più piatti in rapida rotazione, realizzati in alluminio o vetro, rivestiti di materiale ferromagnetico, e da due testine per ogni disco (una per lato), le quali, durante il funzionamento "volano" dalla superficie del disco leggendo o scrivendo i dati. La testina si dice che vola dalla superficie del disco poiché è tenuta sollevata dall'aria mossa dalla rotazione stessa dei dischi la cui frequenza o velocità di rotazione può superare i 15.000 giri al minuto. I valori standard di rotazione sono 4.200, 5.400, 7.200, 10.000 e 15.000 giri al minuto (RPM).

### **Organizzazione Logica dei Dati**

Tipicamente per la memorizzazione di dati digitali il disco rigido necessita dell'operazione preliminare di formattazione logica con scelta del particolare sistema logico di archiviazione dei dati da utilizzare noto appunto come File System, tramite il quale il sistema operativo è in grado di scrivere e recuperare i dati.

### **Organizzazione Fisica dei Dati**

I dati, a livello fisico, sono memorizzati su disco seguendo uno schema di allocazione fisica ben definito in base al quale si può raggiungere la zona dove leggere/scrivere i dati sul disco. Uno dei più diffusi è il cosiddetto *CHS*, sigla del termine inglese *Cylinder/Head/Sector* (Cilindro/Testina/Settore). In questa struttura i dati sono memorizzati avendo come indirizzo fisico un numero per ciascuna delle seguenti entità fisiche:

#### **Piatto**

Un disco rigido si compone di uno o più dischi paralleli, di cui ogni superficie, detta "piatto" e identificata da un numero univoco, è destinata alla memorizzazione dei dati.

#### **Traccia**

Ogni piatto si compone di numerosi anelli concentrici numerati, detti tracce, ciascuna identificata da un numero univoco.

#### **Cilindro**

L'insieme di tracce alla stessa distanza dal centro presenti su tutti i dischi o piatti è detto *cilindro*. Corrisponde a tutte le tracce aventi il medesimo numero, ma diverso piatto (Possiamo vederli come una sorta di HDD più interni).

#### **Settore**

Ogni piatto è suddiviso in settori circolari, ovvero in "spicchi" radiali uguali ciascuno identificato da un numero univoco. (Generalmente la sua grandezza è di 512byte)

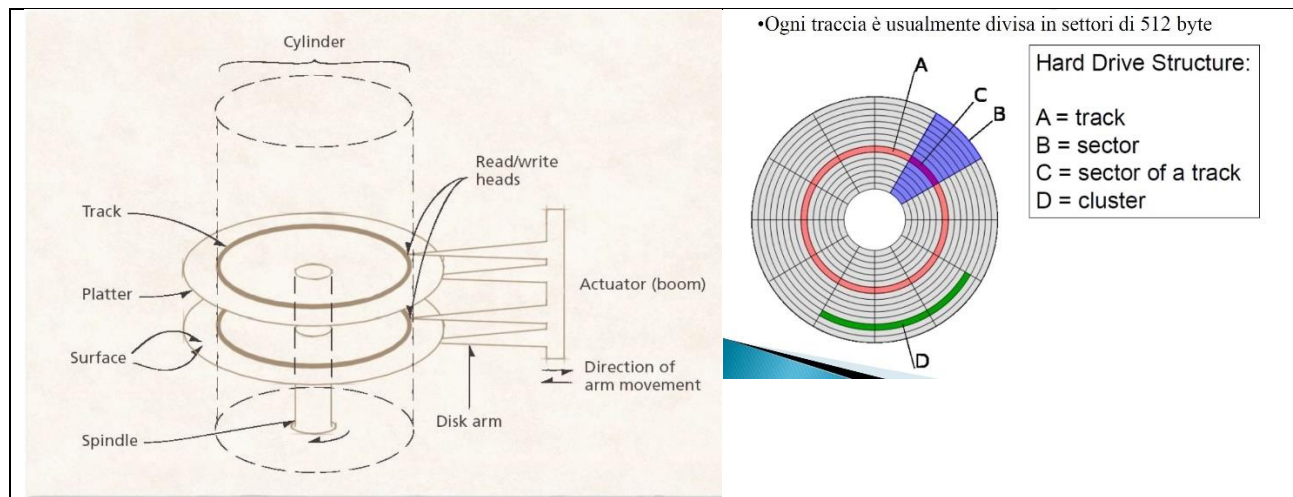


## Testina

Su ogni piatto è presente una testina per accedere in scrittura o in lettura ai dati memorizzati sul piatto. Se una testina è posizionata sopra una traccia, tutte le altre testine saranno posizionate nel cilindro a cui la traccia appartiene.

## Cluster

Insieme di frammenti contigui. (Settori di una determinata traccia)



I Dischi Meccanici tuttavia impiegano del tempo per accedere ai dati, dato appunto dai componenti meccanici del disco, quindi si impiega del tempo solitamente espresso in ms, per far posizionare la testina sulla traccia corretta, per accedere ai cluster del file, e per spedire i dati, infatti questi ritardi sono espressi da:

### ◦Seek time (in ms)

Tempo che ci mettono le testine ad arrivare su un dato cilindro (dipende da fattori costruttivi)

### ◦Latenza di rotazione (in ms)

Tempo che ci mettono i dati richiesti ad arrivare sotto la testina (in media  $1 / (\text{RPM} / 60 * 2)$ )

### ◦Tempo di trasmissione (in bit/sec)

Tempo che ci mettono tutti i dati richiesti ad essere letti =  $\text{RPM} / 60 * \text{bit per cilindro}$ .

I dischi meccanici possono durare moltissimo tempo, a patto che la testina non inizi a dare problemi, poiché se malauguratamente essa durante la rotazione dei piatti, vi dovesse strisciare sopra, e quindi non riuscire a “volarci” sopra, procurerebbe una perdita dei dati non più recuperabili dovuti alla “rigatura” del piattello provocato dalla testina. Questo fenomeno viene chiamato Head Crash



## - UNITA' A STATO SOLIDO

Un'unità a stato solido o disco a stato solido (SSD *solid-state disk*) è una tipologia di dispositivo di memoria di massa basata su semiconduttore, che utilizza memoria allo stato solido (in particolare memoria flash) per l'archiviazione dei dati, anziché supporti di tipo magnetico come nel caso dell'hard disk classico.

(Giusto una Lettura)

Le unità allo stato solido si basano quindi su memoria flash per l'immagazzinamento dei dati, ovvero modificano lo stato elettronico di celle di transistor. Per questo non richiedono parti meccaniche in movimento (dischi, motori e testine), né componenti magnetici, il che comporta notevoli vantaggi alla riduzione dei consumi elettrici e dell'usura.

Un'unità SSD dispone di diversi componenti necessari alla gestione del suo funzionamento.

### **Controller**

Il controller è costituito da un microprocessore che si occupa di coordinare tutte le operazioni della memoria di massa. Il software che governa questo componente è un firmware preinstallato dal produttore. Oltre alle operazioni di lettura/scrittura si occupa della gestione di:

- Error-correcting code: controllo e correzione degli errori in fase di lettura/scrittura
- Wear leveling: distribuzione della scrittura in maniera uniforme su tutto il drive
- Bad block: rilevamento e riallocazione trasparente con blocchi di riserva dei settori danneggiati
- Memoria cache: interna al dispositivo
- Garbage collection: rilevamento e riduzione automatica della frammentazione dell'organizzazione interna del disco
- Criptazione dei dati
- LBA (logical block address) scrambler: tecnica sperimentale che sfrutta le pagine dati frammentate per ridurre il numero di scritture e cancellazioni.

### **Memoria Cache**

La memoria Cache è una memoria, a seconda del livello a cui appartiene, solitamente di pochi MB, utilizzata dal processore per immagazzinare temporaneamente informazioni che verranno richieste in seguito dal sistema. Essa viene quindi riempita e svuotata molte volte.

### **Super-condensatore**

Una novità introdotta dalle memorie a stato solido è la possibilità di terminare le operazioni di scrittura anche in caso di mancanza di tensione. Questo avviene grazie alla presenza di un supercondensatore o, più raramente, di una batteria di backup, che garantisce energia sufficiente per concludere l'operazione in corso. Questa tecnica permette di garantire una maggiore integrità dei dati ed evitare che il *file system* risulti corrotto.

### **Interfaccia**

La connessione può avvenire con cavi di tipo SATA, sia per quanto riguarda la connessione dati che per l'alimentazione. In definitiva è possibile collegare un SSD utilizzando una normale interfaccia SATA2 (3Gb/s) o SATA3 (6Gb/s). Vi sono inoltre SSD che utilizzano l'interfaccia PCI Express; quest'ultima può arrivare fino a una velocità di trasferimento di 20Gb/s.

Gli SSD:

- **Soggetti a wearing.** Tecnica utilizzata per prolungare la vita degli SSD, infatti a differenza degli HDD hanno una capacità limitata in scrittura, entro il quale la vita dell'unità termina, la tecnica di wearing consiste nel tenere alcuni blocchi di memoria flash "in panchina" ovvero quando qualche blocco risulta inutilizzabile, essi vengono logicamente sostituiti da uno di quelli in panchina, per questo le case produttrici indicano le grandezze in termini di 120, 240 gb, proprio perché i restanti servono per prolungare la vita del unità.
- **Necessità di 'trimming':** Il TRIM permette a un sistema operativo di indicare i blocchi che non sono più in uso in un SSD (Negli HDD non esiste) come per esempio i blocchi liberati dopo l'eliminazione di uno o più file. Generalmente nell'operazione di cancellazione eseguita da un Sistema Operativo i blocchi data vengono contrassegnati come non in uso. Il TRIM permette all'OS di passare questa informazione al controller dell'SSD, che altrimenti non sarebbe in grado di sapere quali blocchi eliminare. Viene effettuata quest'operazione in modo preliminare poiché se gli SSD dovrebbero prima liberare lo spazio non in uso e poi riallocarlo per scrivervi risulterebbe un'operazione davvero dispendiosa a livello di tempo.
- **SeekTime Irrisorio:** Essendo memorie allo stato solido non avendo parti meccaniche, l'accesso ai dati risulta essere quasi 0 ms.

## ***FILE SYSTEM***

I File System sono composti da Codice e speciali strutture dati.

Le strutture dati sono contenute in settori della partizione non visibili direttamente dall'utente finale, possono essere tabelle contenenti i cluster liberi, e tabelle contenenti l'allocazione logica in memoria dei file.

Il codice invece, è una parte integrante del SO e si occupa appunto di gestire queste strutture dati e di fornire all'utente finale le funzionalità per la gestione dei dati.

**Partizione:** è un blocco di settori consecutivi, infatti ogni partizione è formattata con un certo FS a scelta dell'utente.

Ogni file su disco è identificato da un insieme di Cluster, a cui viene associato un nome contenente dei dati tra di loro correlati. Quando memorizziamo un file, al livello più basso stiamo scrivendo su disco bit a bit, questi bit sono raggruppati successivamente in byte, e quindi in settori di 512 byte ciascuno. Qualsiasi FS raggruppa i settori in cluster per esempio un cluster da 4K (4096 byte) -> 8 Settori, e come detto precedentemente quando il FS deve salvare un qualsiasi file sul disco ad esso associa almeno un cluster anche se la sua dimensione è di 10 byte. Inoltre se un file ha necessità di essere memorizzato su più cluster non è detto che essi siano tra di loro consecutivi.

### **Caratteristiche di un FS:**

- Sono totalmente indipendenti dal dispositivo, infatti a prescindere che si tratti di un HDD, DVD, USB, ecc..., la sua interfaccia sarà comune a tutti mentre i dettagli implementativi saranno nascosti
- I moderni FS, devono fornire delle funzionalità di backup e/o recovery per contrastare le possibili perdite di dati, infatti se sto scrivendo un File, e malauguratamente perdessi

l'alimentazione, potrebbe capitare che i cluster sono stati scritti ma le tabelle relative ai cluster liberi/occupati (BitMap, FAT, MFT) non sono state aggiornate quindi il file non risulta scritto.

- Possono fornire funzionalità di crittografia dei file e di compressione

## ***FAT (File Allocation Table)***

La File Allocation Table, in sigla FAT, è un file system sviluppato inizialmente da IBM e Digital

Il più grande problema del File System FAT della Microsoft è la frammentazione. Quando i file vengono eliminati, creati o spostati, le loro varie parti si disperdono sull'unità, rallentandone progressivamente la lettura e la scrittura.

Esistono varie versioni di questo File System, in base a quanti bit sono allocati per numerare i cluster del disco: FAT12, FAT16, FAT32.

### ***FAT32***

FAT32, con numeri per i cluster da 32 bit, anche se in realtà ne vengono utilizzati solo 28. In teoria questo dovrebbe permettere 268 435 456 ( $2^{28}$ ) cluster, cioè una dimensione totale dell'ordine dei 2 terabyte. Con la FAT32 la dimensione del singolo file non può essere superiore ai 4 GB. Questo perché esiste una voce a 32 bit che indica la grandezza del file in byte.

Il File System FAT è un File System classificato tra quelli con *allocazione concatenata*. Una partizione FAT è strutturata in quattro sezioni diverse:

|                              | Area riservata   |                                 |                          | FAT                       |        | Root directory<br>(solo FAT12/16)      | Regione dati                        |
|------------------------------|------------------|---------------------------------|--------------------------|---------------------------|--------|----------------------------------------|-------------------------------------|
|                              | Settore di avvio | Informazioni FS<br>(solo FAT32) | Riservati<br>(opzionale) | FAT #1                    | FAT #2 |                                        |                                     |
| <b>Dimensione in settori</b> | variabile        |                                 |                          | (# FAT)*(settori per FAT) |        | 32 * (# voci root) / bytes per settore | (# cluster) * (settori per cluster) |

### ***FAT SU DISCO (Visibile da WIN HEX)***

- I settori riservati, che si trovano proprio all'inizio. Il primo settore riservato (settore zero) è il settore di avvio, seguito dal BIOS (con alcune informazioni di base del FS, in particolare il suo tipo, e puntatori alla posizione delle altre sezioni). Contiene di solito il codice del boot loader del sistema operativo. La dimensione dei settori riservati è indicata in un campo all'interno del settore di avvio. Nel FAT32 le informazioni si trovano nel settore 1, mentre nel settore 6 vi è una copia di backup del settore di avvio.

- La Regione FAT: Contiene almeno due copie della FAT (per motivi di sicurezza). Rappresentano la mappa della regione dati.
- La Regione della ROOT directory: è una tabella che memorizza le cartelle e i file presenti nella directory di root. È presente solo nella FAT12 e nella FAT16 ed impone una dimensione massima prefissata per la root; nella FAT32 ciò è fatto direttamente nella regione dati, eliminando così il vincolo dimensionale sulla root.
- L'area dati: è dove file e cartelle sono realmente memorizzati e occupa la maggior parte della partizione

## File Allocation Table

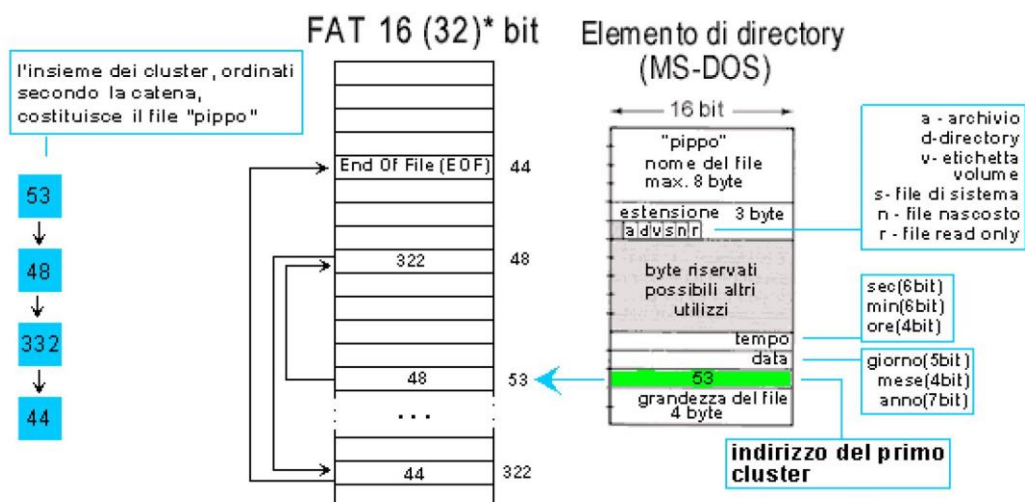
Una partizione è divisa in cluster contigui dalle dimensioni variabili tra 2 e 32 KB. Ogni file è strutturato sul disco come una lista concatenata di cluster non necessariamente contigui: questa è la ragione principale per cui si parla di frammentazione del disco nei filesystem FAT.

Ogni record della FAT è composto da 5 campi:

- Il numero del cluster successivo
- Una speciale EOC (end of clusterchain), che indica la fine di una catena
- Cluster danneggiato
- Cluster riservato
- Cluster libero

Analizziamo ora un esempio di come viene letto il file "Pippo" con un sistema FAT 16:

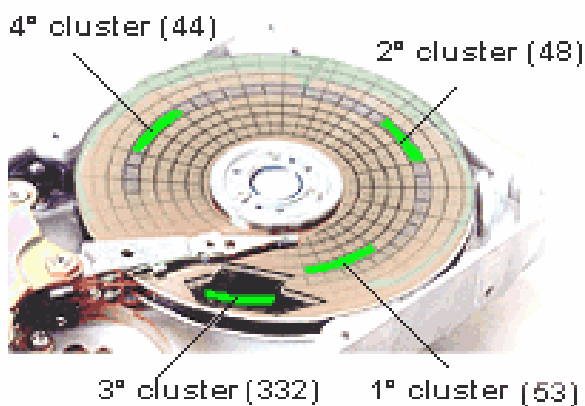
## FAT32



Il File "Pippo" in memoria avrà i suoi Metadati associati (Elemento di Directory), da questi metadati possiamo apprendere il nome del file, la sua estensione e molto altro, a un certo punto troveremo l'indirizzo del primo cluster, cosa si intende?

Nei metadati del file, quando troviamo questo valore, esso indica il primo cluster corrispondente all'inizio del file, quindi quando siamo in lettura, bisognerà accedere nella File Allocation Table (FAT) andare nella cella relativa al primo cluster in questo caso 53, leggere il contenuto del file e a un certo punto troveremo l'indirizzo del cluster successivo da cui continua il file, in questo caso 48, ripetiamo il procedimento fin tanto che siamo arrivati all'ultimo cluster, quando si è arrivati alla lettura dell'ultimo cluster, nella cella a lui corrispondente troveremo un valore speciale detto End Of File (EOF).

La FAT sarà memorizzata sul disco principale del CD e avrà anche una sua copia. Se un disco è formattato come FAT se lo apriamo con WIN HEX, troveremo la voce \$FAT che indica appunto la tabella dei cluster liberi ed occupati.



Come si vede il file è memorizzato su cluster non contigui, e la struttura della FAT ad allocazione concatenata è possibile proprio perché a partire dal primo cluster del file (da cui possiamo risalirci attraverso i metadati) riusciamo a comporre la gerarchia in coda di tutti gli altri.

## ***NTFS ed Ext4***

### **NTFS: è il FS standard di windows**

- Remapping Automatico dei Cluster Deboli
- Compressione
- Crittografia
- Diritti d'accesso sofisticati
- Transazionale (sempre consistente, ovvero in caso in cui salti l'alimentazione o si verificano altri problemi, nel momento in cui scriviamo sul disco, siamo sicuri che qualsiasi cosa di imprevedibile accada ciò che vi viene scritto è effettivamente ciò che ci serve)

### **NTFS su disco.**

Aperto Win Hex, se un disco è formattato come NTFS, troveremo diversi File che sono nascosti all'utente, tra questi:

**MFT:** La **Master File Table** è probabilmente la più importante delle chiavi che definiscono un volume NTFS. Il Master File Table o MFT è il luogo in cui sono registrate le informazioni su

ogni file e directory di un volume formattato come NTFS. Agisce come punto di partenza e funziona come il gestore centrale di un volume NTFS, una sorta di "tavola dei contenuti" per il volume. È analogo al File Allocation Table dei file in una partizione FAT, ma è molto più di un semplice elenco dei cluster usati e disponibili. La MFT è l'elemento principale di una partizione NTFS, (il nome esatto è "\$MFT"), e contiene, come abbiamo detto, l'elenco di tutti i file memorizzati su disco. Questo elenco viene memorizzato in forma di una serie di registrazioni, alla maniera di un database. Quando un file viene eliminato, il record che descrive è contrassegnato come libero, può quindi essere riutilizzato quando si crea un nuovo file, ma il record eliminato nella tabella non elimina a livello fisico il file su disco.

C'è un record nella MFT per ogni file sul disco, troviamo le seguenti informazioni:

- Nome lungo del file.
- Index (numero del file).
- Dimensioni del file.
- Data e ora di creazione / modifica / accesso.
- Gli attributi del file.
- I diritti di accesso (vedi Access Control List)
- Elenco dei blocchi (cluster) contenente il file.

La MFT è usata in coordinamento con il file \$Bitmap che contiene indicatori di occupazione di ogni blocco della partizione.

La MFT inoltre ha anche un altro file associato, rappresentante una copia di se stessa, nel file MFTmirr.

**BitMap: \$BitMap** è un file speciale contenuto all'interno di NTFS. Questo file tiene traccia di tutti cluster usati e non, all'interno del volume formattato con NTFS. Quando un file occupa spazio nel volume NTFS la posizione occupata (in termini di spazio non di coordinate) è segnata all'interno di questo file.

**BadClus:** File indicante i Cluster "Non funzionanti" solitamente questo file ha peso 0kb, se inizia a crescere vuol dire che il disco sta iniziando a rompersi.

### ***Ext4***

- Journalled (Transazionale)
- Ottima Politica di anti-frammentazione (un FS Ext4, ha una percentuale di frammentazione irrisoria anche dopo molti anni)

A differenza del NTFS che offre funzionalità quali crittografia, compressione, "di serie", con un FS Ext4, se volessimo queste funzionalità, esse devono esser Richieste.